Sistem Deteksi Kesiapan Panen Tanaman Cabai Rawit Menggunakan Ekstraksi Fitur Bag-of-Features Berbasis Warna HSV dan Spatial Pyramid

SKRIPSI

Diajukan sebagai persyaratan untuk memperoleh

gelar Sarjana Terapan Teknik

Disusun oleh:

MUHAMMAD NUR ASY'ARI

2241160107

PROGRAM STUDI JARINGAN TELEKOMUNIKASI DIGITAL

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI MALANG

2026

HALAMAN PENGESAHAN

Sistem Deteksi Kesiapan Panen Tanaman Cabai Rawit Menggunakan Ekstraksi Fitur Bag-of-Features Berbasis Warna HSV dan Spatial Pyramid

Oleh :

Muhammad Nur Asy'ari

2241160107

Skripsi ini telah dipertahankan di depan dewan penguji pada 14 Juli 2026, dan disahkan oleh

Pembimbing I :

Chandrasena Setiadi, S.T., M.Tr.T.

NIP. 198303142024211001

.....

Pembimbing II :

Aad Hariyadi, S.ST., M.T.

NIP. 196301041988031002

.....

Penguji I :

Adzikirani, S.S.T., M.Tr.T.

NIP. 199407262022031009

.....

Penguji II :

Ir. Nugroho Suharto, M.T.

NIP. 196204211989031001

.....

Malang, 6 Agustus 2026

Mengetahui,
Ketua Jurusan Teknik Elektro

Prof. Ir. Mohammad Noor Hidayat, S.T., M.Sc., Ph.D.
NIP. 197409252001121003

Menyetujui,
Koordinator Program Studi
Jaringan Telekomunikasi Digital

Mila Kusumawardani, S.T., M.T.
NIP. 197503012000032002

PERNYATAAN KEASLIAN SKRIPSI

Saya yang bertanda tangan dibawah ini:

Nama	:	Muhammad Nur Asy'ari
NIM	:	2241160107
Judul Skripsi	:	Sistem Deteksi Kesiapan Panen Tanaman Cabai Rawit Menggunakan Ekstraksi Fitur Bag-of-Features Berbasis Warna HSV dan Spatial Pyramid

Saya menyatakan dengan sebenar-benarnya bahwa skripsi ini merupakan hasil penelitian, pemikiran, dan pemaparan asli saya sendiri. Saya tidak mencantumkan tanpa pengakuan bahan - bahan yang telah dipublikasikan sebelumnya atau ditulis oleh orang lain, atau sebagai bahan yang pernah diajukan untuk gelar atau ijazah pada Politeknik Negeri Malang atau perguruan tinggi lainnya.

Apabila dikemudian hari terdapat penyimpangan dan ketidakbenaran dalam pernyataan ini, maka saya bersedia menerima sanksi akademik sesuai dengan peraturan yang berlaku di Politeknik Negeri Malang.

Demikian pernyataan ini saya buat.

Malang, 6 Agustus 2026

Mahasiswa,



Muhammad Nur Asy'ari

NIM. 2241160107

ABSTRAK

Muhammad Nur Asy'ari, 2026. "Sistem Deteksi Kesiapan Panen Tanaman Cabai Rawit Menggunakan Ekstraksi Fitur Bag-of-Features Berbasis Warna HSV dan Spatial Pyramid" Program Studi Jaringan Telekomunikasi Digital Jurusan Teknik Elektro, Politeknik Negeri Malang. Dosen Pembimbing I: Chandrasena Setiadi, S.T., M.Tr.T. Dosen Pembimbing II: Aad Hariyadi, S.ST., M.T.

Cabai rawit merupakan salah satu komoditas hortikultura yang memiliki nilai penting dalam sektor pertanian. Penentuan kesiapan panen cabai rawit umumnya masih dilakukan secara manual melalui pengamatan visual, sehingga hasil penilaian dapat berbeda karena dipengaruhi oleh pengalaman dan persepsi masing-masing pengamat. Oleh karena itu, penelitian ini bertujuan untuk merancang dan membangun sistem deteksi kesiapan panen tanaman cabai rawit berbasis citra digital menggunakan kombinasi metode *Bag-of-Features* (*BoF*), *HSV*, *Spatial Pyramid Matching* (*SPM*), *Principal Component Analysis* (*PCA*), dan *Support Vector Machine* (*SVM*). Sistem bekerja dengan mengolah citra tanaman cabai rawit berukuran 128 × 128 piksel. Citra diekstraksi menggunakan *BoF* untuk memperoleh pola lokal, *HSV* untuk merepresentasikan warna, dan *SPM* untuk mempertahankan informasi spasial. Fitur gabungan kemudian direduksi menggunakan *PCA* dan diklasifikasikan menggunakan *SVM* ke dalam empat kelas, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Sistem ini diimplementasikan dalam aplikasi *Android* bernama *Chilivest* yang terhubung dengan *backend Flask* melalui *REST API*. Berdasarkan hasil pengujian, metode *BoF* memperoleh akurasi sebesar 38,69%, *BoF + HSV* sebesar 78,55%, sedangkan *BoF + HSV + SPM* memperoleh hasil terbaik dengan akurasi sebesar 80,19%, *precision macro* sebesar 78,86%, *recall macro* sebesar 78,34%, dan *F1-score macro* sebesar 78,47%. Hasil pengujian juga menunjukkan bahwa aplikasi berjalan sesuai fungsi, *endpoint REST API* memperoleh *status response* 200 OK, dan komunikasi *client-server* memperoleh *packet loss* sebesar 0%. Dengan demikian, sistem *Chilivest* dapat digunakan sebagai prototipe aplikasi deteksi kesiapan panen tanaman cabai rawit berbasis citra.

Kata Kunci: Cabai rawit, kesiapan panen, *Bag-of-Features*, *HSV*, *Spatial Pyramid Matching*, *SVM*, *Android*.

ABSTRACT

Muhammad Nur Asy'ari, *2026. "Harvest Readiness Detection System for Cayenne Pepper Plants Using Bag-of-Features Feature Extraction Based on HSV Color and Spatial Pyramid." Digital Telecommunication Network Study Program, Department of Electrical Engineering, State Polytechnic of Malang. Supervisor I:* **Chandrasena Setiadi, S.T., M.Tr.T.** *Supervisor II:* **Aad Hariyadi, S.ST., M.T.**

Cayenne pepper is one of the horticultural commodities that has an important value in the agricultural sector. The determination of cayenne pepper harvest readiness is generally still carried out manually through visual observation, so the assessment results may vary depending on the experience and perception of each observer. Therefore, this study aims to design and develop an image-based harvest readiness detection system for cayenne pepper plants using a combination of Bag-of-Features (BoF), HSV, Spatial Pyramid Matching (SPM), Principal Component Analysis (PCA), and Support Vector Machine (SVM). The system works by processing cayenne pepper plant images with a size of 128 × 128 pixels. The images are extracted using BoF to obtain local patterns, HSV to represent color information, and SPM to preserve spatial information. The combined features are then reduced using PCA and classified using SVM into four classes, namely No Fruit, Not Ready to Harvest, Partially Ready to Harvest, and Ready to Harvest. This system is implemented in an Android application named Chilivest, which is connected to a Flask backend through a REST API. Based on the testing results, the BoF method obtained an accuracy of 38.69%, BoF + HSV obtained 78.55%, while BoF + HSV + SPM achieved the best result with an accuracy of 80.19%, macro precision of 78.86%, macro recall of 78.34%, and macro F1-score of 78.47%. The test results also show that the application runs according to its functions, the REST API endpoint obtains a 200 OK response status, and the client-server communication obtains a packet loss of 0%. Therefore, the Chilivest system can be used as a prototype of an image-based harvest readiness detection application for cayenne pepper plants.

Keywords: *Cayenne pepper, harvest readiness, Bag-of-Features, HSV, Spatial Pyramid Matching, SVM, Android.*

PRAKATA

Puji syukur ke hadirat Allah SWT atas segala rahmat, hidayah, dan karunia-Nya sehingga penulis dapat menyelesaikan skripsi yang berjudul "Sistem Deteksi Kesiapan Panen Tanaman Cabai Rawit Menggunakan Ekstraksi Fitur Bag-of-Features Berbasis Warna HSV dan Spatial Pyramid" dengan baik. Skripsi ini disusun sebagai salah satu syarat untuk memperoleh gelar Sarjana Terapan Teknik di Politeknik Negeri Malang. Penyusunan skripsi ini tidak terlepas dari bantuan, dukungan, bimbingan, doa, serta motivasi dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan ucapan terima kasih kepada :

1. Allah SWT yang telah memberikan kesehatan, kekuatan, kemudahan, dan kelancaran kepada penulis dalam menyelesaikan skripsi ini.
2. Nabi Muhammad SAW yang telah membawa umat manusia dari zaman kegelapan menuju zaman yang penuh dengan ilmu pengetahuan, serta menjadi suri teladan dalam kehidupan.
3. Kedua orang tua penulis, Bapak Iswanto dan Ibu Nurul Isa Isticharoh, yang selalu memberikan doa, kasih sayang, dukungan, nasihat, serta motivasi yang tidak pernah berhenti kepada penulis.
4. Bapak Supriatna Adhisuwignjo, S.T., M.T. selaku Direktur Politeknik Negeri Malang beserta seluruh jajarannya.
5. Bapak Prof. Ir. Mohammad Noor Hidayat, S.T., M.Sc., Ph.D. selaku Ketua Jurusan Teknik Elektro Politeknik Negeri Malang beserta seluruh jajarannya.
6. Ibu Mila Kusumawardani, S.T., M.T. selaku Koordinator Program Studi D-IV Jaringan Telekomunikasi Digital Politeknik Negeri Malang.
7. Bapak Chandrasena Setiadi, S.T., M.Tr.T., selaku Dosen Pembimbing I yang telah meluangkan waktu, tenaga, dan pikiran untuk memberikan bimbingan, arahan, saran, serta masukan yang sangat berharga selama proses penyusunan skripsi ini.
8. Bapak Aad Hariyadi, S.ST., M.T., selaku Dosen Pembimbing II yang telah meluangkan waktu, tenaga, dan pikiran untuk memberikan bimbingan, arahan, saran, serta masukan yang sangat berharga selama proses penyusunan skripsi ini.

9. Seluruh dosen dan tenaga kependidikan yang telah memberikan ilmu, bantuan, dan arahan selama penulis menempuh pendidikan.
10. Kepada seseorang yang begitu istimewa, Vania Alvioni Fitri, S.Tr.T., dan keluarga yang telah menghiasi masa akhir studi penulis dengan keindahannya, serta memberikan motivasi, doa, dan dukungan kepada penulis dalam menjalani masa studi hingga penyusunan skripsi ini selesai.
11. Teman-teman seperjuangan Kontrakan Beriman yang telah menjadi tempat pulang, teman seperjuangan, serta tidak bosan menemani proses penulis dalam menyusun dan menyelesaikan skripsi ini.
12. Teman-teman Kauman yang sudah memberikan dukungan dan doa kepada penulis sehingga bisa menyelesaikan masa kuliah dan skripsi ini.

Penulis menyadari bahwa skripsi ini masih memiliki keterbatasan dan jauh dari kata sempurna. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan dan pengembangan penelitian di masa mendatang.

Malang, 6 Agustus 2026

Penulis

DAFTAR ISI

HALAMAN PENGESAHAN i
PERNYATAAN KEASLIAN SKRIPSI ii
ABSTRAK iii
ABSTRACT iv
PRAKATA v
DAFTAR ISI vii
DAFTAR GAMBAR x
DAFTAR TABEL xii
DAFTAR LAMPIRAN xiii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Batasan Masalah 3
1.4 Tujuan Penelitian 4
1.5 Manfaat Penelitian 4
1.6 Luaran Penelitian 5
BAB II TINJAUAN PUSTAKA 6
2.1 Penelitian Terdahulu 6
2.2 Kajian Pustaka 10
2.2.1 Tanaman Cabai Rawit 10
2.2.2 Kesiapan Panen Cabai Rawit 10
2.2.3 Pengolahan Citra Digital 11
2.2.4 Bag-of-Features (*BoF*) 11

2.2.5 Fitur Warna *HSV* 13
2.2.6 *Spatial Pyramid Matching* (*SPM*) 15
2.2.7 *Principal Component Analysis* (*PCA*) 17
2.2.8 *Support Vector Machine* (*SVM*) 19
2.2.9 Aplikasi *Android* 21
2.2.10 *Flask Framework* 22
2.2.11 *REST API* 22
2.2.12 *Railway* 23
BAB III METODOLOGI PENELITIAN 24
3.1 Jenis Penelitian 24
3.2 Tahapan Penelitian 24
3.3 Perancangan Sistem 28
3.3.1 Blok Diagram Sistem 28
3.3.2 Flowchart 34
3.3.3 Blok Diagram Arsitektur *Client-Server* 40
3.3.4 Perancangan *REST API* dan *Deployment Backend* 41
3.3.5 Perancangan Desain Aplikasi *Android* 45
3.4 Penyiapan Alat dan Bahan 48
3.5 Parameter Pengujian 49
BAB IV HASIL DAN PEMBAHASAN 51
4.1 Tempat dan Waktu Penelitian 51
4.2 Hasil Penelitian 51
4.2.1 Tampilan Aplikasi *Android* 52
4.2.2 Hasil Pengolahan Citra dan Ekstraksi Fitur 65
4.2.3 Pengujian Perbandingan Metode 86

4.2.4 Pengujian Fungsionalitas Aplikasi 98
4.2.5 Pengujian Deteksi Menggunakan Citra Eksternal 108
4.2.6 Pengujian Komunikasi *Client-Server* 115
4.2.7 Pengujian *Deployment Backend* 122
4.2.8 Pengujian *REST API* 125
4.3 Pembahasan 131
4.3.1 Pembahasan Hasil Implementasi Aplikasi *Android* 131
4.3.2 Pembahasan Hasil Pengolahan Citra dan Ekstraksi Fitur 132
4.3.3 Pembahasan Hasil Perbandingan Metode 134
4.3.4 Pembahasan Fungsionalitas dan Respons Aplikasi *Android* 137
4.3.5 Pembahasan Deteksi Menggunakan Citra Eksternal 138
4.3.6 Pembahasan Komunikasi *Client-Server* 139
4.3.7 Pembahasan Hasil *Deployment Backend* 140
4.3.8 Pembahasan Hasil *Endpoint REST API* 141
BAB V PENUTUP 143
5.1 Kesimpulan 143
5.2 Saran 144
DAFTAR PUSTAKA 146
LAMPIRAN 150

DAFTAR GAMBAR

Gambar 2. 1 Bag of Features 11
Gambar 2. 2 Ruang Warna Hue Saturation Value 13
Gambar 2. 3 Spatial Pyramid Matching 15
Gambar 2. 4 Principal Component Analysis (PCA) 17
Gambar 2. 5 Support Vector Machine (SVM) 19
Gambar 3. 1 Flowchart Tahapan Penelitian 25
Gambar 3. 2 Blok Diagram Sistem 28
Gambar 3. 3 Blok Diagram *Bag of Features* 30
Gambar 3. 4 Blok Diagram Hue Saturation Value 31
Gambar 3. 5 Blok Diagram *Spatial Pyramid Matching* 32
Gambar 3. 6 Blok Diagram Kombinasi Metode 33
Gambar 3. 7 Flowchart Aplikasi *Chilivest* 34
Gambar 3. 8 Flowchart Proses Prediksi pada *Backend Flask* 36
Gambar 3. 9 Flowchart Ekstraksi Fitur *BoF*, *HSV*, dan *SPM* 37
Gambar 3. 10 Flowchart Reduksi Dimensi *PCA* dan Klasifikasi *SVM* 39
Gambar 3. 11 Blok Diagram Arsitektur *Client-Server* 40
Gambar 3. 12 Rancangan Tampilan Awal dan Pemilihan Jenis Pengguna 45
Gambar 3. 13 Rancangan Tampilan Menu Petani 46
Gambar 3. 14 Rancangan Tampilan *Login* Peneliti dan Menu Peneliti 47
Gambar 4. 1 Lokasi google maps Penelitian 51
Gambar 4. 2 Tampilan Logo Aplikasi 52
Gambar 4. 3 Tampilan Splash Screen 52
Gambar 4. 4 Tampilan Pemilihan Jenis Pengguna 53
Gambar 4. 5 Tampilan Onboarding Login Petani 54
Gambar 4. 6 Tampilan Menu Tanaman Anda 55
Gambar 4. 7 Tampilan Loading Proses Prediksi 56
Gambar 4. 8 Tampilan Hasil Prediksi 57
Gambar 4. 9 Tampilan Riwayat Prediksi 58
Gambar 4. 10 Tampilan Menu Anda 59

Gambar 4. 11 Tampilan Login Peneliti 60
Gambar 4. 12 Tampilan Menu Perbandingan Metode 61
Gambar 4. 13 Tampilan Detail Metode 62
Gambar 4. 14 Tampilan Menu Hasil Segmentasi 63
Gambar 4. 15 Tampilan Hasil Segmentasi 64
Gambar 4. 16 Tampilan Riwayat Segmentasi 65
Gambar 4. 17 Citra Input dan Hasil Resize 128 × 128 66
Gambar 4. 18 Hasil Deteksi Keypoint ORB 67
Gambar 4. 19 Grafik Distribusi Jumlah Keypoint ORB 69
Gambar 4. 20 Visualisasi Citra dalam Ruang Warna HSV 71
Gambar 4. 21 Histogram HSV Citra Tanaman Cabai Rawit 72
Gambar 4. 22 Visualisasi Pembagian Grid SPM 74
Gambar 4. 23 Grafik Cumulative Explained Variance PCA 78
Gambar 4. 24 Visualisasi Klasifikasi SVM Berdasarkan Fitur PCA 80
Gambar 4. 25 Hasil Output Akhir Klasifikasi SVM pada Citra Uji 82
Gambar 4. 26 Confusion Matrix Metode BoF 89
Gambar 4. 27 Confusion Matrix Metode BoF + HSV 92
Gambar 4. 28 Confusion Matrix Metode BoF + HSV + SPM 94
Gambar 4. 29 Grafik Perbandingan Metrik Evaluasi Metode 97
Gambar 4. 30 Repository Backend Chilivest di GitHub 122
Gambar 4. 31 Status Backend Chilivest pada Railway 123
Gambar 4. 32 Konfigurasi dan URL Backend Chilivest pada Railway 123
Gambar 4. 33 Pengujian Endpoint /health 125
Gambar 4. 34 Pengujian Endpoint /predict 126
Gambar 4. 35 Pengujian Endpoint /method-comparison 127
Gambar 4. 36 Pengujian Endpoint /predict-segmentation 128

DAFTAR TABEL

Tabel 3. 1 Endpoint API 42
Tabel 3. 2 Format Request Endpoint /predict 43
Tabel 3. 3 Tabel Perangkat Keras yang Digunakan 48
Tabel 3. 4 Tabel Perangkat Lunak yang Digunakan 48
Tabel 4. 1 Ringkasan Keypoint dan Descriptor ORB 70
Tabel 4. 2 Persentase Warna HSV pada Citra 73
Tabel 4. 3 Hasil Pembagian Grid SPM 75
Tabel 4. 4 Dimensi Fitur Hasil Ekstraksi 76
Tabel 4. 5 Ringkasan Hasil PCA 79
Tabel 4. 6 Ringkasan Visualisasi Klasifikasi SVM Berdasarkan Fitur PCA 80
Tabel 4. 7 Hasil Output Akhir Klasifikasi SVM pada Citra Uji 82
Tabel 4. 8 Nilai Decision Score SVM pada Setiap Kelas 86
Tabel 4. 9 Hasil Confusion Matrix Metode BoF 90
Tabel 4. 10 Hasil Evaluasi Metode BoF 91
Tabel 4. 11 Hasil Confusion Matrix Metode BoF + HSV 92
Tabel 4. 12 Hasil Evaluasi Metode BoF + HSV 93
Tabel 4. 13 Hasil Confusion Matrix Metode BoF + HSV + SPM 95
Tabel 4. 14 Hasil Evaluasi Metode BoF + HSV + SPM 96
Tabel 4. 15 Ringkasan Perbandingan Metrik Evaluasi 96
Tabel 4. 16 Pengujian Fungsionalitas Aplikasi Menu Petani 99
Tabel 4. 17 Pengujian Fungsionalitas Aplikasi Menu Peneliti 103
Tabel 4. 18 Hasil Pengujian Menggunakan Gambar Langsung 109
Tabel 4. 19 Pengujian Implementasi Komunikasi Client-Server 116
Tabel 4. 20 Hasil Pengujian QoS Koneksi Client ke Server 119
Tabel 4. 21 Hasil Pengujian Delay 120
Tabel 4. 22 Hasil Pengujian Deployment Backend 124
Tabel 4. 23 Hasil Pengujian Endpoint REST API 129

DAFTAR LAMPIRAN

Lampiran 1. Hasil Script Aplikasi 150
Lampiran 2. Dataset Tersimpan di Google Drive 158
Lampiran 3. Dokumentasi Pengujian Aplikasi 159
Lampiran 4. Daftar Riwayat Hidup 160

BAB I
PENDAHULUAN

1.1 Latar Belakang

Cabai rawit (*Capsicum frutescens*) merupakan salah satu komoditas hortikultura yang memiliki nilai penting dalam sektor pertanian dan banyak dikembangkan melalui berbagai genotipe untuk memperoleh karakter agronomi serta komponen hasil yang baik [1]. Kualitas cabai rawit dipengaruhi oleh ketepatan waktu panen, karena buah yang dipanen terlalu cepat atau terlalu lambat dapat mengalami perubahan mutu, warna, nilai jual, dan daya simpan [2]. Pada praktiknya, kesiapan panen cabai rawit umumnya dapat diamati dari perubahan warna buah, mulai dari hijau, oranye, hingga merah. Perubahan warna tersebut menjadi salah satu indikator visual yang berkaitan dengan tingkat kematangan buah [2], [3]. Oleh karena itu, penentuan kesiapan panen menjadi salah satu aspek penting dalam menjaga kualitas hasil panen dan mendukung proses pengambilan keputusan di bidang pertanian.

Saat ini, proses penentuan kesiapan panen cabai rawit masih banyak dilakukan secara manual melalui pengamatan visual oleh petani. Cara tersebut bergantung pada pengalaman, ketelitian, dan persepsi masing-masing pengamat. Penilaian kematangan buah secara manual dapat menimbulkan perbedaan hasil, terutama karena adanya variasi persepsi pengamat dan kondisi visual pada objek yang diamati [2]. Perbedaan tersebut dapat terjadi pada tanaman yang memiliki campuran buah berwarna hijau, oranye, dan merah dalam satu citra. Selain itu, pencatatan hasil pengamatan secara manual dapat menyulitkan proses penelusuran hasil pengamatan sebelumnya apabila dilakukan secara berulang. Oleh karena itu, diperlukan sistem berbasis citra yang dapat membantu proses deteksi kesiapan panen berdasarkan fitur visual yang dihitung oleh sistem dan menyimpan hasil prediksi dalam riwayat aplikasi.

Perkembangan pengolahan citra digital dan kecerdasan buatan dapat dimanfaatkan untuk membantu proses deteksi kesiapan panen tanaman cabai rawit berdasarkan citra. Pada penelitian ini, metode *Bag-of-Features* (*BoF*) digunakan untuk mengekstraksi ciri visual berbasis fitur lokal dari citra tanaman cabai rawit. Metode

BoF telah digunakan dalam pengenalan citra tanaman karena dapat merepresentasikan pola visual lokal pada citra [4]. Namun, *BoF* tidak secara langsung merepresentasikan informasi warna yang menjadi salah satu indikator pada tingkat kematangan buah. Oleh karena itu, penelitian ini menggabungkan *BoF* dengan fitur warna Hue, Saturation, Value (*HSV*) untuk merepresentasikan karakteristik warna cabai rawit. Penggunaan ruang warna *HSV* telah diterapkan pada klasifikasi kematangan buah dan klasifikasi karakteristik buah berbasis warna [3], [5]. Selain itu, *Spatial Pyramid Matching* (*SPM*) digunakan untuk mempertahankan informasi spasial melalui pembagian citra ke dalam beberapa *level grid*, sehingga sistem tidak hanya mengenali ciri warna dan tekstur, tetapi juga memperhatikan sebaran fitur pada citra [6]. Fitur gabungan *BoF*, *HSV*, dan *SPM* kemudian diproses melalui tahapan *normalisasi*, reduksi dimensi menggunakan *Principal Component Analysis* (*PCA*), dan diklasifikasikan menggunakan *Support Vector Machine* (*SVM*) ke dalam empat kelas, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. *SVM* dipilih karena banyak digunakan dalam klasifikasi citra buah dan dapat mengolah fitur warna maupun tekstur untuk proses klasifikasi [7]. Sistem ini diimplementasikan dalam aplikasi *Android* bernama *Chilivest* yang berfungsi sebagai *client* untuk mengambil atau memilih citra, mengirimkan citra ke *backend Flask* yang *dideploy* pada *Railway* melalui *REST API*, serta menampilkan hasil prediksi kepada pengguna. Dengan rancangan tersebut, proses deteksi dilakukan pada *server* sehingga aplikasi membutuhkan koneksi internet dan *backend* aktif. Riwayat prediksi pada tahap pengembangan ini disimpan secara lokal pada perangkat *Android*. Berdasarkan pengembangan awal, sistem memperoleh akurasi sekitar 80% dan masih dapat ditingkatkan melalui penambahan *dataset* serta evaluasi lebih lanjut. Melalui penelitian ini, sistem yang dikembangkan diharapkan dapat menjadi alat bantu awal bagi petani atau pengguna dalam memperkirakan kesiapan panen cabai rawit berdasarkan citra tanaman.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dipaparkan diatas, maka dapat dirumuskan sebuah rumusan masalah penelitian yakni sebagai berikut :

1. Bagaimana menerapkan metode *Bag-of-Features* (*BoF*) untuk mengekstraksi fitur bentuk dan tekstur dari citra tanaman cabai rawit dalam proses deteksi kesiapan panen?
2. Bagaimana menambahkan informasi warna lokal menggunakan *HSV patches* dan informasi spasial menggunakan *Spatial Pyramid Matching* (*SPM*) ke dalam sistem *BoF* agar lebih akurat dalam mengestimasi proporsi warna buah cabai sebagai indikator kesiapan panen?
3. Seberapa besar Tingkat akurasi klasifikasi yang dapat dicapai dengan menggunakan algoritma *Support Vector Machine* (*SVM*) dalam mendeteksi kesiapan panen tanaman cabai rawit berdasarkan kombinasi fitur *BoF*, *HSV*, dan *SPM*?

1.3 Batasan Masalah

Adapun batasan masalah yang dibuat pada penelitian ini agar berjalan dengan baik adalah sebagai berikut :

1. Pembahasan objek penelitian terutama difokuskan pada tanaman cabai rawit (*Capsicum frutescens*) secara keseluruhan, bukan buah cabai satuan, dengan pengamatan pada proporsi warna buah dalam satu citra tanaman untuk menentukan kesiapan panen.
2. Citra cabai yang digunakan dalam penelitian ini difokuskan pada *dataset* Gambar yang diambil pada kondisi pencahayaan yang stabil tanpa bayangan yang tajam, sudut pengambilan Gambar yang tegak atau relatif sejajar terhadap objek, latar belakang netral, serta tanpa adanya *occlusion* (halangan pada objek).
3. Pembahasan metode ekstraksi fitur bentuk dan tekstur terutama difokuskan pada penggunaan *Bag-of-Features (BoF)* dengan *keypoint* detector *ORB* dan proses kuantisasi menggunakan *K-Means Clustering*.
4. Pembahasan informasi warna terutama difokuskan pada fitur warna dalam ruang *HSV* yang diambil dalam bentuk *HSV histogram* per *patch/grid* atau *HSV patches* sebagai representasi warna lokal.

5. Algoritma klasifikasi dalam penelitian ini terutama difokuskan pada penggunaan *Support Vector Machine (SVM)*, sehingga penelitian tidak membahas eksplorasi model *deep learning* atau algoritma klasifikasi lainnya.

1.4 Tujuan Penelitian

Tujuan dari penelitian ini adalah sebagai berikut :

1. Mengembangkan sistem ekstraksi fitur citra cabai rawit menggunakan metode *Bag-of-Features* (*BoF*) berbasis *keypoint* lokal (*ORB*) untuk mengenali bentuk dan tekstur yang berkaitan dengan indikasi visual kesiapan panen.
2. Meningkatkan performa representasi fitur dengan mengintegrasikan informasi warna lokal berbasis *HSV patches* dan informasi spasial menggunakan *Spatial Pyramid Matching* (*SPM*) ke dalam sistem *BoF*.
3. Menerapkan algoritma *Support Vector Machine* (*SVM*) untuk melakukan deteksi kesiapan panen tanaman cabai rawit secara otomatis, serta mengevaluasi tingkat akurasi sistem yang dihasilkan berdasarkan proporsi dominan warna buah dalam satu Gambar tanaman, sebagai indikator keputusan panen.

1.5 Manfaat Penelitian

Manfaat dari penelitian ini adalah sebagai berikut :

1. Bagi pengembangan ilmu pengetahuan, penelitian ini dapat memperluas kajian dalam bidang pengolahan citra digital dan kecerdasan buatan, khususnya dalam penerapan metode *Bag-of-Features*, fitur warna *HSV*, dan *Spatial Pyramid Matching* untuk klasifikasi visual berbasis warna dan spasial.
2. Bagi sektor pertanian, sistem deteksi yang dikembangkan dapat menjadi solusi inovatif untuk membantu petani dalam menentukan tingkat waktu panen tanaman cabai rawit.
3. Bagi pengembangan teknologi terapan, hasil dari penelitian ini dapat diimplementasikan ke dalam bentuk aplikasi *mobile* yang praktis, sehingga dapat diakses langsung oleh pengguna lapangan seperti petani, pengepul, maupun pelaku distribusi hasil pertanian.

1.6 Luaran Penelitian

Luaran yang diharapkan melalui penelitian ini adalah sebagai berikut :

1. Model sistem klasifikasi citra yang mampu mendeteksi kesiapan panen tanaman cabai rawit berdasarkan proporsi warna dominan (hijau, oranye, merah) dalam satu citra tanaman secara otomatis menggunakan kombinasi metode *Bag-of-Features*, *HSV patches*, dan *Spatial Pyramid Matching*.
2. Implementasi algoritma klasifikasi *Support Vector Machine* (*SVM*) untuk memetakan hasil ekstraksi fitur ke dalam kategori "siap panen" atau "belum siap" berdasarkan dominasi warna cabai dalam Gambar tanaman secara menyeluruh dengan akurasi yang terukur melalui pengujian berbasis metrik evaluasi (seperti akurasi, *precision*, *recall*, dan *F1-score*).
3. Prototipe aplikasi *mobile* berbasis citra yang dapat digunakan oleh petani atau pelaku distribusi untuk mendeteksi kesiapan cabai rawit secara praktis melalui antarmuka yang sederhana dan responsif.

BAB II
TINJAUAN PUSTAKA

2.1 Penelitian Terdahulu

Penelitian terdahulu digunakan sebagai dasar dalam menentukan metode, pendekatan, dan ruang lingkup penelitian yang akan dilakukan. Beberapa penelitian sebelumnya telah membahas klasifikasi citra menggunakan fitur warna, tekstur, informasi spasial, serta algoritma *machine learning*. Penelitian-penelitian tersebut menjadi acuan dalam pengembangan sistem deteksi kesiapan panen tanaman cabai rawit menggunakan kombinasi *Bag-of-Features* (*BoF*), Hue, Saturation, Value (*HSV*), *Spatial Pyramid Matching* (*SPM*), *Principal Component Analysis* (*PCA*), dan *Support Vector Machine* (*SVM*). Berikut adalah data penelitian yang sudah pernah dilakukan dalam bentuk tabel dibawah ini.

Tabel 2. 1 Penelitian Terdahulu

No	Penulis & Tahun	Judul Penelitian	Metodologi	Temuan Utama	Kelemahan/ Riset Gap
1	G. Moreira, S. A. Magalhães, T. Pinho, F. N. Dos Santos, dan M. Cunha (2022)	*Benchmark* of *Deep Learning* and a Proposed *HSV* Colour Space Models for the Detection and Classification of Greenhouse Tomato [3]	*HSV Colour Space* + *Deep Learning*	Ruang warna *HSV* dapat digunakan untuk mendukung proses deteksi dan klasifikasi tingkat kematangan tomat berdasarkan perubahan warna buah.	Penelitian berfokus pada tomat dan belum menggunakan kombinasi *BoF*, *HSV*, dan *SPM* untuk deteksi kesiapan panen cabai rawit.

2	M. R. Saputra dan H. Irsyad (2022)	Klasifikasi Tingkat Kemanisan Alpukat Berdasarkan Fitur Hue Saturation Value (*HSV*) dengan Menggunakan *Support Vector Machine* (*SVM*) [5]	*HSV + SVM*	Fitur *HSV* dapat digunakan sebagai dasar klasifikasi buah, sedangkan *SVM* digunakan sebagai algoritma klasifikasi berdasarkan fitur warna.	Penelitian hanya menggunakan fitur warna *HSV* dan belum menggabungkan fitur bentuk, tekstur, serta informasi spasial.
3	Y. Zhang, J. Cui, Z. Wang, J. Kang, dan Y. Min (2020)	Leaf *Image* Recognition Based on *Bag of Features* [4]	*Bag-of-Features + SVM*	*Bag-of-Features* dapat digunakan untuk mengenali citra daun berdasarkan fitur visual lokal.	Penelitian berfokus pada pengenalan citra daun dan belum mempertimbangkan informasi warna sebagai indikator kematangan buah.
4	Y. Yang, D. Zhu, F. Ren, dan C. Cheng (2020)	A Novel Self-Taught Learning *Framework* Using *Spatial Pyramid Matching* for Scene	*Spatial Pyramid Matching + BoF*	*Spatial Pyramid Matching* dapat mempertahankan informasi spasial citra melalui pembagian area Gambar ke dalam beberapa *level*.	Penelitian diterapkan pada klasifikasi scene dan belum diterapkan pada objek tanaman cabai rawit yang

		Classification [6]			memiliki variasi warna buah.
5	Nica Astrianda (2020)	Klasifikasi Kematangan Buah Tomat Dengan Variasi Model Warna Menggunakan *Support Vector Machine* [8]	Model Warna + *SVM*	Variasi model warna dapat digunakan untuk membantu proses klasifikasi kematangan buah tomat menggunakan metode *SVM*.	Penelitian berfokus pada buah tomat dan belum menggunakan kombinasi fitur *BoF*, *HSV*, dan *SPM* untuk mendeteksi kesiapan panen cabai rawit.
6	Yusuf Amrozi, Dian Yuliati, Agung Susilo, Nur Novianto, dan Rikza Ramadhan (2022)	Klasifikasi Jenis Buah Pisang Berdasarkan Citra Warna dengan Metode *SVM* [9]	Citra Warna + *SVM*	Citra warna dapat digunakan sebagai fitur visual dalam proses klasifikasi objek buah menggunakan algoritma *SVM*.	Penelitian berfokus pada klasifikasi jenis buah pisang, bukan deteksi tingkat kesiapan panen atau kematangan tanaman cabai rawit.
7	Mutmainnah Muchtar dan Rafiqah Arjaliyah Muchtar (2024)	Perbandingan Metode *KNN* dan *SVM* dalam Klasifikasi Kematangan Buah Mangga Berdasarkan Citra *HSV* dan	*HSV* + Fitur Statistik + *KNN* + *SVM*	Fitur *HSV* dan fitur statistik dapat digunakan untuk klasifikasi kematangan buah, serta *SVM* dapat dibandingkan dengan metode *KNN* untuk	Penelitian berfokus pada kematangan buah mangga dan belum menggabungkan fitur lokal *BoF* serta informasi spasial *SPM*.

		Fitur Statistik [10]		mengetahui performa terbaik.	
8	Bagoes Maulana Alfaruq, Danang Erwanto, dan Iska Yanuartanti (2023)	Klasifikasi Kematangan Buah Tomat Dengan Metode *Support Vector Machine* [11]	*GLCM* + Momen Warna + *SVM*	*SVM* dengan fitur tekstur *GLCM* dan momen warna berhasil mengklasifikasikan kematangan buah tomat dengan nilai *accuracy* sebesar 0,91, *precision* 0,91, *recall* 0,92, dan F-measure 0,92.	Penelitian berfokus pada kematangan buah tomat satuan, sedangkan penelitian ini berfokus pada deteksi kesiapan panen tanaman cabai rawit secara keseluruhan menggunakan *BoF*, *HSV*, dan *SPM*.
9	V. Vinoharan dan A. Ramanan (2021)	An Efficient *Bag-of-Feature* Representation for Object Classification [12]	*Bag-of-Features*	Representasi *Bag-of-Features* dapat digunakan untuk klasifikasi objek berdasarkan fitur visual lokal.	Penelitian berfokus pada klasifikasi objek umum dan belum menggabungkan fitur warna *HSV* serta informasi spasial untuk deteksi kesiapan panen. q
10	B. Gajjar, H. Mewada, dan A. Patani (2021)	*Sparse Coded Spatial Pyramid Matching* and *Multi-Kernel* Integrated *SVM* for *Non-Linear Scene Classification* [13]	*Spatial Pyramid Matching* + *Multi-Kernel SVM*	Kombinasi *Spatial Pyramid Matching* dan *SVM* dapat digunakan untuk mendukung klasifikasi citra dengan mempertahankan informasi spasial.	Penelitian diterapkan pada *scene classification* dan belum diterapkan pada citra tanaman cabai rawit berbasis indikator warna panen.

2.2 Kajian Pustaka

2.2.1 Tanaman Cabai Rawit

Cabai rawit (*Capsicum frutescens*) merupakan salah satu tanaman hortikultura yang memiliki buah berukuran kecil dengan rasa pedas dan banyak dimanfaatkan sebagai bahan pangan maupun komoditas pertanian. Dalam budidaya cabai rawit, terdapat berbagai genotipe atau galur yang dapat menunjukkan perbedaan karakter agronomi, seperti tinggi tanaman, jumlah buah, bobot buah, dan komponen hasil lainnya [1].

Dalam penelitian ini, tanaman cabai rawit digunakan sebagai objek utama yang diamati melalui citra digital. Citra tanaman digunakan untuk melihat kondisi visual tanaman dan buah, terutama berdasarkan keberadaan buah serta perubahan warna buah. Informasi visual tersebut kemudian digunakan sebagai dasar dalam proses deteksi kesiapan panen tanaman cabai rawit.

2.2.2 Kesiapan Panen Cabai Rawit

Kesiapan panen merupakan kondisi ketika tanaman atau buah telah mencapai tahap tertentu sehingga dapat dipanen sesuai dengan kebutuhan penggunaan. Pada buah, tingkat kematangan dapat memengaruhi mutu, warna, tekstur, daya simpan, dan nilai jual. Penentuan tingkat kematangan buah umumnya dapat diamati melalui ciri visual, salah satunya perubahan warna pada permukaan buah [2].

Pada cabai rawit, perubahan warna buah menjadi salah satu indikator visual yang dapat digunakan untuk memperkirakan kesiapan panen. Buah cabai rawit umumnya mengalami perubahan warna dari hijau menuju oranye hingga merah seiring proses pematangan. Penggunaan warna sebagai indikator kematangan juga telah diterapkan pada penelitian klasifikasi kematangan buah berbasis citra, seperti pada tomat dengan pemanfaatan ruang warna *HSV* [3].

Dalam penelitian ini, kesiapan panen cabai rawit dikelompokkan ke dalam empat kelas, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Pengelompokan tersebut didasarkan pada kondisi visual tanaman dan buah

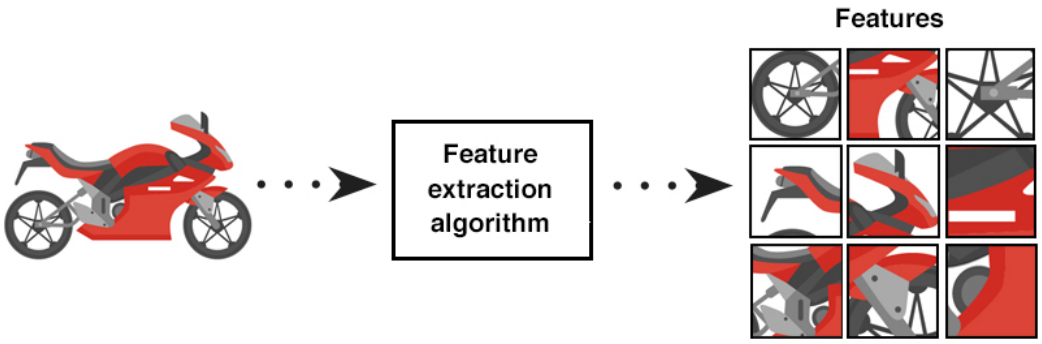
dalam citra, terutama keberadaan buah serta perubahan warna buah yang terlihat pada tanaman.

2.2.3 Pengolahan Citra Digital

Pengolahan citra digital merupakan proses pengolahan Gambar menggunakan komputer untuk memperoleh informasi tertentu dari citra. Citra digital tersusun dari piksel yang memiliki nilai intensitas atau warna, sehingga informasi pada citra dapat dianalisis berdasarkan karakteristik visual seperti warna, tekstur, bentuk, dan pola tertentu. Dalam bidang pertanian, pengolahan citra digital dapat digunakan untuk membantu proses identifikasi kondisi tanaman melalui tahapan seperti akuisisi citra, peningkatan kualitas citra, segmentasi, ekstraksi fitur, dan klasifikasi [14].

Dalam penelitian ini, pengolahan citra digital digunakan untuk memproses citra tanaman cabai rawit sebelum dilakukan klasifikasi kesiapan panen. Tahapan yang dilakukan meliputi penyesuaian ukuran citra, ekstraksi fitur lokal, pembentukan fitur warna, pengambilan informasi spasial, dan klasifikasi citra. Informasi visual yang diperoleh dari citra kemudian digunakan sebagai dasar dalam mengelompokkan tanaman cabai rawit ke dalam kelas Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen.

2.2.4 Bag-of-Features (*BoF*)



Gambar 2. 1 *Bag of Features*
Sumber : *https://www.analyticsvidhya.com/blog/2023/01/bag-of-features-simplifying-image-recognition-for-non-experts/*

Bag-of-Features (BoF) merupakan metode representasi citra yang bekerja dengan mengubah kumpulan fitur lokal pada Gambar menjadi bentuk *histogram* visual. Konsep *BoF* mengadaptasi pendekatan *Bag-of-Words* pada pengolahan teks, tetapi objek yang dihitung bukan kata, melainkan fitur visual yang terdapat pada citra. Pada metode ini, citra tidak langsung diklasifikasikan berdasarkan Gambar secara utuh, tetapi terlebih dahulu diubah menjadi kumpulan ciri lokal yang dapat direpresentasikan dalam bentuk numerik [4], [12].

a. Ekstraksi Fitur Lokal

Tahapan awal pada *BoF* adalah mengekstraksi fitur lokal dari citra. Fitur lokal merupakan ciri yang diperoleh dari bagian tertentu pada Gambar, seperti titik sudut, tepi, pola tekstur, atau area dengan perubahan intensitas tertentu. Pada penelitian ini, ekstraksi fitur lokal dilakukan menggunakan *Oriented FAST and Rotated BRIEF* (*ORB*). *ORB* digunakan untuk mendeteksi *keypoint* dan menghasilkan *descriptor* dari citra. *Descriptor* tersebut berisi informasi numerik yang merepresentasikan karakteristik lokal pada citra tanaman cabai rawit

b. Pembentukan *Visual Codebook*

Setelah *descriptor* diperoleh dari citra latih, tahap berikutnya adalah pembentukan *visual codebook*. *Visual codebook* merupakan kumpulan *visual word* yang dibentuk melalui proses klasterisasi *descriptor*. Pada penelitian ini, proses klasterisasi dilakukan menggunakan *K-Means*. Setiap pusat klaster yang dihasilkan oleh *K-Means* dianggap sebagai *visual word*. *Visual word* tersebut berfungsi sebagai representasi dari pola fitur lokal yang terdapat pada kumpulan citra latih [4], [12].

c. Kuantisasi *Descriptor* dan Pembentukan *Histogram*

Tahap selanjutnya adalah proses kuantisasi *descriptor* ke dalam *visual word*. Pada tahap ini, setiap *descriptor* dari suatu citra dicocokkan dengan *visual word* terdekat yang terdapat pada *visual codebook*. Setelah proses pencocokan dilakukan, jumlah kemunculan setiap *visual word* dihitung untuk membentuk *histogram BoF*. Secara umum, *histogram BoF* dapat dituliskan pada Persamaan [4], [12].

Formula *histogram BoF*:

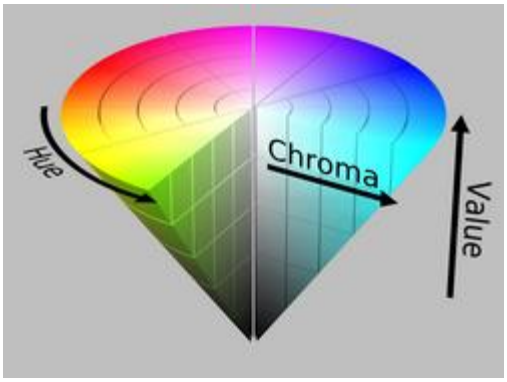
1. Eks Ekstraksi fitur lokal (*ORB/SIFT*) → deskriptor di
2. Pembentukan kodebook melalui *K-Means* → *cluster centers* $\{c_k\}$
3. *Histogram visual word*:

$$h = [h_1, h_2, \dots, h_K], h_k = \sum_i I(NN(d_i) = c_k)$$

dengan $I(.)$ *indicator* dan NN *nearest centroid* [4], [12]

Oleh karena itu, dalam penelitian ini, *BoF* digunakan sebagai fondasi ekstraksi fitur bentuk dan tekstur, yang kemudian akan diperkuat dengan penambahan fitur warna lokal berbasis *HSV* dan penyesuaian spasial menggunakan *Spatial Pyramid Matching* (*SPM*).

2.2.5 Fitur Warna *HSV*



Gambar 2. 2 Ruang Warna *Hue Saturation Value*
Sumber : *https://en.wikipedia.org/wiki/HSL_and_HSV*

Hue, Saturation, Value (*HSV*) merupakan ruang warna yang digunakan untuk merepresentasikan warna berdasarkan tiga komponen utama, yaitu hue, saturation, dan value. Hue menunjukkan jenis warna, saturation menunjukkan tingkat kejenuhan warna, sedangkan value menunjukkan tingkat kecerahan warna. Ruang warna *HSV* banyak digunakan dalam pengolahan citra karena dapat merepresentasikan karakteristik warna objek yang diamati pada citra digital [3], [5], [8], [10] .

Pada penelitian klasifikasi kematangan buah, warna menjadi salah satu ciri visual yang penting karena perubahan warna dapat menunjukkan tingkat kematangan objek. Moreira *et al.* menggunakan ruang warna *HSV* untuk mendukung proses deteksi

dan klasifikasi tingkat kematangan tomat berdasarkan perbedaan warna buah [3]. Saputra dan Irsyad juga menerapkan fitur *HSV* bersama algoritma *Support Vector Machine* (*SVM*) untuk klasifikasi tingkat kemanisan alpukat berdasarkan karakteristik warna [5]. Hal ini menunjukkan bahwa fitur *HSV* dapat digunakan sebagai salah satu fitur visual dalam proses klasifikasi citra buah.

a. Komponen Hue

Hue merupakan komponen yang menunjukkan jenis warna pada citra, seperti merah, hijau, kuning, biru, dan warna lainnya. Pada citra tanaman cabai rawit, komponen hue berperan penting karena perubahan warna buah dari hijau menuju oranye hingga merah dapat digunakan sebagai indikator visual dalam memperkirakan kesiapan panen. Informasi hue digunakan untuk membedakan warna dominan pada citra tanaman cabai rawit .

b. Komponen Saturation

Saturation merupakan komponen yang menunjukkan tingkat kejenuhan atau kemurnian warna. Nilai saturation yang tinggi menunjukkan warna yang lebih kuat, sedangkan nilai saturation yang rendah menunjukkan warna yang lebih pucat atau mendekati abu-abu. Dalam penelitian ini, saturation digunakan untuk membantu merepresentasikan intensitas warna buah cabai rawit yang tampak pada citra.

c. Komponen Value

Value merupakan komponen yang menunjukkan tingkat kecerahan warna pada citra. Nilai value yang tinggi menunjukkan bagian citra yang lebih terang, sedangkan nilai value yang rendah menunjukkan bagian citra yang lebih gelap. Komponen ini dapat membantu sistem dalam merepresentasikan variasi kecerahan pada citra tanaman cabai rawit, terutama pada citra yang diambil dalam kondisi pencahayaan alami.

d. *Histogram* Warna *HSV*

Fitur warna *HSV* dapat direpresentasikan dalam bentuk *histogram*. *Histogram* warna digunakan untuk menghitung frekuensi kemunculan nilai warna pada setiap

kanal *HSV*. Secara umum, *histogram HSV* dapat dituliskan pada Persamaan [3], [5], [12].

$$F_{HSV} = \left[h_1^H, h_2^H, \dots, h_p^H, h_1^S, h_2^S, \dots, h_q^S, h_1^V, h_2^V, \dots, h_r^V\right]$$

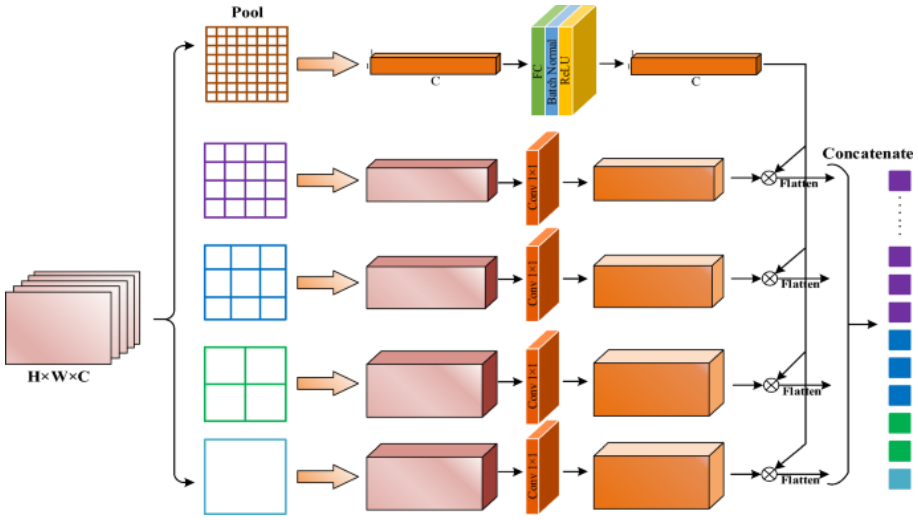
dengan nilai *histogram* pada setiap kanal dapat dituliskan sebagai berikut:

$$h_j^X = \sum_i I\left(bin(x_i^X) = j\right)$$

Dengan tiap h_j^X adalah jumlah piksel pada $bin\ j$.

Dalam konteks penelitian ini, fitur warna dalam ruang *HSV* akan diekstraksi dalam bentuk *histogram* warna lokal dari setiap *patch* citra, yang kemudian dikombinasikan dengan fitur tekstur dari *BoF* dan fitur spasial dari *SPM*, membentuk representasi fitur yang lebih komprehensif.

2.2.6 *Spatial Pyramid Matching* (*SPM*)



Gambar 2. 3 *Spatial Pyramid Matching*
Sumber : *https://link.springer.com/article/10.1007/s10845-023-02231-z*

Spatial Pyramid Matching (*SPM*) merupakan metode yang digunakan untuk mempertahankan informasi spasial pada citra. Pada metode *Bag-of-Features* (*BoF*), citra direpresentasikan dalam bentuk *histogram visual word*, tetapi posisi atau lokasi kemunculan fitur pada citra tidak diperhatikan secara langsung. Hal tersebut dapat menjadi keterbatasan ketika objek yang diamati memiliki sebaran fitur yang berbeda

pada beberapa bagian citra. Untuk mengatasi hal tersebut, *SPM* membagi citra ke dalam beberapa *level grid*, kemudian menghitung *histogram* fitur pada setiap bagian *grid* [6], [13].

a. Pembagian Citra ke dalam *Level Pyramid*

Pada *SPM*, citra dibagi ke dalam beberapa tingkatan atau *level*. Pada *level* 0, citra dipandang sebagai satu bagian utuh berukuran 1×1. Pada *level* 1, citra dibagi menjadi 2×2 bagian. Pada *level* 2, citra dibagi menjadi 4×4 bagian. Setiap *level* menghasilkan jumlah blok yang berbeda sesuai dengan banyaknya pembagian *grid*. Semakin tinggi *level pyramid*, semakin rinci pembagian area citra yang digunakan untuk memperoleh informasi spasial.

Jumlah blok pada setiap *level pyramid* dapat dituliskan pada Persamaan [6], [13].

$$B_l = 2^l x 2^l = 4^l$$

Berdasarkan Persamaan (2.4), *level* 0 menghasilkan 1 blok, *level* 1 menghasilkan 4 blok, dan *level* 2 menghasilkan 16 blok. Dengan demikian, apabila digunakan *level* 0 sampai *level* 2, maka jumlah seluruh blok yang dihasilkan dapat dihitung menggunakan Persamaan (2.5).

$$B_{total} = \sum_{l=0}^{L} 4^l$$

b. Pembentukan *Histogram* pada Setiap Blok

Setelah citra dibagi ke dalam beberapa blok, fitur pada setiap blok dihitung dalam bentuk *histogram*. *Histogram* tersebut dapat berasal dari fitur visual seperti *BoF* atau fitur warna seperti *HSV*. Dengan cara ini, sistem tidak hanya mengetahui frekuensi kemunculan fitur pada citra, tetapi juga memperoleh informasi mengenai lokasi atau sebaran fitur pada bagian tertentu dari citra.

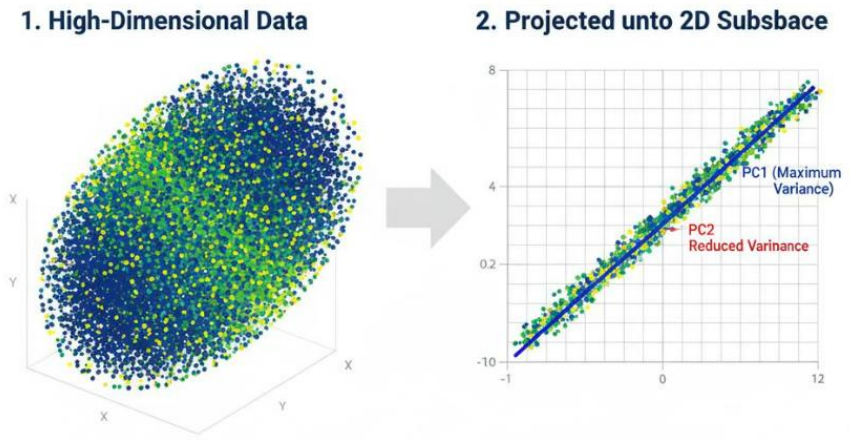
Secara umum, representasi fitur *SPM* dapat dituliskan pada Persamaan [6], [15].

$$\Phi(I) = [w_0 h_0, w_1 h_1, \dots, w_L h_L]$$

Berdasarkan Persamaan (2.6), *histogram* dari setiap *level* digabungkan menjadi satu vektor fitur. *Histogram* dari *level* yang berbeda dapat diberikan bobot sesuai dengan tingkat detail informasi spasial yang dihasilkan. Semakin tinggi *level pyramid*, semakin rinci informasi posisi fitur yang diperoleh.

Dalam penelitian ini, *SPM* akan digunakan untuk mengelompokkan fitur *BoF* dan *HSV* dalam tingkatan spasial, guna mempertahankan distribusi posisi warna dan bentuk cabai. Hal ini diharapkan dapat meningkatkan sensitivitas sistem terhadap perbedaan kesiapan panen tanaman cabai rawit secara *visual*.

2.2.7 *Principal Component Analysis* (*PCA*)



Gambar 2. 4 *Principal Component Analysis (PCA)*
Sumber : *https://medium.com/@ml.enesguler/understanding-principal-component-analysis-pca-b1d8f5aaf766*

Principal Component Analysis (*PCA*) merupakan metode reduksi dimensi yang digunakan untuk mengubah data berdimensi tinggi menjadi data dengan dimensi yang lebih rendah, tetapi tetap mempertahankan informasi utama dari data. *PCA* bekerja dengan mencari komponen utama atau principal component yang merepresentasikan variasi terbesar dari data. Dengan menggunakan *PCA*, fitur yang memiliki jumlah dimensi besar dapat diringkas menjadi beberapa komponen utama yang lebih sederhana [16].

Dalam pengolahan citra digital, *PCA* dapat digunakan untuk mengurangi dimensi fitur setelah proses ekstraksi fitur dilakukan. Hal ini diperlukan karena gabungan fitur *Bag-of-Features* (*BoF*), *HSV*, dan *Spatial Pyramid Matching* (*SPM*) dapat menghasilkan vektor fitur dengan dimensi yang besar. Dimensi fitur yang terlalu besar dapat meningkatkan kompleksitas proses pelatihan dan klasifikasi. Oleh karena itu, *PCA* digunakan untuk mereduksi dimensi fitur sebelum fitur tersebut masuk ke tahap klasifikasi [16], [17].

a. Konsep Dasar *PCA*

Tahapan *PCA* diawali dengan pemusatan data, yaitu mengurangi setiap nilai fitur dengan rata-rata fitur tersebut. Setelah itu, *PCA* mencari arah baru yang disebut principal component. Komponen utama tersebut dipilih berdasarkan nilai variansi terbesar, sehingga informasi penting dari data tetap dapat dipertahankan meskipun jumlah dimensinya dikurangi.

Secara umum, proses proyeksi data ke komponen utama dapat dituliskan pada Persamaan (2.7) [14].

$$Z = X_c W$$

Berdasarkan Persamaan (2.7), data yang telah dipusatkan diproyeksikan ke dalam ruang fitur baru menggunakan principal component yang dipilih. Hasil dari proses tersebut adalah fitur baru dengan jumlah dimensi yang lebih kecil dibandingkan fitur awal.

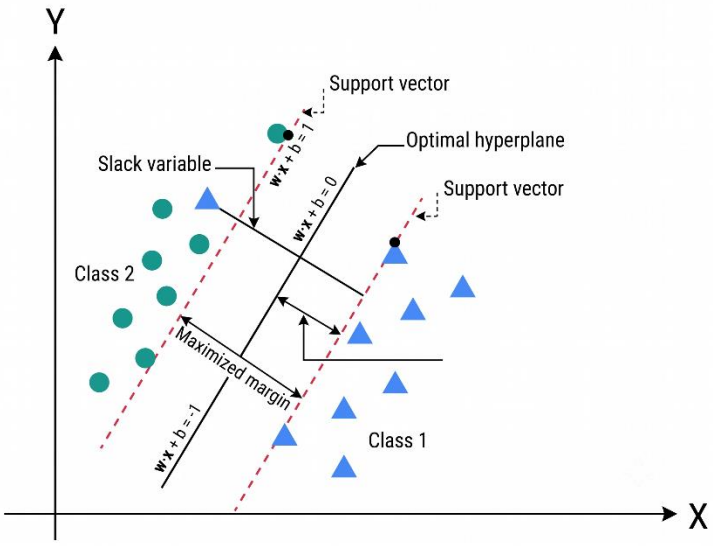
b. Explained Variance

Explained variance digunakan untuk mengetahui seberapa besar informasi yang dipertahankan oleh setiap principal component. Nilai ini menjadi dasar dalam menentukan jumlah komponen utama yang digunakan. Secara umum, rasio variansi yang dijelaskan oleh komponen ke-k dapat dituliskan pada Persamaan (2.8) [16].

$$EVR_k = \frac{\lambda_k}{\sum_{j=1}^{d} \lambda_j}$$

Nilai explained variance digunakan untuk menentukan jumlah komponen *PCA* yang mampu mempertahankan sebagian besar informasi dari data. Dalam penelitian ini, jumlah komponen *PCA* ditentukan berdasarkan cumulative explained variance, sehingga fitur hasil reduksi tetap mempertahankan informasi utama dari gabungan fitur *BoF*, *HSV*, dan *SPM*.

2.2.8 *Support Vector Machine* (*SVM*)



Gambar 2. 5 *Support Vector Machine (SVM)*
Sumber : *https://www.ibm.com/id-id/think/topics/support-vector-machine*

Support Vector Machine (*SVM*) merupakan algoritma *machine learning* yang digunakan untuk proses klasifikasi dan regresi. Pada proses klasifikasi, *SVM* bekerja dengan mencari bidang pemisah terbaik atau hyperplane yang dapat memisahkan data ke dalam kelas tertentu. Hyperplane terbaik ditentukan berdasarkan margin terbesar antara data dari kelas yang berbeda. Data yang berada paling dekat dengan hyperplane disebut *support* vector, karena data tersebut berpengaruh dalam menentukan posisi hyperplane [7], [18], [19].

SVM banyak digunakan dalam klasifikasi citra karena mampu menangani data dengan jumlah fitur yang besar. Pada penelitian klasifikasi citra, fitur yang telah diekstraksi dari Gambar dapat digunakan sebagai masukan untuk *SVM*. Siswantoro

menggunakan *SVM* untuk klasifikasi citra buah berdasarkan gabungan fitur warna dan tekstur [7]. Solihin *et al.* juga menggunakan *SVM* sebagai salah satu algoritma klasifikasi pada citra daun tanaman [18]. Selain itu, Nanni *et al.* menjelaskan bahwa *SVM* dapat digunakan untuk mengolah fitur hasil ekstraksi dalam proses klasifikasi citra [19].

a. Hyperplane dan Margin

Konsep dasar *SVM* adalah mencari hyperplane yang dapat memisahkan data antar kelas. Pada kasus klasifikasi linear, hyperplane dapat dituliskan pada Persamaan [7], [18], [19].

$$w^T x + b = 0$$

Hyperplane tersebut digunakan sebagai batas pemisah antara kelas. *SVM* berusaha mencari hyperplane dengan margin terbesar agar pemisahan antar kelas menjadi lebih baik. Margin merupakan jarak antara hyperplane dengan data terdekat dari masing-masing kelas.

b. Fungsi Keputusan *SVM*

Setelah hyperplane diperoleh, *SVM* menentukan kelas data berdasarkan posisi data terhadap hyperplane. Fungsi keputusan *SVM* dapat dituliskan pada Persamaan [7], [18], [19].

$$f(x) = sign(w^T x + b)$$

Berdasarkan Persamaan (2.10), data akan diklasifikasikan berdasarkan nilai fungsi keputusan. Jika nilai fungsi berada pada salah satu sisi hyperplane, maka data akan masuk ke kelas tertentu. Sebaliknya, jika berada pada sisi lain, data akan masuk ke kelas yang berbeda.

c. Kernel pada *SVM*

Pada data citra, hubungan antar fitur tidak selalu dapat dipisahkan secara linear. Oleh karena itu, *SVM* dapat menggunakan fungsi kernel untuk memetakan data ke ruang berdimensi lebih tinggi, sehingga data yang tidak dapat dipisahkan secara linear menjadi lebih mudah dipisahkan. Salah satu kernel yang banyak digunakan

adalah *Radial Basis Function* (*RBF*). Kernel *RBF* dapat dituliskan pada Persamaan [7], [19].

$$K(x_i, x_j) = exp(-\gamma \parallel x_i - x_j \parallel^2)$$

Kernel *RBF* digunakan untuk membantu *SVM* dalam memisahkan data yang memiliki pola *non-linear*. Parameter γberpengaruh terhadap jangkauan pengaruh suatu data terhadap data lainnya, sedangkan parameter Cpada *SVM* berpengaruh terhadap keseimbangan antara margin dan kesalahan klasifikasi.

SVM digunakan untuk mengklasifikasikan citra tanaman cabai rawit ke dalam empat kelas, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Pemilihan *SVM* didasarkan pada kemampuannya dalam mengolah fitur hasil ekstraksi citra dan penggunaannya pada beberapa penelitian klasifikasi citra berbasis fitur warna, tekstur, dan bentuk [7], [11], [18], [20]. Detail parameter *SVM* yang digunakan dalam penelitian ini, seperti nilai C, γ, dan jenis kernel, dijelaskan lebih lanjut pada Bab III bagian pelatihan dan pengujian model.

2.2.9 Aplikasi *Android*

Aplikasi *Android* merupakan perangkat lunak yang dikembangkan untuk berjalan pada sistem operasi *Android*. *Android* banyak digunakan pada perangkat bergerak seperti *smartphone* dan *tablet* karena mendukung fitur kamera, penyimpanan lokal, koneksi internet, serta antarmuka pengguna yang interaktif. Dalam sistem berbasis citra, aplikasi *Android* dapat digunakan sebagai media untuk mengambil atau memilih Gambar, mengirim data ke sistem pemrosesan, serta menampilkan hasil prediksi kepada pengguna. Pada bidang pertanian, aplikasi *Android* telah digunakan untuk mendukung sistem deteksi berbasis citra, seperti deteksi penyakit daun padi berbasis *MobileNet* pada aplikasi *Android* dan evaluasi model deteksi penyakit tanaman pada perangkat *mobile* [21], [22].

Dalam penelitian ini, aplikasi *Android* digunakan sebagai *client* pada sistem deteksi kesiapan panen tanaman cabai rawit. Aplikasi yang dikembangkan bernama *Chilivest* dan berfungsi untuk mengambil atau memilih citra tanaman cabai rawit, mengirimkan citra ke *backend Flask* melalui *REST API*, menerima hasil prediksi dari

server, serta menampilkan hasil deteksi kepada pengguna. Proses klasifikasi dilakukan pada *backend*, sedangkan aplikasi *Android* berperan sebagai antarmuka pengguna dalam sistem deteksi kesiapan panen berbasis citra.

2.2.10 *Flask Framework*

Flask merupakan *microframework* berbasis bahasa pemrograman *Python* yang digunakan untuk membangun aplikasi web dan layanan *backend*. *Flask* bersifat ringan dan fleksibel karena dapat dikembangkan sesuai kebutuhan melalui pustaka atau ekstensi tambahan. Dalam pengembangan sistem berbasis *machine learning*, *Flask* dapat digunakan untuk menghubungkan model yang telah dilatih dengan antarmuka pengguna melalui layanan web atau Application Programming Interface (*API*) [23].

Dalam penelitian ini, *Flask* digunakan untuk beberapa keperluan penting, antara lain:

1. Menerima citra tanaman cabai rawit yang dikirim dari aplikasi *Android Chilivest* melalui *REST API*.
2. Menjalankan proses pengolahan citra dan klasifikasi menggunakan model yang telah dibuat.
3. Mengirimkan hasil prediksi kesiapan panen kembali ke aplikasi *Android* dalam bentuk *response* yang dapat ditampilkan kepada pengguna.

Kemampuan *Flask* dalam membangun layanan *backend* berbasis *Python* menjadikannya sesuai untuk mengintegrasikan aplikasi *Android* dengan model klasifikasi pada sisi *server*. Dengan menggunakan *Flask*, proses klasifikasi dapat dilakukan pada *backend*, sedangkan aplikasi *Android* berperan sebagai *client* untuk mengirim citra dan menampilkan hasil prediksi kepada pengguna.

2.2.11 *REST API*

REST API merupakan antarmuka pemrograman aplikasi yang digunakan untuk menghubungkan *client* dan *server* melalui protokol *HTTP*. REST bekerja dengan konsep resource yang dapat diakses melalui *Uniform Resource Identifier* (*URI*), kemudian *server* memberikan *response* dalam format tertentu, seperti *JSON*. Pada sistem berbasis *machine learning*, *REST API* dapat digunakan untuk menerima *input*,

menjalankan proses prediksi, dan mengirimkan hasil kembali kepada pengguna [23], [24].

Dalam penelitian ini, *REST API* digunakan sebagai media komunikasi antara aplikasi *Android Chilivest* dan *backend Flask*. Aplikasi *Android* mengirimkan citra tanaman cabai rawit ke *server*, kemudian *backend Flask* memproses citra tersebut dan mengirimkan hasil prediksi kembali dalam format *JSON*. Dengan demikian, *REST API* berperan sebagai penghubung antara aplikasi *Android* dan *backend* pada sistem deteksi kesiapan panen tanaman cabai rawit.

2.2.12 *Railway*

Railway merupakan platform *deployment* berbasis *cloud* yang dapat digunakan untuk menjalankan aplikasi *backend* secara *online*. Konsep penggunaan *Railway* berkaitan dengan *cloud computing*, yaitu layanan komputasi yang memanfaatkan jaringan internet untuk menyediakan sumber daya seperti *server*, penyimpanan, jaringan, platform, dan perangkat lunak. *Cloud computing* memungkinkan aplikasi dapat diakses dari berbagai perangkat selama terhubung dengan internet [25].

Dalam penelitian ini, *Railway* digunakan sebagai platform untuk menjalankan *backend Flask* pada sistem deteksi kesiapan panen tanaman cabai rawit. *Backend* yang telah *dideploy* pada *Railway* berfungsi untuk menerima citra dari aplikasi *Android Chilivest*, menjalankan proses pengolahan citra dan klasifikasi, kemudian mengirimkan hasil prediksi kembali kepada aplikasi *Android* melalui *REST API*. Dengan demikian, *Railway* berperan sebagai media *deployment* agar *backend Flask* dapat diakses secara *online* oleh aplikasi *Android*.

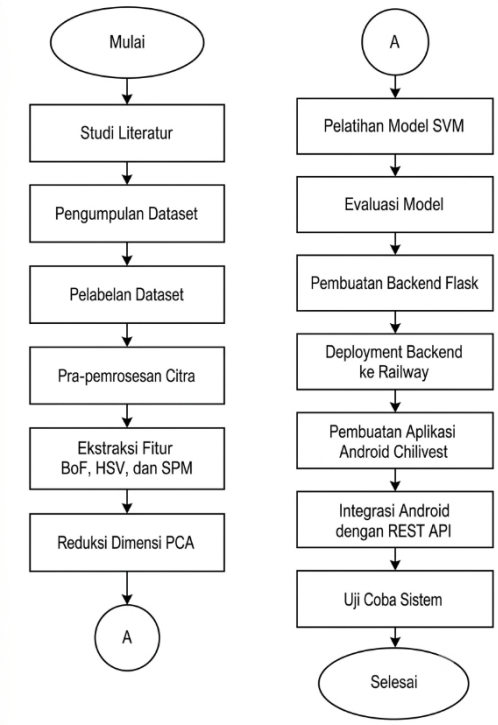
BAB III
METODOLOGI PENELITIAN

3.1 Jenis Penelitian

Jenis penelitian yang dilakukan dalam karya ilmiah ini adalah penelitian terapan (applied research) dengan pendekatan rekayasa sistem dan eksperimen. Penelitian ini bertujuan untuk merancang dan membangun sistem deteksi kesiapan panen tanaman cabai rawit berbasis citra digital menggunakan kombinasi metode *Bag-of-Features* (*BoF*), Hue Saturation Value (*HSV*), *Spatial Pyramid Matching* (*SPM*), *Principal Component Analysis* (*PCA*), dan *Support Vector Machine* (*SVM*). Sistem yang dikembangkan berupa aplikasi *Android Chilivest* yang terhubung dengan *backend Flask* melalui *REST API* untuk melakukan proses prediksi pada sisi *server*.

3.2 Tahapan Penelitian

Pada sub bab ini dijelaskan tahapan penelitian yang dilakukan secara berurutan mulai dari studi literatur hingga uji coba sistem. Tahapan penelitian disusun agar proses perancangan sistem, pembuatan model klasifikasi, pengembangan *backend*, serta integrasi aplikasi *Android* dapat dilakukan secara terarah. Adapun tahapan penelitian yang dilakukan ditunjukkan pada Gambar 3.1.



Gambar 3. 1 *Flowchart Tahapan Penelitian*

1. Studi Literatur

   Tahap awal penelitian dilakukan dengan mencari dan mempelajari referensi yang berkaitan dengan topik penelitian. Referensi yang dipelajari meliputi tanaman cabai rawit, kesiapan panen, pengolahan citra digital, *Bag-of-Features*, *HSV*, *Spatial Pyramid Matching*, *PCA*, *SVM*, aplikasi *Android*, *Flask*, *REST API*, dan *Railway*. Studi literatur digunakan sebagai dasar dalam menentukan metode, alur kerja sistem, serta parameter pengujian yang digunakan dalam penelitian.

2. Pengumpulan *Dataset*

   Pada tahap ini dilakukan pengumpulan *dataset* berupa citra tanaman cabai rawit. Citra yang dikumpulkan digunakan sebagai data utama dalam proses pelatihan dan pengujian model. *Dataset* diperoleh berdasarkan kondisi visual tanaman cabai rawit yang menunjukkan perbedaan tingkat kesiapan panen.

3. Pelabelan *Dataset*

Dataset yang telah dikumpulkan kemudian diberi label sesuai dengan kelas yang digunakan dalam penelitian. Kelas tersebut terdiri dari Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Pelabelan dilakukan agar setiap citra memiliki kategori yang jelas sebelum digunakan pada proses pelatihan model.

4. Pra-pemrosesan Citra

Citra yang telah diberi label kemudian melalui tahap *pra-pemrosesan*. Tahap ini dilakukan untuk menyesuaikan data citra sebelum masuk ke proses ekstraksi fitur. Pra-pemrosesan citra meliputi penyesuaian ukuran citra dan penyiapan format citra agar seluruh data memiliki bentuk masukan yang seragam.

5. Ekstraksi Fitur *BoF*, *HSV*, dan *SPM*

Tahap ekstraksi fitur dilakukan untuk memperoleh informasi visual dari citra tanaman cabai rawit. *Bag-of-Features* digunakan untuk merepresentasikan fitur lokal pada citra, *HSV* digunakan untuk memperoleh informasi warna, sedangkan *Spatial Pyramid Matching* digunakan untuk mempertahankan informasi spasial atau sebaran fitur pada citra. Hasil dari tahap ini berupa vektor fitur yang digunakan sebagai masukan pada proses berikutnya.

6. Reduksi Dimensi *PCA*

Fitur hasil ekstraksi kemudian direduksi menggunakan *Principal Component Analysis*. Tahap ini dilakukan karena gabungan fitur *BoF*, *HSV*, dan *SPM* menghasilkan dimensi fitur yang cukup besar. Dengan menggunakan *PCA*, fitur yang digunakan pada proses klasifikasi menjadi lebih ringkas tanpa menghilangkan informasi utama dari citra.

7. Pelatihan Model *SVM*

Fitur yang telah melalui proses reduksi dimensi digunakan untuk melatih model *Support Vector Machine*. Model *SVM* dilatih agar dapat mengenali pola fitur dari masing-masing kelas kesiapan panen. Hasil dari tahap ini adalah model klasifikasi yang mampu mengelompokkan citra tanaman cabai

rawit ke dalam kelas Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen.

8. Evaluasi Model

   Model yang telah dilatih kemudian dievaluasi untuk mengetahui kinerja klasifikasi. Evaluasi dilakukan menggunakan *data uji* yang belum digunakan pada proses pelatihan. Parameter evaluasi yang digunakan meliputi *accuracy*, *precision*, *recall*, *F1-score*, *confusion matrix*, dan waktu prediksi.

9. Pembuatan *Backend Flask*

   Setelah model klasifikasi diperoleh, tahap selanjutnya adalah pembuatan *backend* menggunakan *Flask*. *Backend* berfungsi untuk menerima citra yang dikirim dari aplikasi *Android*, menjalankan proses pengolahan citra dan klasifikasi, serta mengirimkan hasil prediksi kembali ke aplikasi.

10. *Deployment Backend* ke *Railway*

    Backend Flask yang telah dibuat kemudian *dideploy* ke *Railway* agar dapat diakses secara *online*. Tahap ini dilakukan supaya aplikasi *Android* dapat terhubung dengan *backend* melalui jaringan internet. Dengan adanya *deployment*, proses prediksi dapat dilakukan pada sisi *server*.

11. Pembuatan Aplikasi *Android Chilivest*

    Pada tahap ini dilakukan pembuatan aplikasi *Android* bernama *Chilivest*. Aplikasi ini berfungsi sebagai antarmuka pengguna untuk mengambil atau memilih citra tanaman cabai rawit. Selain itu, aplikasi juga digunakan untuk menampilkan hasil prediksi kesiapan panen kepada pengguna.

12. Integrasi *Android* dengan *REST API*

    Aplikasi *Android Chilivest* kemudian diintegrasikan dengan *backend Flask* melalui *REST API*. Pada tahap ini, aplikasi mengirimkan citra tanaman cabai rawit ke *server* dan menerima hasil prediksi dalam format *JSON*. Integrasi ini menjadi penghubung antara aplikasi *Android* sebagai *client* dan *backend Flask* sebagai *server*.

13. Uji Coba Sistem

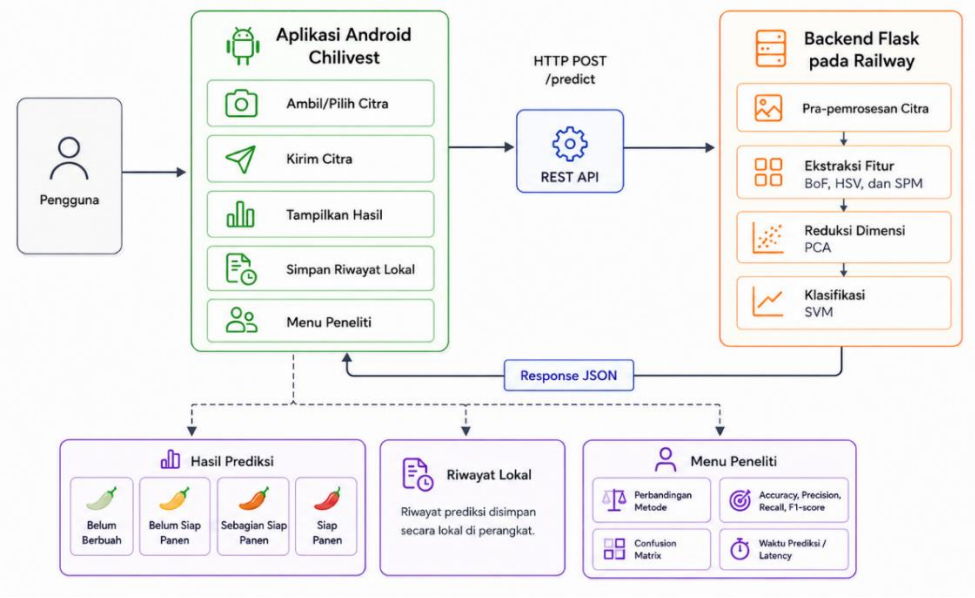
Tahap terakhir adalah uji coba sistem secara keseluruhan. Uji coba dilakukan untuk memastikan aplikasi *Android* dapat mengirim citra, *backend* dapat memproses citra, model dapat menghasilkan prediksi, dan hasil prediksi dapat ditampilkan pada aplikasi. Tahap ini dilakukan untuk memastikan sistem berjalan sesuai dengan rancangan yang telah dibuat.

3.3 Perancangan Sistem

Perancangan sistem menjelaskan Gambaran umum alur kerja sistem deteksi kesiapan panen tanaman cabai rawit. Sistem ini dirancang menggunakan aplikasi *Android Chilivest* sebagai *client*, *backend Flask* sebagai *server*, dan model klasifikasi untuk memproses citra tanaman cabai rawit.

3.3.1 Blok Diagram Sistem

Blok diagram sistem digunakan untuk mengGambarkan hubungan antar komponen utama pada sistem deteksi kesiapan panen tanaman cabai rawit. Sistem ini terdiri dari pengguna, aplikasi *Android Chilivest*, *REST API*, *backend Flask* pada *Railway*, hasil prediksi, riwayat lokal, dan menu peneliti. Blok diagram sistem ditunjukkan pada Gambar 3.2.



Gambar 3. 2 *Blok Diagram Sistem*

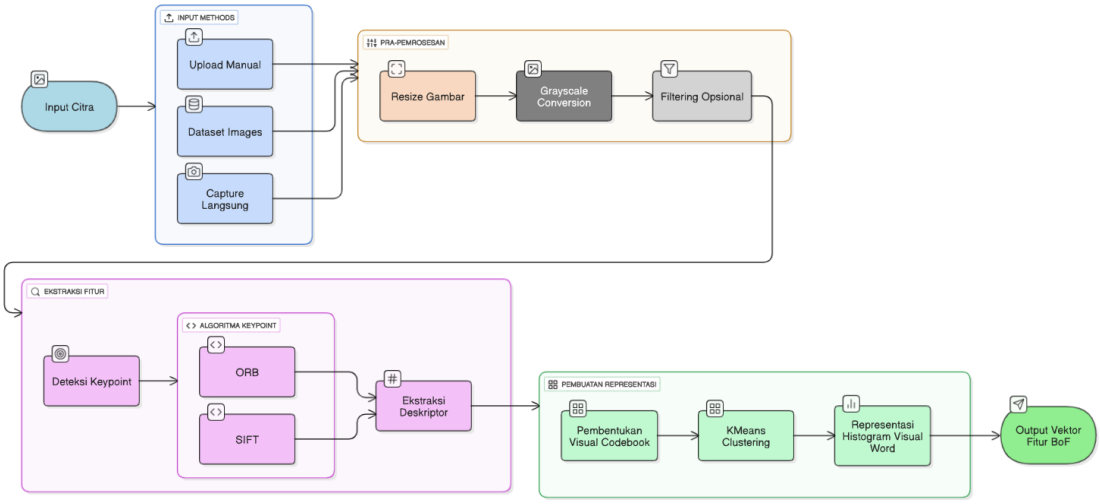
Pada Gambar 3.2, pengguna menggunakan aplikasi *Android Chilivest* untuk mengambil citra tanaman cabai rawit melalui kamera atau memilih citra dari galeri. Citra yang telah dipilih kemudian dikirimkan dari aplikasi *Android* ke *backend Flask* melalui *REST API* menggunakan metode *HTTP POST* pada *endpoint* /predict. Aplikasi *Android* berperan sebagai *client* yang menyediakan antarmuka pengguna, mengirim citra, menerima hasil prediksi, menampilkan hasil, menyimpan riwayat prediksi secara lokal, serta menyediakan menu peneliti.

Backend Flask yang *dideploy* pada *Railway* berperan sebagai *server* untuk memproses citra yang dikirimkan oleh aplikasi *Android*. Proses yang dilakukan pada *backend* meliputi *pra-pemrosesan* citra, ekstraksi fitur menggunakan *Bag-of-Features*, *HSV*, dan *Spatial Pyramid Matching*, reduksi dimensi menggunakan *PCA*, serta klasifikasi menggunakan *SVM*. Setelah proses klasifikasi selesai, *backend* mengirimkan hasil prediksi kembali ke aplikasi *Android* dalam bentuk *response JSON*.

Hasil prediksi yang diterima oleh aplikasi kemudian ditampilkan kepada pengguna dalam empat kelas, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Selain ditampilkan pada halaman hasil prediksi, data hasil prediksi juga dapat disimpan pada riwayat lokal di perangkat *Android*. Penyimpanan riwayat lokal digunakan agar pengguna dapat melihat kembali hasil prediksi yang pernah dilakukan pada perangkat yang sama.

Selain fitur prediksi dan riwayat, aplikasi *Chilivest* juga menyediakan menu peneliti. Menu ini digunakan untuk menampilkan informasi yang berkaitan dengan penelitian, seperti perbandingan metode, nilai *accuracy*, *precision*, *recall*, *F1-score*, *confusion matrix*, serta waktu prediksi atau *latency*. Dengan adanya menu peneliti, sistem tidak hanya menampilkan hasil prediksi kepada pengguna, tetapi juga menyediakan informasi pendukung untuk melihat kinerja metode yang digunakan dalam penelitian.

1. *Bag of Features* (*BoF*)



Gambar 3. 3 *Blok Diagram Bag of Features*

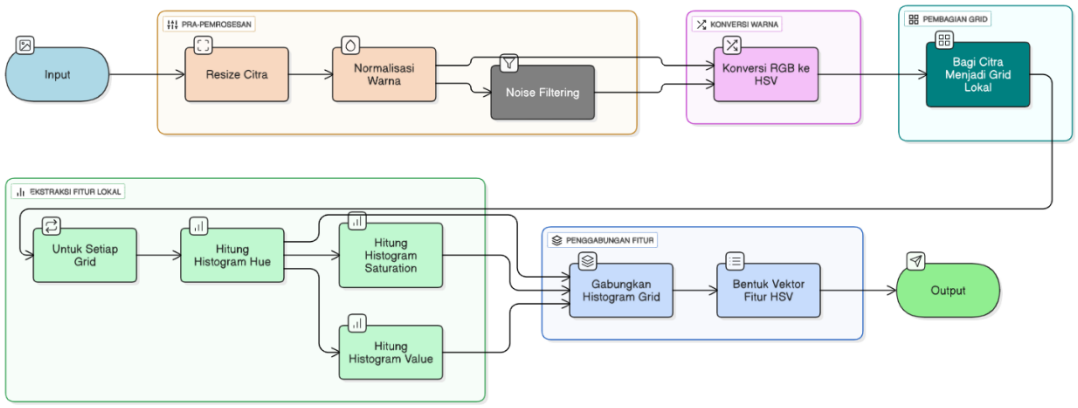
Metode ini terdiri dari empat tahapan utama, yaitu *input* citra, *pra-pemrosesan*, ekstraksi fitur, dan pembentukan representasi fitur. Citra dapat dimasukkan melalui unggahan manual, *dataset*, atau tangkapan langsung kamera.

Pada tahap *pra-pemrosesan*, citra diubah ukurannya, dikonversi ke *grayscale*, dan difilter untuk mengurangi *noise*. Tujuannya adalah menyiapkan citra agar hasil ekstraksi fitur lebih optimal.

Ekstraksi fitur dilakukan dengan mendeteksi *keypoint* menggunakan algoritma *ORB* atau *SIFT*, lalu dilakukan ekstraksi deskriptor untuk menghasilkan representasi numerik dari area sekitar titik penting tersebut.

Tahap akhir adalah pembentukan representasi, di mana fitur yang telah diekstraksi dikelompokkan menggunakan *KMeans clustering* untuk membentuk *visual codebook*. *Histogram* frekuensi kemunculan *visual words* disusun menjadi *vektor fitur BoF*, yang menjadi *output* akhir metode ini dan siap digunakan dalam proses klasifikasi.

2. Hue Saturation Value (*HSV*)



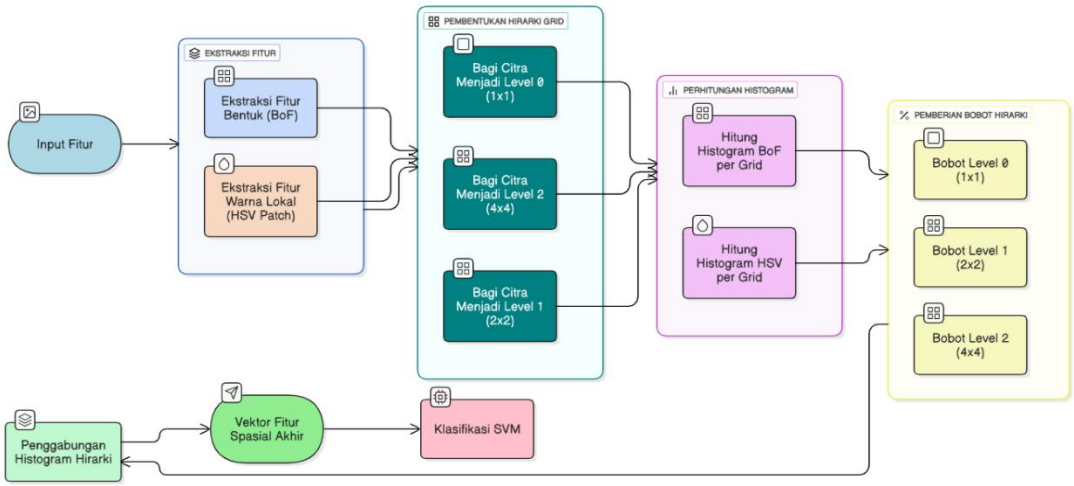
Gambar 3. 4 *Blok Diagram Hue Saturation Value*

Metode ini bertujuan untuk mengekstraksi fitur warna dari citra menggunakan ruang warna *HSV* (*Hue, Saturation, Value*). Proses dimulai dengan *input* citra yang diperoleh dari *dataset* atau tangkapan langsung kamera. Citra kemudian dipra-proses melalui *resize*, *normalisasi* warna, dan penghilangan *noise* agar siap diproses lebih lanjut.

Selanjutnya, citra dikonversi dari *RGB* ke *HSV* karena *HSV* lebih stabil terhadap pencahayaan dan lebih sesuai dengan persepsi warna manusia. *Citra HSV* lalu dibagi menjadi beberapa *grid* lokal agar analisis warna dapat dilakukan secara detail pada tiap bagian.

Setiap *grid* dihitung histogramnya untuk masing-masing komponen *HSV—hue*, *saturation*, dan *value*—yang mencerminkan sebaran rona, kejenuhan, dan kecerahan. *Histogram* dari seluruh *grid* kemudian digabungkan menjadi satu vektor fitur *HSV*. Vektor ini mewakili informasi warna dalam citra dan digunakan sebagai *input* dalam proses klasifikasi atau pengenalan objek.

3. *Spatial Pyramid Matching* (*SPM*)



Gambar 3. 5 *Blok Diagram Spatial Pyramid Matching*

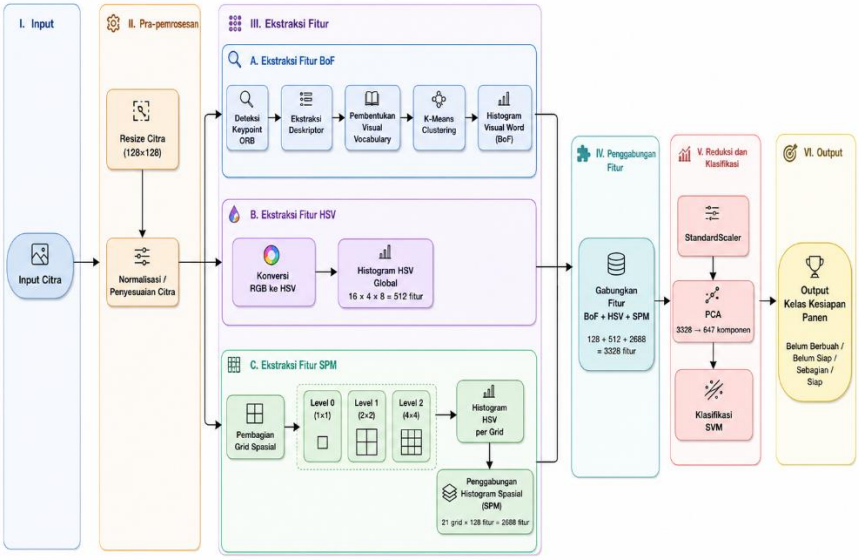
Metode *Spatial Pyramid Matching* (*SPM)* bertujuan menggabungkan informasi spasial dan fitur lokal untuk meningkatkan performa klasifikasi citra. Proses diawali dengan *input* fitur yang diperoleh dari dua jenis ekstraksi, yaitu fitur bentuk menggunakan *Bag of Features* (*BoF*) dan fitur warna lokal berbasis *patch HSV*.

Citra kemudian dibagi ke dalam tiga *level* hierarki spasial. *Level* 0 membagi citra menjadi satu bagian utuh (1×1), *level* 1 menjadi 2×2 *grid*, dan *level* 2 menjadi 4×4 *grid*. Pembagian ini memungkinkan sistem menangkap informasi dari *global* hingga detail lokal.

Untuk setiap *grid* di masing-masing *level*, dilakukan perhitungan *histogram* berdasarkan dua jenis fitur: *histogram BoF* untuk informasi bentuk dan *histogram HSV* untuk informasi warna. Setelah itu, setiap *level* diberikan bobot yang berbeda untuk menyeimbangkan kontribusi fitur dari skala *global* dan lokal.

Histogram dari seluruh *grid* dan *level* digabungkan menjadi satu vektor fitur spasial akhir, yang merepresentasikan keseluruhan karakteristik citra. Vektor ini kemudian digunakan dalam proses klasifikasi menggunakan algoritma *Support Vector Machine* (*SVM*).

4. Blok Diagram Kombinasi Metode



Gambar 3. 6 Blok Diagram Kombinasi Metode

Blok diagram kombinasi metode yang terdiri dari metode *BoF* + *HSV* + *SPM* + *PCA* + *SVM* menunjukkan alur proses klasifikasi kesiapan panen cabai rawit berbasis citra. Proses diawali dari *input* citra tanaman cabai rawit, kemudian citra masuk ke tahap *pra-pemrosesan* berupa *resize* citra menjadi 128 × 128 piksel dan *normalisasi* atau penyesuaian citra. Tahap ini dilakukan agar seluruh citra memiliki ukuran dan kondisi data yang seragam sebelum dilakukan ekstraksi fitur.

Setelah *pra-pemrosesan*, citra diproses melalui tiga jalur ekstraksi fitur, yaitu *BoF*, *HSV*, dan *SPM*. Pada jalur *BoF*, citra diproses menggunakan *ORB* untuk mendeteksi *keypoint* dan menghasilkan deskriptor, kemudian dilakukan pembentukan *visual vocabulary* menggunakan *K-Means* hingga menghasilkan *histogram visual word*. Pada jalur *HSV*, citra dikonversi ke ruang warna *HSV* dan dibentuk *histogram HSV global* dengan dimensi 512 fitur. Sementara itu, pada jalur *SPM*, citra dibagi ke dalam beberapa *level grid* spasial, yaitu *level* 0, *level* 1, dan *level* 2, kemudian *histogram* pada setiap *grid* digabungkan hingga menghasilkan fitur *SPM* berdimensi 2688 fitur.

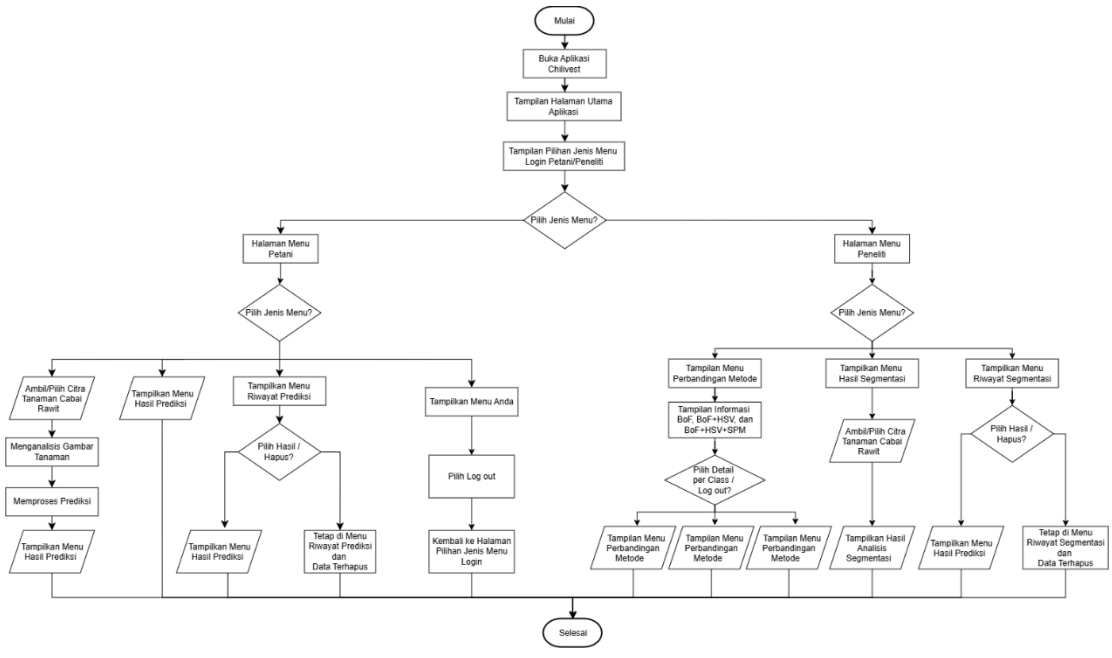
Fitur dari ketiga metode tersebut kemudian digabungkan menjadi satu vektor fitur, yaitu *BoF* 128 fitur, *HSV* 512 fitur, dan *SPM* 2688 fitur, sehingga menghasilkan total

3328 fitur. Selanjutnya, fitur gabungan dinormalisasi menggunakan *StandardScaler* dan direduksi menggunakan *PCA* menjadi 647 komponen. Hasil reduksi fitur tersebut kemudian diklasifikasikan menggunakan *SVM* untuk menghasilkan kelas kesiapan panen cabai rawit, yaitu Belum Berbuah, Belum Siap, Sebagian, dan Siap.

3.3.2 Flowchart

Flowchart sistem digunakan untuk mengGambarkan alur kerja sistem deteksi kesiapan panen tanaman cabai rawit pada aplikasi *Chilivest* secara keseluruhan. Flowchart ini menjelaskan hubungan antara alur penggunaan aplikasi *Android*, proses prediksi pada *backend Flask*, serta tahapan metode yang digunakan dalam sistem. Agar alur lebih mudah dipahami, flowchart sistem dibagi menjadi beberapa bagian, yaitu Flowchart Aplikasi *Chilivest*, Flowchart Proses Prediksi pada *Backend Flask*, Flowchart Ekstraksi Fitur *BoF*, *HSV*, dan *SPM*, serta Flowchart Reduksi Dimensi *PCA* dan Klasifikasi *SVM*. Pembagian tersebut dilakukan agar setiap proses dapat dijelaskan secara lebih terarah dan tidak terlalu padat dalam satu flowchart.

1. Flowchart Aplikasi *Chilivest*



Gambar 3. 7 Flowchart Aplikasi *Chilivest*

Berdasarkan Gambar 3.7, proses dimulai ketika pengguna membuka aplikasi Chilivest. Sistem menampilkan halaman utama dan halaman pilihan jenis menu *login*, yaitu Menu Petani dan Menu Peneliti. Pengguna kemudian memilih menu sesuai kebutuhan.

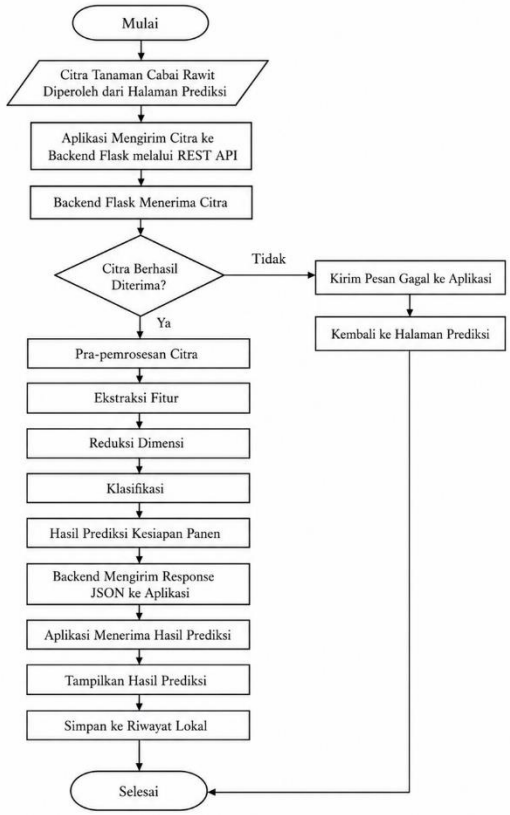
Apabila pengguna memilih Menu Petani, sistem menampilkan halaman Menu Petani yang berisi fitur Ambil/Pilih Citra Tanaman Cabai Rawit, Hasil Prediksi, Riwayat Prediksi, dan Menu Anda. Fitur Ambil/Pilih Citra digunakan untuk mengambil atau memilih citra tanaman cabai rawit, kemudian sistem memproses citra melalui *backend* dan menampilkan hasil pada Menu Hasil Prediksi. Selain itu, pengguna dapat melihat kembali data pada Riwayat Prediksi atau memilih *Log Out* melalui Menu Anda.

Apabila pengguna memilih Menu Peneliti, sistem menampilkan halaman Menu Peneliti yang terdiri dari Menu Perbandingan Metode, Menu Hasil Segmentasi, dan Menu Riwayat Segmentasi. Menu Perbandingan Metode digunakan untuk menampilkan evaluasi metode BoF, BoF + HSV, dan BoF + HSV + SPM, seperti nilai *accuracy*, *precision*, *recall*, *F1-score*, detail *per class*, dan *confusion matrix*.

Menu Hasil Segmentasi digunakan untuk menampilkan analisis tambahan terhadap citra tanaman cabai rawit. Hasil yang ditampilkan meliputi citra *input resize*, *mask* kandidat cabai, *overlay* segmentasi, hasil prediksi, proporsi warna HSV seluruh citra, proporsi warna HSV area kandidat cabai, alasan, dan rekomendasi. Hasil segmentasi ini hanya digunakan sebagai informasi pendukung dan tidak menjadi penentu kelas prediksi SVM.

Menu Riwayat Segmentasi digunakan untuk menampilkan kembali data hasil segmentasi yang telah tersimpan. Pengguna dapat memilih salah satu hasil segmentasi untuk dilihat kembali atau menghapus data tertentu. Secara keseluruhan, *flowchart* ini menunjukkan dua alur utama aplikasi Chilivest, yaitu alur Petani untuk prediksi dan riwayat prediksi, serta alur Peneliti untuk evaluasi metode, hasil segmentasi, dan riwayat segmentasi.

2. Flowchart Proses Prediksi pada *Backend Flask*



Gambar 3. 8 Flowchart Proses Prediksi pada *Backend Flask*

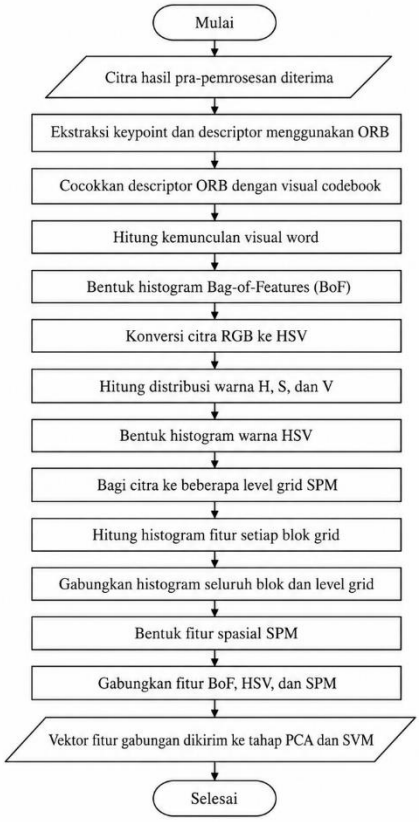
Berdasarkan Gambar 3.8, proses dimulai ketika citra tanaman cabai rawit diperoleh dari halaman prediksi pada aplikasi *Chilivest*. Citra tersebut dapat berasal dari kamera perangkat atau galeri yang dipilih oleh pengguna. Setelah citra diperoleh, aplikasi mengirimkan citra ke *backend Flask* melalui *REST API* agar proses prediksi dapat dilakukan pada sisi *server*.

Backend Flask kemudian menerima citra yang dikirimkan oleh aplikasi. Pada tahap ini, sistem melakukan pengecekan apakah citra berhasil diterima atau tidak. Jika citra tidak berhasil diterima, maka *backend* akan mengirimkan pesan gagal ke aplikasi dan pengguna diarahkan kembali ke halaman prediksi. Kondisi ini digunakan untuk mengantisipasi apabila citra tidak terkirim, format citra tidak sesuai, atau terjadi gangguan pada proses pengiriman data.

Jika citra berhasil diterima, maka *backend* melanjutkan proses prediksi. Proses tersebut meliputi *pra-pemrosesan* citra, ekstraksi fitur, reduksi dimensi, dan klasifikasi. Tahap *pra-pemrosesan* digunakan untuk menyesuaikan citra agar dapat diproses oleh sistem. Setelah itu, citra diproses menggunakan tahapan metode yang telah dirancang, yaitu ekstraksi fitur *BoF*, *HSV*, dan *SPM*, reduksi dimensi *PCA*, serta klasifikasi menggunakan *SVM*.

Setelah proses klasifikasi selesai, *backend* menghasilkan kelas prediksi kesiapan panen tanaman cabai rawit. Hasil prediksi kemudian dikirimkan kembali ke aplikasi *Android* dalam bentuk *response JSON*. Aplikasi *Chilivest* menerima hasil tersebut, menampilkannya kepada pengguna pada halaman hasil prediksi, dan menyimpan hasil prediksi ke riwayat lokal pada perangkat.

3. Flowchart Ekstraksi Fitur *BoF*, *HSV*, dan *SPM*



Gambar 3. 9 Flowchart Ekstraksi Fitur *BoF, HSV,* dan *SPM*

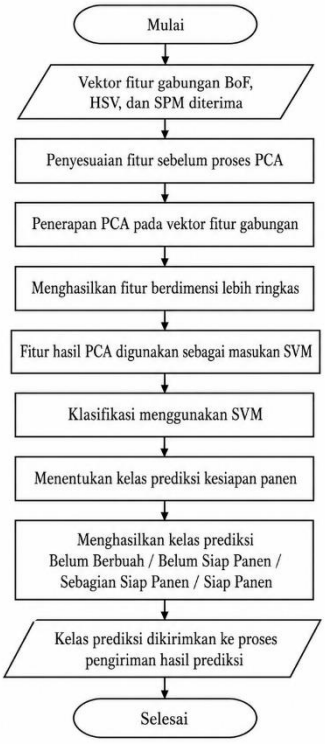
Berdasarkan Gambar 3.9, proses dimulai ketika citra hasil *pra-pemrosesan* diterima oleh sistem. Citra tersebut kemudian diproses untuk memperoleh fitur lokal menggunakan *ORB*. Pada tahap ini, sistem melakukan ekstraksi *keypoint* dan *descriptor* dari citra. *Descriptor* hasil *ORB* selanjutnya dicocokkan dengan *visual codebook* yang telah dibentuk sebelumnya, kemudian sistem menghitung kemunculan *visual word* untuk membentuk *histogram Bag-of-Features* (*BoF*).

Setelah fitur *BoF* diperoleh, sistem melakukan ekstraksi fitur warna. Citra dikonversi dari ruang warna *RGB* ke *HSV*, kemudian sistem menghitung distribusi warna pada kanal H, S, dan V. Hasil dari proses ini digunakan untuk membentuk *histogram* warna *HSV* yang merepresentasikan informasi warna pada citra tanaman cabai rawit.

Tahap berikutnya adalah ekstraksi fitur spasial menggunakan *Spatial Pyramid Matching*. Pada tahap ini, citra dibagi ke dalam beberapa *level grid*. Setiap blok *grid* dihitung *histogram* fiturnya, kemudian *histogram* dari seluruh blok dan seluruh *level grid* digabungkan untuk membentuk fitur spasial *SPM*. Tahapan ini digunakan agar sistem tidak hanya memperoleh informasi fitur, tetapi juga mempertahankan informasi sebaran fitur pada area citra.

Setelah fitur *BoF*, *HSV*, dan *SPM* diperoleh, sistem menggabungkan seluruh fitur tersebut menjadi satu vektor fitur gabungan. Vektor fitur gabungan ini kemudian dikirimkan ke tahap berikutnya, yaitu reduksi dimensi menggunakan *PCA* dan klasifikasi menggunakan *SVM*. Dengan demikian, flowchart ini menjelaskan proses pembentukan fitur utama sebelum data masuk ke tahap klasifikasi.

4. Flowchart Reduksi Dimensi *PCA* dan Klasifikasi *SVM*



Gambar 3. 10 *Flowchart Reduksi Dimensi PCA dan Klasifikasi SVM*

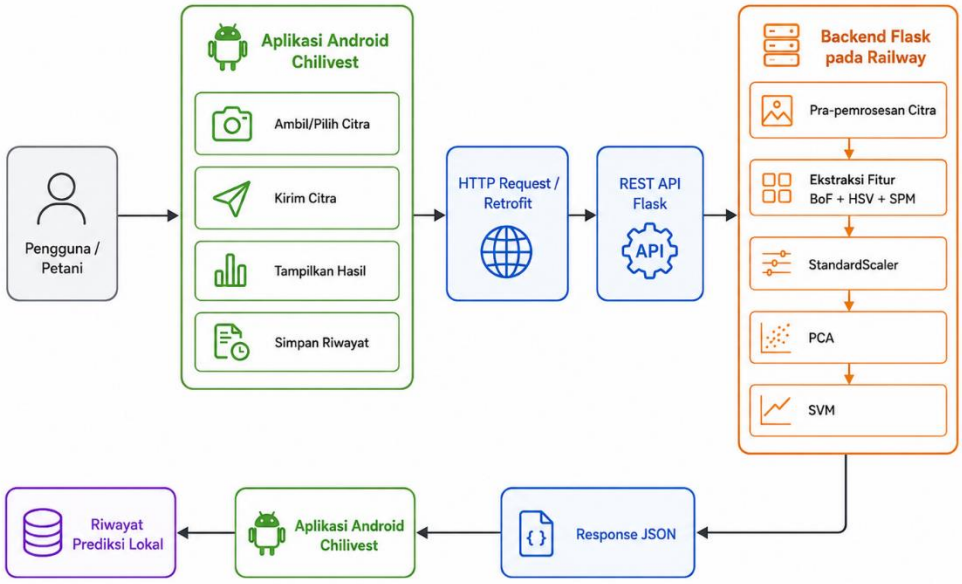
Berdasarkan Gambar 3.10, proses dimulai ketika vektor fitur gabungan *BoF*, *HSV*, dan *SPM* diterima oleh sistem. Vektor fitur tersebut kemudian melalui tahap penyesuaian fitur sebelum proses *PCA*. Tahap ini dilakukan agar data fitur memiliki bentuk yang sesuai sebelum masuk ke proses reduksi dimensi.

Setelah fitur disesuaikan, sistem menerapkan *PCA* pada vektor fitur gabungan. *PCA* digunakan untuk mereduksi dimensi fitur sehingga fitur yang digunakan menjadi lebih ringkas, tetapi tetap mempertahankan informasi utama dari citra. Fitur hasil *PCA* kemudian digunakan sebagai masukan pada model *SVM*.

Pada tahap klasifikasi, *SVM* digunakan untuk menentukan kelas prediksi kesiapan panen tanaman cabai rawit berdasarkan fitur yang telah direduksi. Sistem kemudian menghasilkan salah satu kelas prediksi, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, atau Siap Panen. Kelas prediksi tersebut selanjutnya dikirimkan ke proses pengiriman hasil prediksi agar dapat ditampilkan pada aplikasi *Chilivest*.

3.3.3 Blok Diagram Arsitektur *Client-Server*

Sistem deteksi kesiapan panen tanaman cabai rawit yang dikembangkan menggunakan arsitektur *client-server*. Pada arsitektur ini, aplikasi *Android Chilivest* berperan sebagai *client* yang digunakan oleh pengguna atau petani untuk mengambil, memilih, dan mengirimkan citra tanaman cabai rawit. Sementara itu, *backend Flask* yang *dideploy* pada *Railway* berperan sebagai *server* yang bertugas menerima citra, menjalankan proses pengolahan citra, melakukan klasifikasi menggunakan model yang telah dilatih, serta mengirimkan hasil prediksi kembali ke aplikasi *Android* dalam bentuk *response JSON*. Dengan pendekatan ini, proses komputasi model tidak dilakukan langsung pada perangkat *Android*, melainkan diproses pada *backend server*.



Gambar 3. 11 Blok Diagram Arsitektur *Client-Server*

Gambar 3.11 menunjukkan arsitektur *client-server* pada sistem deteksi kesiapan panen tanaman cabai rawit. Pada sistem ini, aplikasi *Android Chilivest* berperan sebagai *client* yang digunakan oleh pengguna atau petani untuk mengambil atau memilih citra tanaman cabai rawit. Citra tersebut kemudian dikirimkan ke *backend Flask* melalui *REST API* menggunakan *HTTP request*.

Pengiriman citra dilakukan melalui *endpoint* /predict dengan metode *POST* dan format *multipart/form-data*, menggunakan *field image* sebagai *input* Gambar. *Backend*

Flask yang berjalan pada *Railway* menerima citra tersebut, kemudian melakukan *pra-pemrosesan* berupa validasi Gambar dan penyesuaian ukuran citra menjadi 128 × 128 piksel.

Setelah citra diproses, *backend* melakukan ekstraksi fitur menggunakan kombinasi *BoF*, *HSV*, dan *SPM*. Fitur yang dihasilkan kemudian melalui tahap *StandardScaler*, reduksi dimensi menggunakan *PCA*, dan klasifikasi menggunakan *SVM*. Hasil klasifikasi selanjutnya dikemas dalam bentuk *response JSON* yang berisi prediksi kesiapan panen, nilai *confidence*, estimasi waktu panen, persentase warna, alasan prediksi, dan rekomendasi.

Response JSON yang diterima aplikasi *Android* ditampilkan kepada pengguna sebagai hasil deteksi kesiapan panen. Selain itu, hasil prediksi juga dapat disimpan ke dalam riwayat prediksi lokal pada perangkat *Android*. Dengan arsitektur ini, proses pengolahan citra dan klasifikasi dilakukan pada *backend server*, sedangkan aplikasi *Android* berfungsi sebagai antarmuka pengguna untuk mengambil citra, mengirim data, menerima hasil, dan menyimpan riwayat prediksi

3.3.4 Perancangan *REST API* dan *Deployment Backend*

Perancangan *REST API* dan *deployment backend* dilakukan untuk menghubungkan aplikasi *Android Chilivest* dengan *backend Flask* yang menjalankan model deteksi kesiapan panen tanaman cabai rawit. *REST API* berfungsi sebagai penghubung antara aplikasi *Android* sebagai *client* dan *backend Flask* sebagai *server*. Melalui *REST API*, aplikasi *Android* dapat mengirimkan citra tanaman cabai rawit ke *server*, kemudian *server* memproses citra tersebut menggunakan model klasifikasi yang telah dilatih. Hasil prediksi dikirimkan kembali ke aplikasi *Android* dalam bentuk JavaScript Object Notation (*JSON*) agar dapat ditampilkan kepada pengguna.

Komunikasi pada *REST API* sistem Chilivest menggunakan pola *request* dan *response* melalui protokol *HTTP*. Aplikasi Android berperan sebagai *client* yang mengirimkan permintaan data ke *backend*, sedangkan *backend* Flask berperan sebagai *server* yang menerima *request*, menjalankan proses sesuai *endpoint*, dan mengirimkan *response* kembali ke aplikasi. Pada proses prediksi, aplikasi Android mengirimkan

citra tanaman cabai rawit ke *endpoint* /predict menggunakan *method* POST dengan format multipart/form-data. *File* citra dikirim menggunakan *field* image, kemudian *backend* Flask melakukan validasi citra, *resize* citra, ekstraksi fitur BoF, HSV, dan SPM, normalisasi fitur, reduksi dimensi PCA, serta klasifikasi menggunakan SVM. Setelah proses selesai, *backend* mengirimkan *response* dalam format JSON yang berisi hasil prediksi, nilai *confidence*, persentase warna, status tanaman, alasan prediksi, dan rekomendasi. *Response* tersebut kemudian diterima oleh aplikasi Android dan ditampilkan pada halaman hasil prediksi.

REST API pada sistem ini dirancang dengan beberapa *endpoint* yang memiliki fungsi berbeda. *Endpoint* tersebut digunakan untuk mengecek status *backend*, melakukan prediksi kesiapan panen, serta mengambil data hasil evaluasi metode yang ditampilkan pada menu peneliti.

Tabel 3. 1 *Endpoint API*

No.	*Endpoint*	*Method*	Fungsi	*Input*	*Output*
1.	/health	*GET*	Mengecek status *backend*, status model, dan konfigurasi *pipeline*	Tidak ada	Status *backend*, status model, *metadata* model, dan informasi *pipeline*
2.	/predict	*POST*	Melakukan prediksi kesiapan panen tanaman cabai rawit berdasarkan citra	File citra tanaman cabai rawit dengan *field image*	Hasil prediksi dalam bentuk *JSON*
3.	/method-comparison	*GET*	Mengambil data hasil evaluasi perbandingan metode	Tidak ada	Data evaluasi metode *BoF*, *BoF + HSV*, dan *BoF + HSV + SPM*
4.	/predict-segmentation	*POST*	Melakukan prediksi dan	File citra tanaman cabai	Hasil prediksi, proporsi warna,

			menampilkan hasil segmentasi sebagai informasi visual tambahan	rawit dengan *field* image	citra *input resize*, *mask*, dan *overlay* segmentasi dalam format JSON

Endpoint /predict merupakan *endpoint* utama pada sistem ini karena digunakan oleh aplikasi *Android* untuk mengirimkan citra tanaman cabai rawit ke *backend Flask*. Pengiriman citra dilakukan menggunakan metode *POST* dengan format *multipart/form-data*, sehingga file Gambar dapat dikirim dan diterima oleh *backend*. Setelah citra diproses, *backend* mengembalikan hasil prediksi dalam bentuk *JSON* yang berisi informasi hasil klasifikasi dan rekomendasi kepada pengguna.

Selain *endpoint* /predict, *endpoint* /health digunakan sebagai komunikasi awal untuk memastikan *backend*, model, dan konfigurasi *pipeline* dalam keadaan aktif. *Endpoint* /method-comparison digunakan oleh menu peneliti untuk mengambil data evaluasi metode dari *backend*. *Endpoint* /predict-segmentation digunakan pada menu Hasil Segmentasi untuk menampilkan hasil prediksi beserta informasi visual tambahan berupa citra *input resize*, *mask* kandidat cabai, dan *overlay* segmentasi. Dengan rancangan tersebut, *REST API* tidak hanya berfungsi untuk proses prediksi, tetapi juga mendukung pengecekan status *server* dan pengiriman data evaluasi metode ke aplikasi Android.

Tabel 3. 2 *Format Request Endpoint* /predict

Bagian	Komponen	Keterangan
Request	*Method*	*POST*
Request	*Endpoint*	/predict
Request.	*Content-Type*	*multipart/form-data*
Request	*Field*	*image*
Request	Tipe Data	File citra tanaman cabai rawit
Response	Format Data	*JSON*
Response	Isi *Response*	*prediction*, *confidence*, *estimated_harvest*, *description*,

		green_percentage, *orange_percentage*, *red_percentage*, *plant_status*, *reasons*, dan *recommendation*
Response	Fungsi	Menampilkan hasil prediksi, estimasi panen, persentase warna, alasan prediksi, dan rekomendasi pada aplikasi *Android*

Pada sisi *backend*, citra yang diterima melalui *endpoint* /predict akan melalui beberapa tahapan pemrosesan. Tahapan tersebut meliputi validasi citra, penyesuaian ukuran citra, ekstraksi fitur menggunakan kombinasi *Bag-of-Features* (*BoF*), *HSV*, dan *Spatial Pyramid Matching* (*SPM*), *normalisasi* fitur menggunakan *StandardScaler*, reduksi dimensi menggunakan *Principal Component Analysis* (*PCA*), serta klasifikasi menggunakan *Support Vector Machine* (*SVM*). Hasil klasifikasi kemudian dikemas dalam bentuk *response JSON* dan dikirimkan kembali ke aplikasi *Android*.

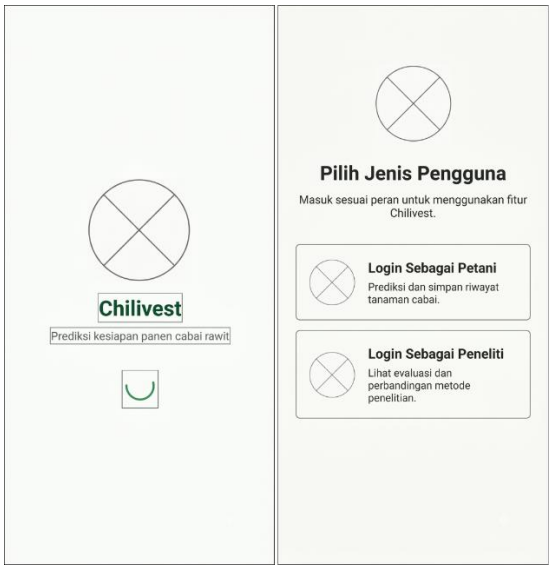
Backend Flask dirancang menggunakan beberapa file utama agar dapat berjalan dengan baik pada *server deployment*. File *app.py* digunakan sebagai file utama yang memuat *endpoint REST API* dan proses prediksi. File *requirements.txt* berisi daftar *library Python* yang dibutuhkan oleh *backend*, sedangkan *Procfile* digunakan untuk menentukan perintah saat *backend* dijalankan pada platform *deployment*. Selain itu, folder models/ digunakan untuk menyimpan file model dan komponen pendukung, seperti model *SVM*, *vocabulary BoF*, scaler, *PCA*, *metadata*, serta data hasil evaluasi metode.

Deployment backend dilakukan agar *REST API* dapat diakses oleh aplikasi *Android* melalui jaringan internet. Proses *deployment* dimulai dari project *backend Flask* yang diunggah ke *repository GitHub*. *Repository* tersebut kemudian dihubungkan dengan *Railway* sebagai platform *deployment*. *Railway* akan melakukan proses *build*, menginstal *dependencies* yang dibutuhkan, dan menjalankan aplikasi *Flask* menggunakan *Gunicorn*. Setelah proses *deployment* berhasil, *backend* akan memiliki URL aktif yang dapat digunakan oleh aplikasi *Android* untuk mengakses *endpoint REST API*.

3.3.5 Perancangan Desain Aplikasi *Android*

Perancangan desain aplikasi *Android* dilakukan untuk memudahkan pengguna dalam menggunakan sistem deteksi kesiapan panen tanaman cabai rawit. Aplikasi ini dirancang sebagai antarmuka utama antara pengguna dengan sistem deteksi, sehingga pengguna dapat mengambil atau memilih citra tanaman cabai rawit, mengirimkan citra untuk diproses, melihat hasil prediksi, serta mengakses riwayat hasil prediksi. Selain itu, aplikasi juga menyediakan menu khusus peneliti untuk menampilkan hasil evaluasi metode yang digunakan dalam penelitian.

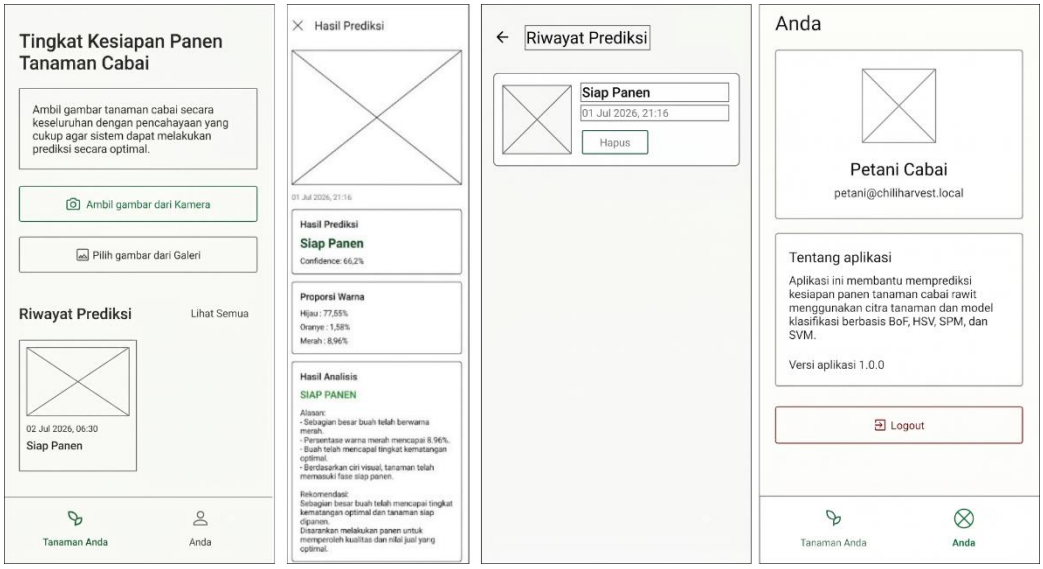
Konsep desain aplikasi *Chilivest* dibuat sederhana, rapi, dan mudah dipahami oleh pengguna. Tampilan aplikasi menggunakan elemen visual berupa tombol, *card*, ikon, dan navigasi bawah agar pengguna dapat berpindah menu dengan lebih mudah. Warna hijau digunakan sebagai warna utama karena menyesuaikan dengan tema pertanian dan tanaman cabai. Perancangan ini bertujuan agar aplikasi dapat digunakan oleh petani sebagai pengguna utama, serta tetap menyediakan akses informasi evaluasi metode bagi peneliti.



Gambar 3. 12 Rancangan Tampilan Awal dan Pemilihan Jenis Pengguna

Gambar 3.12 menunjukkan rancangan tampilan awal aplikasi *Chilivest*. Tampilan awal terdiri dari halaman *loading* atau *splash screen* yang menampilkan logo aplikasi, nama aplikasi, serta keterangan singkat mengenai fungsi aplikasi sebagai

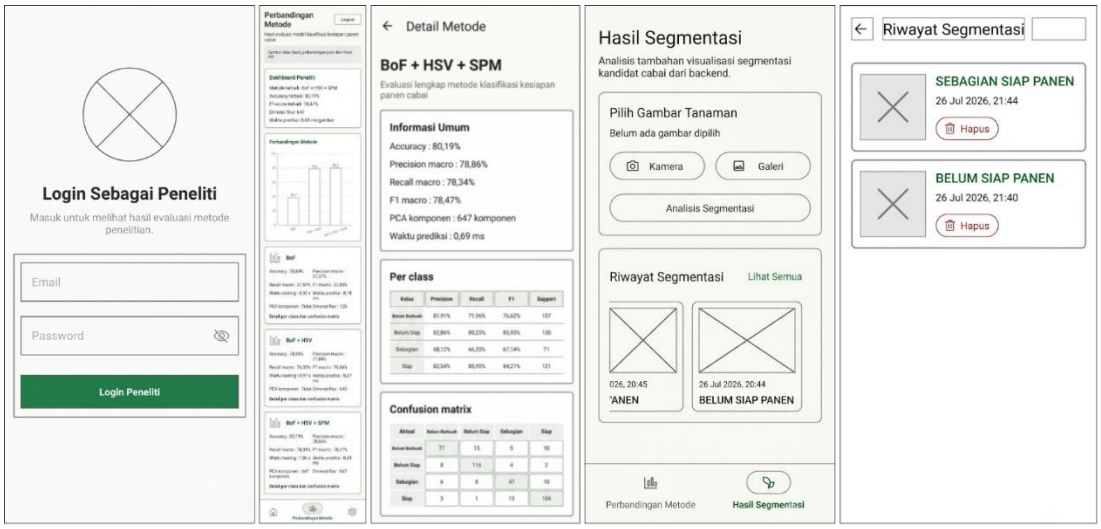
sistem prediksi kesiapan panen cabai rawit. Setelah proses *loading* selesai, pengguna diarahkan ke halaman pemilihan jenis pengguna. Pada halaman ini tersedia dua pilihan akses, yaitu *Login* Sebagai Petani dan *Login* Sebagai Peneliti. Pilihan petani digunakan untuk melakukan prediksi dan menyimpan riwayat tanaman cabai, sedangkan pilihan peneliti digunakan untuk melihat hasil evaluasi dan perbandingan metode penelitian.



Gambar 3. 13 Rancangan Tampilan Menu Petani

Gambar 3.13 menunjukkan rancangan tampilan menu untuk pengguna petani. Pada halaman utama petani, aplikasi menampilkan judul Tingkat Kesiapan Panen Tanaman Cabai sebagai identitas fitur utama. Pengguna dapat mengambil citra tanaman cabai melalui tombol Ambil Gambar dari Kamera atau memilih citra dari perangkat melalui tombol Pilih Gambar dari Galeri. Pada halaman ini juga ditampilkan bagian riwayat prediksi yang memuat hasil prediksi sebelumnya.

Setelah citra diproses, aplikasi menampilkan halaman hasil prediksi. Halaman ini berisi citra tanaman yang dianalisis, waktu prediksi, hasil klasifikasi, nilai *confidence*, proporsi warna, hasil analisis, alasan prediksi, dan rekomendasi tindakan. Selain itu, aplikasi juga menyediakan halaman Riwayat Prediksi untuk menampilkan kembali hasil deteksi yang pernah dilakukan. Pada menu Anda, pengguna dapat melihat informasi akun, deskripsi singkat aplikasi, versi aplikasi, serta tombol *logout*.



Gambar 3. 14 Rancangan Tampilan *Login* Peneliti dan Menu Peneliti

Gambar 3.14 menunjukkan rancangan tampilan untuk pengguna peneliti pada aplikasi Chilivest. Pada halaman *login* peneliti, pengguna diminta memasukkan *email* dan *password* untuk masuk ke menu peneliti. Setelah berhasil masuk, aplikasi menampilkan halaman peneliti yang terdiri dari menu Perbandingan Metode, Hasil Segmentasi, dan Riwayat Segmentasi.

Menu Perbandingan Metode digunakan untuk menampilkan hasil evaluasi metode BoF, BoF + HSV, dan BoF + HSV + SPM. Pada halaman detail metode, aplikasi menampilkan informasi evaluasi seperti nilai *accuracy*, *precision*, *recall*, *F1-score*, jumlah komponen PCA, waktu prediksi, evaluasi *per class*, dan *confusion matrix*.

Menu Hasil Segmentasi digunakan untuk menampilkan analisis tambahan terhadap citra tanaman cabai rawit. Informasi yang ditampilkan meliputi citra *input resize*, *mask* kandidat cabai, *overlay* segmentasi, hasil prediksi, proporsi warna HSV, alasan, dan rekomendasi. Hasil segmentasi hanya digunakan sebagai informasi pendukung dan tidak menjadi penentu kelas prediksi SVM.

Menu Riwayat Segmentasi digunakan untuk menampilkan kembali data hasil segmentasi yang telah tersimpan. Dengan rancangan ini, peneliti dapat melihat hasil evaluasi metode serta analisis segmentasi citra tanaman cabai rawit secara langsung melalui aplikasi Chilivest.

3.4 Penyiapan Alat dan Bahan

Alat dan bahan yang diperlukan untuk membuat sistem aplikasi adalah sebagai berikut :

a. Perangkat Keras

Tabel 3. 3 Tabel Perangkat Keras yang Digunakan

No	Nama Bahan	Jumlah	Keterangan
1	Laptop	1	ASUS X415 Processor i3 Gen 10 RAM 8 Windows 10
2	*Smartphone Android*	1	Digunakan untuk pengujian aplikasi *mobile*
3	HP Kamera	1	Digunakan jika *dataset* dikumpulkan secara mandiri
4	Koneksi Internet	-	Digunakan untuk mengakses *backend*, Google Colab, dan proses pengujian aplikasi.

b. Perangkat Lunak

Tabel 3. 4 Tabel Perangkat Lunak yang Digunakan

No	Nama Bahan	Keterangan
1	Google Colab	Digunakan untuk pemrosesan *dataset*, ekstraksi fitur, pelatihan model, dan evaluasi metode
2	*Python*	Digunakan sebagai bahasa pemrograman utama dalam proses pengolahan citra dan pelatihan model
3	*Flask*	Digunakan untuk membangun *backend REST API*
4	*Railway*	Digunakan sebagai platform *deployment backend Flask*
5.	*GitHub*	Digunakan sebagai *repository* penyimpanan kode *backend*

6.	*Android Studio*	Digunakan untuk mengembangkan aplikasi *Android Chilivest*
7.	Postman	Digunakan untuk menguji *endpoint REST API backend*

c. *Dataset*

- Citra tanaman cabai rawit
- Label klasifikasi: belum berbuah, belum siap, sebagian, siap.

3.5 Parameter Pengujian

a. Pengujian Perbandingan Metode

Pengujian ini dilakukan untuk membandingkan performa metode *BoF*, *BoF + HSV*, dan *BoF + HSV + SPM*. Parameter yang diuji meliputi *accuracy*, *precision*, *recall*, *F1-score*, *confusion matrix*, waktu *training*, dan waktu prediksi.

b. Pengujian Fungsionalitas dan Respons Aplikasi *Android*

Pengujian ini dilakukan untuk memastikan aplikasi *Android Chilivest* dapat berjalan sesuai rancangan. Fitur yang diuji meliputi tampilan awal aplikasi, pemilihan jenis pengguna, ambil Gambar dari kamera, pilih Gambar dari galeri, proses *loading* analisis, tampilan hasil prediksi, riwayat prediksi, menu akun, menu peneliti, detail metode, dan *logout*.

c. Pengujian *REST API*

Pengujian ini dilakukan untuk memastikan *backend* Flask dapat menerima *request* dan mengirimkan *response* sesuai rancangan. *Endpoint* yang diuji meliputi /health, /predict, /method-comparison, dan /predict-segmentation.

d. Pengujian Komunikasi *Client-Server*

Pengujian ini dilakukan untuk memastikan aplikasi *Android* dapat mengirim citra ke *backend Flask* melalui *REST API* dan menerima hasil prediksi dalam bentuk *JSON*. Hasil *response* dari *backend* kemudian ditampilkan pada aplikasi *Android*.

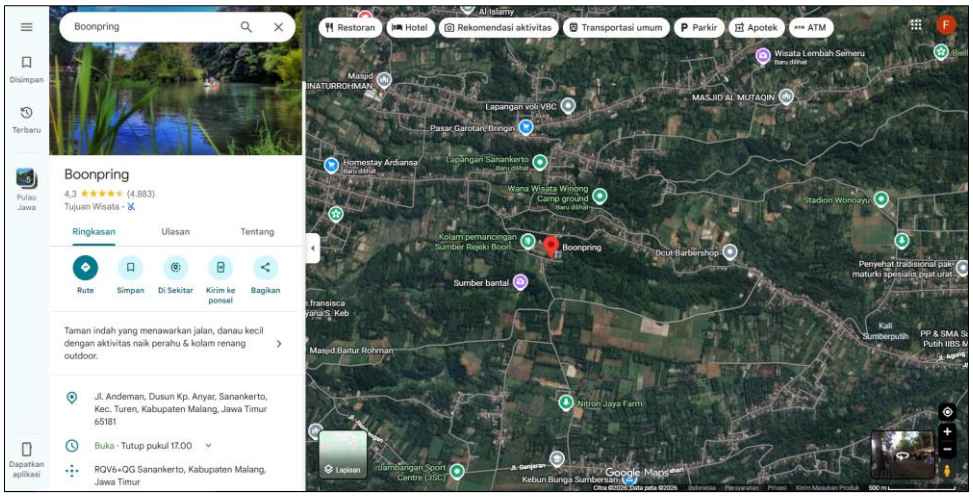
e. Pengujian *Deployment Backend*

Pengujian ini dilakukan untuk memastikan *backend Flask* dapat berjalan secara *online* melalui *Railway*. Parameter yang diuji meliputi keberhasilan *build*, status *deployment*, URL *backend* aktif, model berhasil dimuat, dan *endpoint* dapat diakses oleh aplikasi *Android*.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Tempat dan Waktu Penelitian

Penelitian ini dilaksanakan di daerah sekitar kawasan wisata alam Boonpring, yang berlokasi di Jl. Andeman, Dusun Kp. Anyar, Sanankerto, Kecamatan Turen, Kabupaten Malang, Jawa Timur. Lokasi ini dipilih karena memiliki lingkungan terbuka dan terdapat area tanaman yang sesuai untuk kebutuhan pengambilan data penelitian. Dokumentasi lokasi penelitian ditunjukkan pada Gambar 4.1.



Gambar 4. 1 Lokasi *google maps* Penelitian

Waktu penelitian dilaksanakan selama empat bulan dengan jumlah kunjungan langsung ke lokasi penelitian sebanyak dua kali. Kunjungan tersebut dilakukan untuk observasi lokasi, pengambilan data, dan memastikan kesesuaian data dengan kondisi di lapangan.

4.2 Hasil Penelitian

Hasil penelitian menjelaskan implementasi dan pengujian sistem deteksi kesiapan panen tanaman cabai rawit menggunakan aplikasi *Android Chilivest*. Pada bagian ini dijelaskan hasil tampilan aplikasi *Android*, hasil pengolahan citra dan ekstraksi fitur, pengujian perbandingan metode *BoF*, *BoF + HSV*, dan *BoF + HSV + SPM*, serta pengujian fungsionalitas aplikasi. Selain itu, bagian ini juga memuat hasil pengujian *REST API*, komunikasi *client-server* antara aplikasi *Android* dan *backend*

Flask, serta pengujian *deployment backend* pada *Railway* untuk memastikan sistem dapat berjalan secara *online* dan sesuai dengan rancangan yang telah dibuat.

4.2.1 Tampilan Aplikasi *Android*

1. Tampilan Logo Aplikasi



Gambar 4. 2 Tampilan Logo Aplikasi

Gambar 4.2 menunjukkan tampilan logo aplikasi *Chilivest* yang digunakan sebagai identitas visual aplikasi. Logo tersebut menampilkan ilustrasi sederhana berbentuk tanaman dan cabai dengan dominasi warna hijau serta merah. Warna hijau merepresentasikan tema pertanian dan tanaman, sedangkan warna merah mengGambarkan objek utama penelitian, yaitu cabai rawit. Logo ini digunakan sebagai ikon aplikasi pada perangkat *Android* dan menjadi tampilan identitas awal sebelum pengguna masuk ke halaman aplikasi.

Berdasarkan potongan program pada Lampiran 1 bagian Logo Aplikasi, file AndroidManifest.xml digunakan untuk mendefinisikan ikon aplikasi *Chilivest*. Atribut *android*:icon dan *android*:roundIcon mengarah ke file drawable ic_app_icon, sehingga logo aplikasi dapat ditampilkan sebagai ikon utama pada perangkat *Android*. Pada dokumentasi program, file manifest juga menunjukkan bahwa ikon aplikasi direferensikan menggunakan *@drawable/ic_app_icon*.

2. Tampilan *Splash Screen*



Gambar 4. 3 Tampilan *Splash Screen*

Gambar 4.3 menunjukkan tampilan *splash screen* aplikasi *Chilivest* ketika aplikasi pertama kali dijalankan. Halaman ini menampilkan logo aplikasi, nama aplikasi *Chilivest*, keterangan singkat "Prediksi kesiapan panen cabai rawit", serta indikator *loading*. *Splash screen* berfungsi sebagai halaman pembuka sebelum sistem mengarahkan pengguna ke halaman berikutnya, yaitu halaman pemilihan jenis pengguna atau halaman utama apabila pengguna sebelumnya sudah memiliki sesi *login*.

Berdasarkan potongan program pada Lampiran 1 bagian *Splash Screen*, potongan program tersebut terdapat pada *SplashActivity*. Program digunakan untuk menampilkan *layout activity_splash* sebagai tampilan awal aplikasi. Setelah *splash screen* tampil selama 900 ms, sistem menjalankan fungsi *navigateNext*() untuk menentukan halaman tujuan berikutnya. Pada dokumentasi program *Chilivest*, alur aplikasi dimulai dari *Splash Screen* → Pilih Jenis Pengguna → Petani/Peneliti, sehingga *splash screen* menjadi halaman pembuka sebelum pengguna masuk ke alur aplikasi utama.

3. Tampilan Pemilihan Jenis Pengguna

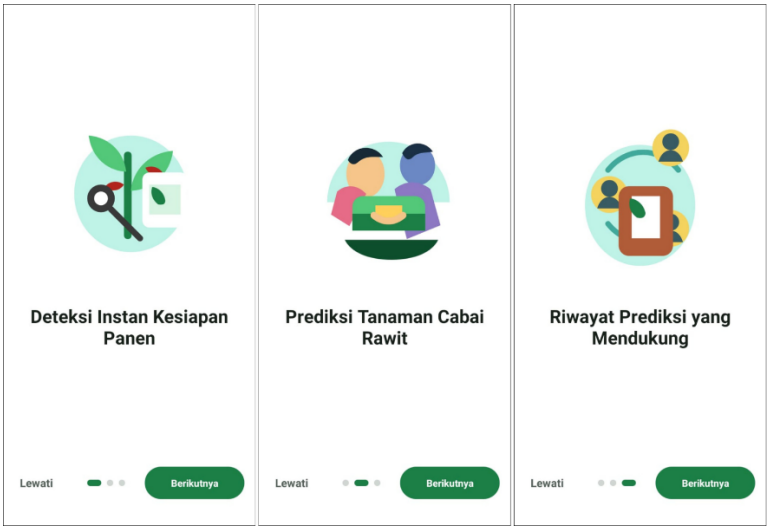


Gambar 4. 4 Tampilan Pemilihan Jenis Pengguna

Gambar 4.4 menunjukkan tampilan halaman pemilihan jenis pengguna pada aplikasi *Chilivest*. Pada halaman ini pengguna dapat memilih dua jenis akses, yaitu *Login* Sebagai Petani dan *Login* Sebagai Peneliti. Menu petani digunakan untuk melakukan prediksi kesiapan panen dan menyimpan riwayat tanaman cabai, sedangkan menu peneliti digunakan untuk melihat evaluasi serta perbandingan metode penelitian. Tampilan ini menjadi penghubung setelah *splash screen* sebelum pengguna masuk ke fitur utama aplikasi.

Berdasarkan potongan program pada Lampiran 1 bagian Jenis Pengguna, potongan program tersebut terdapat pada *RoleSelectionActivity*. Program ini digunakan untuk menampilkan *layout activity_role_selection* dan memberikan aksi pada dua pilihan kartu pengguna. Ketika pengguna memilih kartu petani, aplikasi akan berpindah ke *FarmerOnboardingActivity*. Sedangkan ketika pengguna memilih kartu peneliti, aplikasi akan berpindah ke *ResearcherLoginActivity*. Pada dokumentasi program *Chilivest*, alur aplikasi setelah *splash screen* memang diarahkan ke halaman pemilihan peran pengguna, yaitu petani atau peneliti.

4. Tampilan Onboarding Login Petani



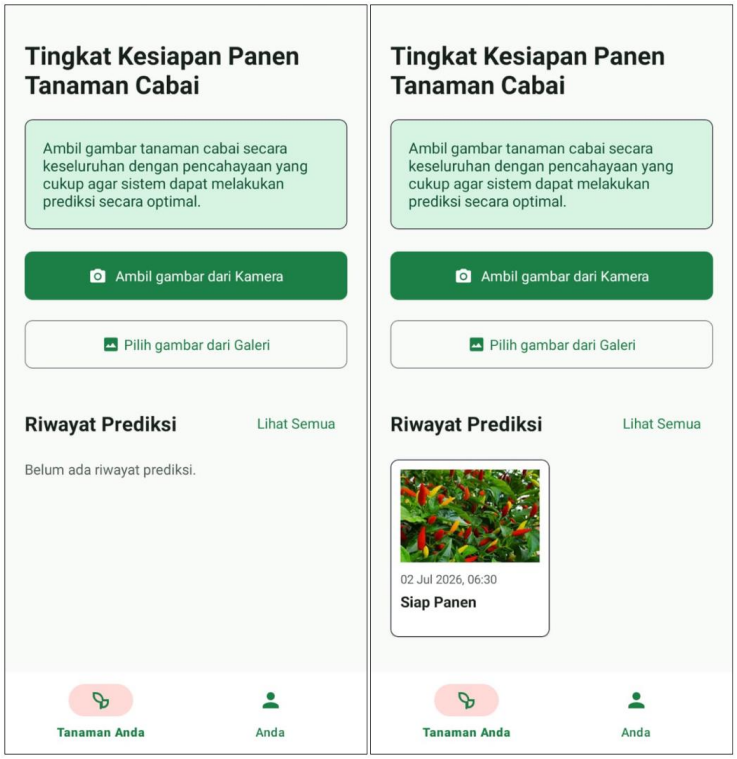
Gambar 4. 5 Tampilan *Onboarding Login* Petani

Gambar 4.5 menunjukkan tampilan *onboarding* untuk pengguna petani pada aplikasi *Chilivest*. Tampilan ini terdiri dari tiga halaman pengenalan fitur, yaitu Deteksi Instan Kesiapan Panen, Prediksi Tanaman Cabai Rawit, dan Riwayat Prediksi yang

Mendukung. Halaman *onboarding* digunakan untuk memberikan Gambaran awal kepada petani mengenai fungsi utama aplikasi sebelum masuk ke menu utama. Pengguna dapat menekan tombol Berikutnya untuk berpindah halaman atau menekan Lewati untuk langsung masuk ke menu petani.

Berdasarkan potongan program pada Lampiran 1 bagian *Onboarding Login* Petani, potongan program tersebut terdapat pada *FarmerOnboardingActivity*. Variabel pages digunakan untuk menyimpan daftar halaman *onboarding* yang ditampilkan melalui *ViewPager2*. Tombol Berikutnya digunakan untuk menggeser halaman *onboarding* ke halaman selanjutnya, sedangkan tombol Lewati digunakan untuk langsung menjalankan fungsi *enterFarmerArea*() agar pengguna masuk ke menu utama petani. Pada dokumentasi program *Chilivest*, alur petani setelah memilih peran memang diarahkan ke halaman *onboarding* sebelum masuk ke menu petani.

5. Tampilan Menu Tanaman Anda

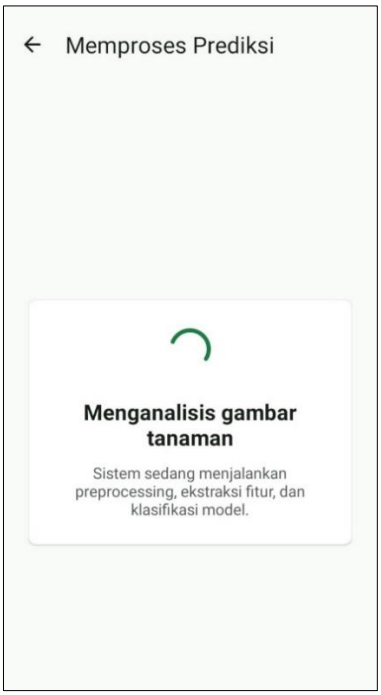


Gambar 4. 6 Tampilan Menu Tanaman Anda

Gambar 4.6 menunjukkan tampilan menu Tanaman Anda pada aplikasi *Chilivest*. Menu ini merupakan halaman utama untuk pengguna petani. Pada halaman ini terdapat informasi singkat mengenai cara pengambilan Gambar tanaman cabai, tombol Ambil Gambar dari Kamera, tombol Pilih Gambar dari Galeri, serta bagian Riwayat Prediksi. Tampilan sebelah kiri menunjukkan kondisi ketika belum terdapat riwayat prediksi, sedangkan tampilan sebelah kanan menunjukkan kondisi ketika hasil prediksi sebelumnya sudah tersimpan dan ditampilkan pada bagian riwayat.

Berdasarkan potongan program pada Lampiran 1 bagian Menu Tanaman Anda, potongan program tersebut terdapat pada *PlantFragment*. Program ini digunakan untuk menjalankan fungsi utama pada menu Tanaman Anda. Tombol kamera akan memanggil fungsi *requestCameraOrOpen*() untuk membuka kamera, tombol galeri akan membuka pilihan Gambar dari perangkat, sedangkan tombol Lihat Semua akan mengarahkan pengguna ke halaman daftar riwayat prediksi. Pada dokumentasi program *Chilivest*, *PlantFragment* memang berfungsi sebagai komponen UI menu Tanaman Anda untuk memilih Gambar dan memulai proses prediksi.

6. Tampilan Loading Proses Prediksi

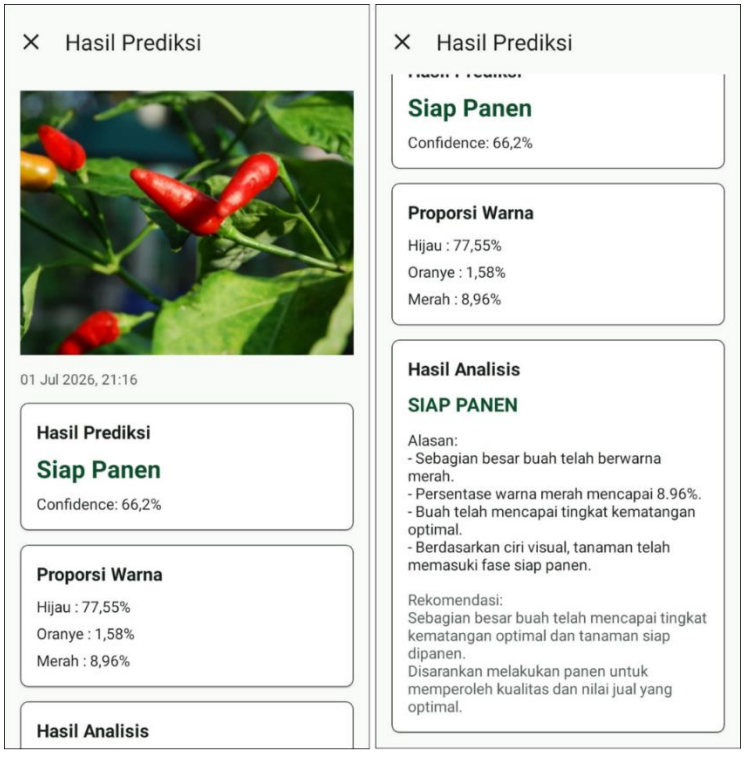


Gambar 4. 7 Tampilan Loading Proses Prediksi

Gambar 4.7 menunjukkan tampilan *loading* ketika aplikasi sedang memproses citra tanaman cabai rawit. Tampilan ini muncul setelah pengguna mengambil Gambar dari kamera atau memilih Gambar dari galeri. Pada halaman ini terdapat keterangan bahwa sistem sedang menjalankan tahapan *preprocessing*, ekstraksi fitur, dan klasifikasi model. Halaman *loading* digunakan untuk memberikan informasi kepada pengguna bahwa proses prediksi sedang berlangsung sebelum hasil prediksi ditampilkan.

Berdasarkan potongan program pada Lampiran 1 bagian *Loading* Proses Prediksi, potongan program tersebut terdapat pada *PredictionLoadingActivity*. Fungsi *showLoading*() digunakan untuk menampilkan indikator proses saat prediksi sedang berjalan. Pada kondisi ini, tombol ulangi dan tombol tutup disembunyikan, sedangkan teks informasi proses ditampilkan kepada pengguna. Berdasarkan dokumentasi program *Chilivest*, halaman *PredictionLoadingActivity* berfungsi sebagai komponen UI untuk proses *loading* prediksi sebelum aplikasi berpindah ke halaman hasil prediksi.

7. Tampilan Hasil Prediksi

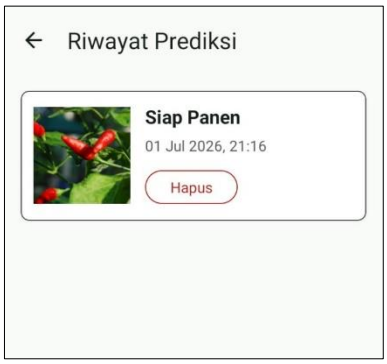


Gambar 4. 8 Tampilan Hasil Prediksi

Gambar 4.8 menunjukkan tampilan halaman hasil prediksi pada aplikasi *Chilivest* setelah citra tanaman cabai rawit selesai diproses oleh sistem. Pada halaman ini ditampilkan citra yang dianalisis, waktu prediksi, hasil klasifikasi, nilai *confidence*, proporsi warna hijau, oranye, dan merah, serta hasil analisis berupa alasan dan rekomendasi. Pada contoh tampilan tersebut, sistem menghasilkan prediksi Siap Panen dengan nilai *confidence* sebesar 66,2%. Tampilan ini bertujuan agar pengguna dapat memahami hasil deteksi secara lebih jelas, tidak hanya dari kelas prediksi, tetapi juga dari informasi warna dan rekomendasi yang diberikan.

Berdasarkan potongan program pada Lampiran 1 bagian Hasil Prediksi, potongan program tersebut terdapat pada *PredictionResultActivity*. Fungsi *bindResult*() digunakan untuk menampilkan data hasil prediksi yang tersimpan pada objek *PredictionHistoryEntity* ke dalam komponen tampilan aplikasi. Data yang ditampilkan meliputi tanggal prediksi, hasil klasifikasi, *confidence*, persentase warna hijau, oranye, merah, status tanaman, alasan prediksi, dan rekomendasi. Pada dokumentasi program *Chilivest*, *PredictionResultActivity* memang berfungsi sebagai komponen UI untuk menampilkan hasil prediksi setelah proses analisis selesai.

8. Tampilan Riwayat Prediksi



Gambar 4. 9 Tampilan Riwayat Prediksi

Gambar 4.9 menunjukkan tampilan halaman Riwayat Prediksi pada aplikasi *Chilivest*. Halaman ini digunakan untuk menampilkan hasil prediksi yang telah dilakukan sebelumnya oleh pengguna. Informasi yang ditampilkan pada setiap item riwayat meliputi Gambar tanaman cabai, hasil prediksi, waktu prediksi, serta tombol

Hapus untuk menghapus data riwayat tertentu. Fitur ini memudahkan pengguna untuk melihat kembali hasil analisis sebelumnya tanpa harus melakukan prediksi ulang.

Berdasarkan potongan program pada Lampiran 1 bagian Riwayat Prediksi dan Hapus Riwayat, potongan program tersebut terdapat pada *HistoryListActivity*. Program pertama digunakan untuk menampilkan daftar riwayat prediksi ke dalam *RecyclerView* melalui *HistoryAdapter*. Ketika salah satu item riwayat dipilih, aplikasi akan membuka kembali halaman hasil prediksi berdasarkan history.id. Selain itu, data riwayat diamati melalui *viewModel*.allHistory, sehingga daftar riwayat pada tampilan akan diperbarui sesuai data yang tersimpan.

Program hapus Riwayat digunakan untuk menampilkan dialog konfirmasi ketika pengguna menekan tombol Hapus. Jika pengguna memilih tombol Hapus, maka aplikasi akan memanggil *viewModel.deleteHistory*(history) untuk menghapus data riwayat yang dipilih. Dengan mekanisme ini, halaman riwayat prediksi dapat menampilkan, membuka kembali, dan menghapus hasil prediksi yang tersimpan pada aplikasi.

9. Tampilan Menu Anda

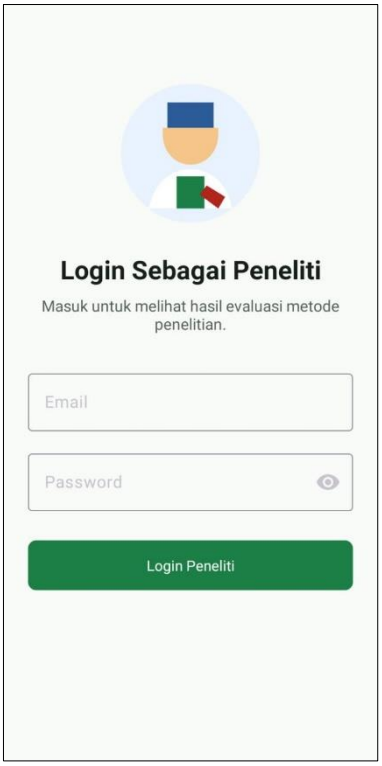


Gambar 4. 10 Tampilan Menu Anda

Gambar 4.10 menunjukkan tampilan menu Anda pada aplikasi *Chilivest*. Menu ini berisi informasi akun pengguna, seperti nama pengguna Petani Cabai dan email lokal pengguna. Selain itu, halaman ini juga menampilkan informasi singkat mengenai aplikasi *Chilivest*, versi aplikasi, serta tombol *Logout*. Menu ini digunakan agar pengguna dapat melihat identitas akun, mengetahui fungsi aplikasi secara umum, dan keluar dari aplikasi ketika sudah selesai digunakan.

Berdasarkan potongan program pada Lampiran 1 bagian Menu Anda, potongan program tersebut terdapat pada *ProfileFragment*. Program ini digunakan untuk menampilkan data sesi pengguna pada menu Anda, seperti nama, email, dan versi aplikasi. Selain itu, tombol *Logout* digunakan untuk menghapus sesi pengguna melalui *viewModel.logout*(), kemudian aplikasi diarahkan kembali ke halaman pemilihan jenis pengguna. Pada dokumentasi program *Chilivest*, *ProfileFragment* berfungsi sebagai komponen UI profil pengguna dan *logout*.

10. Tampilan Login Peneliti

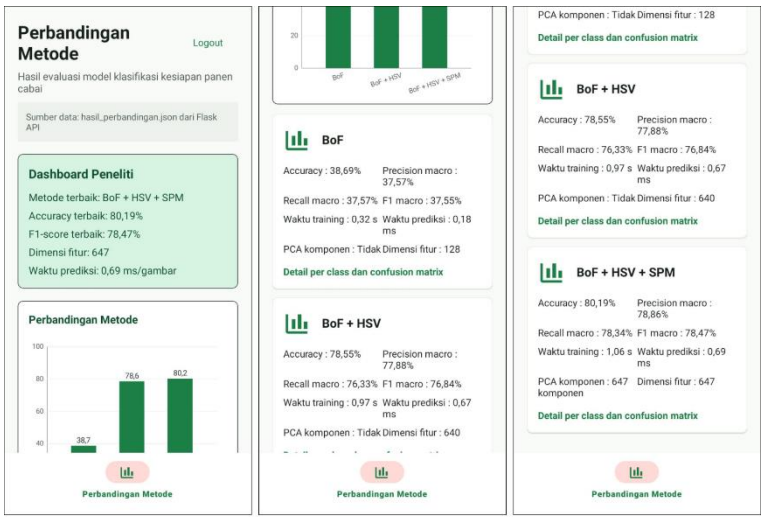


Gambar 4. 11 Tampilan Login Peneliti

Gambar 4.11 menunjukkan tampilan halaman *Login* Sebagai Peneliti pada aplikasi *Chilivest*. Halaman ini digunakan sebagai akses khusus bagi peneliti untuk masuk ke menu evaluasi metode penelitian. Pada tampilan ini terdapat kolom *input* email, kolom *input* password, dan tombol *Login* Peneliti. Setelah *login* berhasil, pengguna akan diarahkan ke halaman peneliti yang berisi hasil evaluasi dan perbandingan metode klasifikasi.

Berdasarkan potongan program pada Lampiran 1 bagian *Login* Peneliti, potongan program tersebut terdapat pada *ResearcherLoginActivity*. Program ini digunakan untuk mengambil *input* email dan password dari pengguna, kemudian mengirimkannya ke ResearcherLoginViewModel melalui fungsi *login*(). Jika *login* berhasil, aplikasi akan menjalankan fungsi navigateToResearcherArea() untuk berpindah ke menu utama peneliti. Jika *login* gagal, aplikasi akan menampilkan pesan kesalahan pada kolom password dan melalui *Snackbar*. Pada dokumentasi program *Chilivest*, *ResearcherLoginActivity* berfungsi sebagai komponen UI untuk *login* peneliti sebelum masuk ke *dashboard* evaluasi metode.

11. Tampilan Menu Perbandingan Metode



Gambar 4. 12 Tampilan Menu Perbandingan Metode

Gambar 4.12 menunjukkan tampilan menu Perbandingan Metode pada aplikasi *Chilivest* yang digunakan oleh pengguna peneliti. Halaman ini menampilkan *dashboard* peneliti berisi metode terbaik, nilai *accuracy* terbaik, *F1-score* terbaik,

dimensi fitur, dan waktu prediksi. Selain itu, halaman ini juga menampilkan grafik perbandingan *accuracy* serta kartu hasil evaluasi untuk tiga metode, yaitu *BoF*, *BoF + HSV*, dan *BoF + HSV + SPM*. Informasi yang ditampilkan meliputi *accuracy*, *precision macro*, *recall macro*, *F1 macro*, waktu *training*, waktu prediksi, komponen *PCA*, dan dimensi fitur.

Berdasarkan potongan program pada Lampiran 1 bagian Menu Perbandingan Metode, potongan program tersebut terdapat pada *MethodComparisonFragment*. Program pertama digunakan untuk mengamati data metode dari *MethodComparisonViewModel*, kemudian menampilkan metode terbaik pada *dashboard*, membuat grafik perbandingan melalui *setupChart*(), dan menampilkan kartu hasil evaluasi melalui *setupCards*(). Program kedua digunakan untuk mengisi ringkasan *dashboard* peneliti, seperti metode terbaik, *accuracy* terbaik, *F1-score* terbaik, dimensi fitur, dan waktu prediksi. Pada dokumentasi program *Chilivest*, *MethodComparisonFragment* memang berfungsi sebagai komponen UI area peneliti untuk *dashboard* dan perbandingan metode.

12. Tampilan Detail Metode

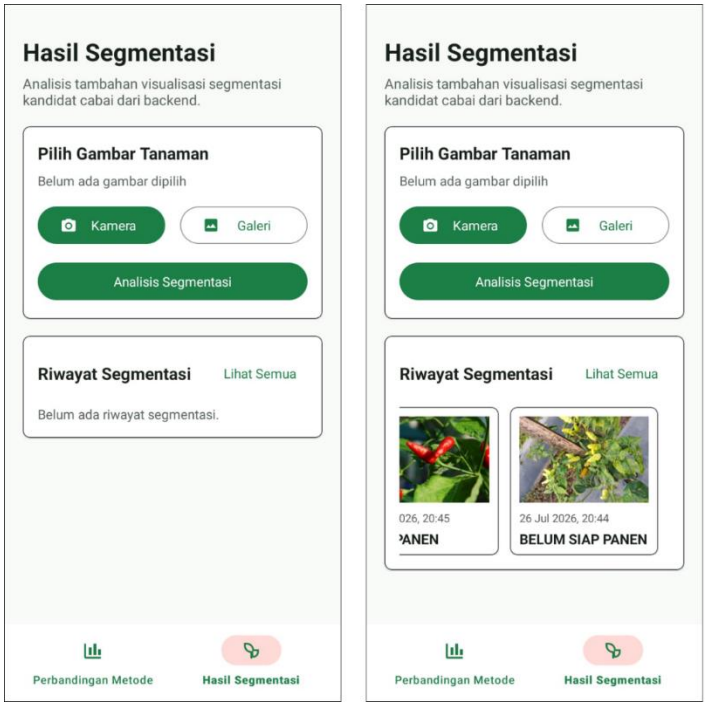


Gambar 4. 13 Tampilan Detail Metode

Gambar 4.13 menunjukkan tampilan Detail Metode pada aplikasi *Chilivest*. Halaman ini digunakan untuk menampilkan hasil evaluasi lengkap dari setiap metode, yaitu *BoF*, *BoF + HSV*, dan *BoF + HSV + SPM*. Informasi yang ditampilkan meliputi nilai *accuracy*, *precision macro*, *recall macro*, *F1 macro*, jumlah komponen *PCA*, waktu prediksi, tabel evaluasi per kelas, serta *confusion matrix*. Tampilan ini membantu peneliti melihat performa setiap metode secara lebih rinci tanpa harus membuka hasil evaluasi di luar aplikasi.

Berdasarkan potongan program pada Lampiran 1 bagian Detail Metode, potongan program tersebut terdapat pada *MethodDetailActivity*. Fungsi *bindMethod*() digunakan untuk menampilkan data detail metode ke dalam halaman aplikasi. Data yang ditampilkan meliputi informasi umum metode, tabel evaluasi per kelas, dan *confusion matrix*. Pada dokumentasi program *Chilivest*, *MethodDetailActivity* berfungsi sebagai halaman detail evaluasi metode yang memuat *accuracy*, *precision macro*, *recall macro*, *F1 macro*, *PCA* komponen, waktu prediksi, per class, dan *confusion matrix*.

13. Tampilan Menu Hasil Segmentasi

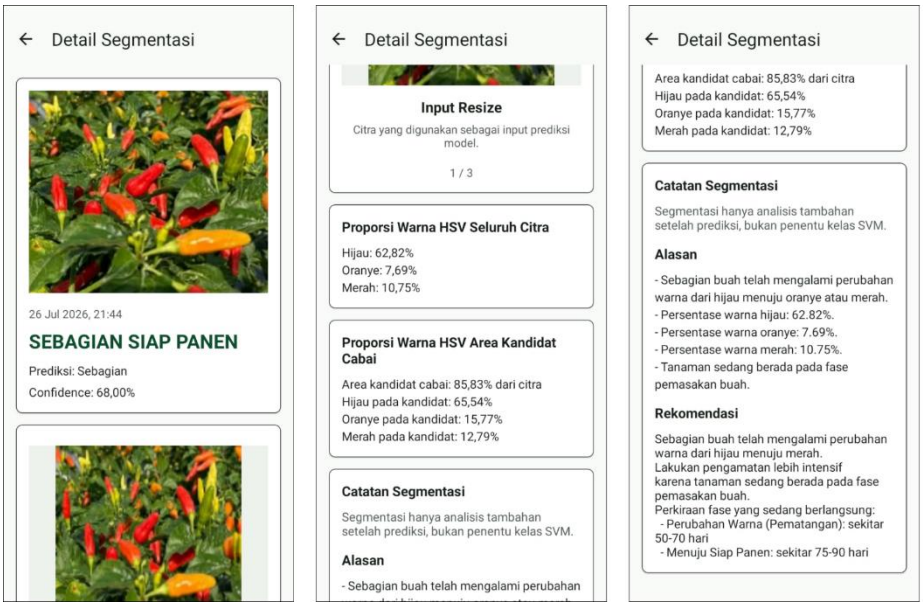


Gambar 4. 14 Tampilan Menu Hasil Segmentasi

Gambar 4.14 menunjukkan tampilan Menu Hasil Segmentasi pada akun Peneliti. Menu ini digunakan untuk melakukan analisis tambahan terhadap citra tanaman cabai rawit melalui proses pengambilan Gambar menggunakan kamera atau pemilihan Gambar dari galeri. Setelah Gambar dipilih, pengguna dapat menekan tombol Analisis Segmentasi untuk mengirim citra ke *backend* dan memperoleh hasil prediksi serta informasi segmentasi.

Pada halaman ini juga terdapat bagian Riwayat Segmentasi yang digunakan untuk menampilkan data hasil segmentasi yang telah tersimpan. Jika belum terdapat data, sistem menampilkan keterangan bahwa riwayat segmentasi masih kosong. Apabila sudah terdapat data, sistem akan menampilkan daftar riwayat berupa Gambar, waktu analisis, dan status hasil prediksi. Menu Hasil Segmentasi dirancang untuk membantu Peneliti melihat analisis visual berupa citra *input resize*, *mask* kandidat cabai, dan *overlay* segmentasi pada proses pengujian citra tanaman cabai rawit.

14. Tampilan Hasil Segmentasi



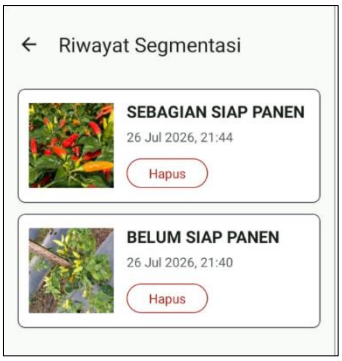
Gambar 4. 15 Tampilan Hasil Segmentasi

Gambar 4.15 menunjukkan tampilan hasil segmentasi pada akun Peneliti setelah pengguna melakukan analisis citra tanaman cabai rawit. Pada halaman ini, aplikasi menampilkan hasil prediksi berupa status tanaman, kelas prediksi, dan nilai

confidence. Selain itu, sistem juga menampilkan citra hasil analisis dalam bentuk *slider* yang terdiri dari citra *input resize*, *mask* kandidat cabai, dan *overlay* segmentasi.

Pada bagian detail hasil, aplikasi menampilkan proporsi warna HSV seluruh citra, proporsi warna HSV area kandidat cabai, catatan segmentasi, alasan, dan rekomendasi. Informasi segmentasi ini digunakan sebagai analisis tambahan untuk membantu Peneliti memahami sebaran warna dan area kandidat cabai, sedangkan keputusan prediksi tetap diperoleh dari model utama dan tidak ditentukan oleh hasil segmentasi.

15. Tampilan Riwayat Segmentasi



Gambar 4. 16 Tampilan Riwayat Segmentasi

Gambar 4.16 menunjukkan tampilan Riwayat Segmentasi pada akun Peneliti. Halaman ini digunakan untuk menampilkan daftar hasil segmentasi yang telah dilakukan sebelumnya. Setiap data riwayat menampilkan citra tanaman cabai rawit, status hasil prediksi, serta waktu analisis segmentasi.

Pada halaman ini, Peneliti dapat melihat kembali hasil segmentasi yang tersimpan atau menghapus data riwayat tertentu melalui tombol Hapus. Fitur Riwayat Segmentasi memudahkan Peneliti untuk meninjau ulang hasil analisis tanpa perlu melakukan proses unggah citra dan segmentasi ulang.

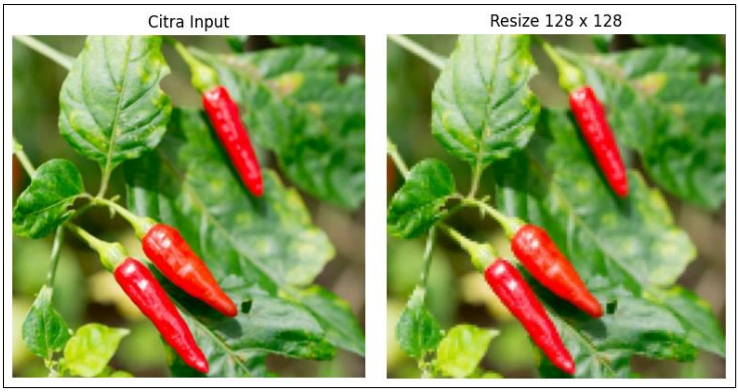
4.2.2 Hasil Pengolahan Citra dan Ekstraksi Fitur

Pada tahap ini dilakukan proses pengolahan citra dan ekstraksi fitur sebelum data citra diklasifikasikan menggunakan model SVM. Proses pengolahan diawali dengan memasukkan citra tanaman cabai rawit sebagai data masukan sistem. Citra

tersebut kemudian melalui tahap pra-pemrosesan berupa penyesuaian ukuran menjadi 128 × 128 piksel agar seluruh citra memiliki ukuran yang seragam. Penyeragaman ukuran citra diperlukan supaya proses ekstraksi fitur dapat dilakukan secara konsisten pada seluruh data.

Setelah citra melalui tahap pra-pemrosesan, citra kemudian diproses untuk memperoleh fitur visual menggunakan tiga metode utama, yaitu Bag-of-Features (BoF), fitur warna HSV, dan Spatial Pyramid Matching (SPM). BoF digunakan untuk memperoleh pola lokal, bentuk, dan tekstur dari citra. HSV digunakan untuk merepresentasikan informasi warna, sedangkan SPM digunakan untuk mempertahankan informasi spasial atau sebaran fitur pada citra. Fitur hasil ekstraksi tersebut kemudian digabungkan, dinormalisasi, direduksi menggunakan Principal Component Analysis (PCA), dan digunakan sebagai masukan pada proses klasifikasi menggunakan SVM.

1. Pra-pemrosesan Citra



Gambar 4. 17 Citra *Input* dan Hasil *Resize* 128 × 128

Gambar 4.17 menunjukkan contoh tahapan awal pengolahan citra tanaman cabai rawit. Citra input merupakan citra tanaman cabai rawit yang memuat objek daun, batang, dan buah cabai. Citra tersebut kemudian dilakukan proses *resize* menjadi 128 × 128 piksel agar seluruh citra memiliki ukuran yang sama sebelum masuk ke tahap ekstraksi fitur. Tahap *resize* ini penting karena ukuran citra yang seragam akan memudahkan proses pembentukan fitur BoF, HSV, SPM, PCA, dan klasifikasi SVM secara konsisten.

Karena citra telah diubah menjadi ukuran 128 × 128 piksel, maka jumlah seluruh piksel yang dianalisis adalah:

$$N_{total} = 128 \times 128 = 16384$$

Keterangan:

$$N_{total}$$

merupakan jumlah seluruh piksel pada citra hasil *resize*. Dengan ukuran tersebut, setiap citra yang masuk ke sistem memiliki jumlah piksel yang sama, yaitu 16.384 piksel. Citra hasil *resize* inilah yang kemudian digunakan sebagai dasar dalam proses ekstraksi fitur.

2. Ekstraksi fitur BoF

Setelah tahap pra-pemrosesan selesai, proses ekstraksi fitur pertama yang dilakukan adalah ekstraksi fitur Bag-of-Features (BoF). Pada penelitian ini, BoF digunakan untuk merepresentasikan pola lokal, bentuk, dan tekstur pada citra tanaman cabai rawit. Tahap awal dalam BoF dilakukan dengan mengubah citra hasil *resize* ke dalam bentuk *grayscale*, kemudian mendeteksi titik-titik penting atau *keypoint* menggunakan metode ORB.

Keypoint merupakan titik pada citra yang memiliki ciri visual tertentu, seperti sudut, tepi, atau perubahan tekstur. Setiap *keypoint* yang terdeteksi kemudian menghasilkan *descriptor* yang merepresentasikan karakteristik lokal pada area tersebut. Hasil deteksi *keypoint* ORB pada citra contoh ditunjukkan pada Gambar 4.18.



Gambar 4. 18 Hasil Deteksi *Keypoint* ORB

Gambar 4.18 menunjukkan hasil deteksi *keypoint* menggunakan metode ORB pada salah satu citra tanaman cabai rawit. Pada citra contoh tersebut, jumlah *keypoint* yang terdeteksi adalah 204 titik. *Keypoint* tersebut menunjukkan titik-titik penting pada citra yang memiliki pola visual tertentu, seperti tepi, sudut, perubahan tekstur, atau area dengan perbedaan intensitas pada objek tanaman cabai rawit.

Setelah *keypoint* terdeteksi, ORB akan menghasilkan *descriptor* untuk setiap titik penting tersebut. *Descriptor* merupakan representasi numerik dari karakteristik lokal pada area sekitar *keypoint*. Pada ORB, setiap *descriptor* memiliki panjang 32 elemen. Oleh karena itu, apabila pada citra contoh terdeteksi 204 *keypoint*, maka bentuk *descriptor* yang dihasilkan adalah 204 × 32.

Descriptor ORB digunakan sebagai dasar dalam pembentukan fitur Bag-of-Features (BoF). Pada metode BoF, kumpulan *descriptor* dari citra digunakan untuk membentuk *visual vocabulary* melalui proses pengelompokan menggunakan K-Means. Setelah itu, setiap *descriptor* dicocokkan dengan *visual word* terdekat, kemudian dihitung jumlah kemunculannya untuk membentuk histogram BoF. Dengan cara ini, pola lokal pada citra tanaman cabai rawit dapat direpresentasikan dalam bentuk vektor fitur.

Jumlah *descriptor* pada satu citra dapat dinyatakan menggunakan Persamaan:

$$D_{ORB} = N_{kp} \times L_{des}$$

Keterangan:

D_{ORB} = ukuran *descriptor ORB*

N_{kp} = jumlah *keypoint* yang terdeteksi

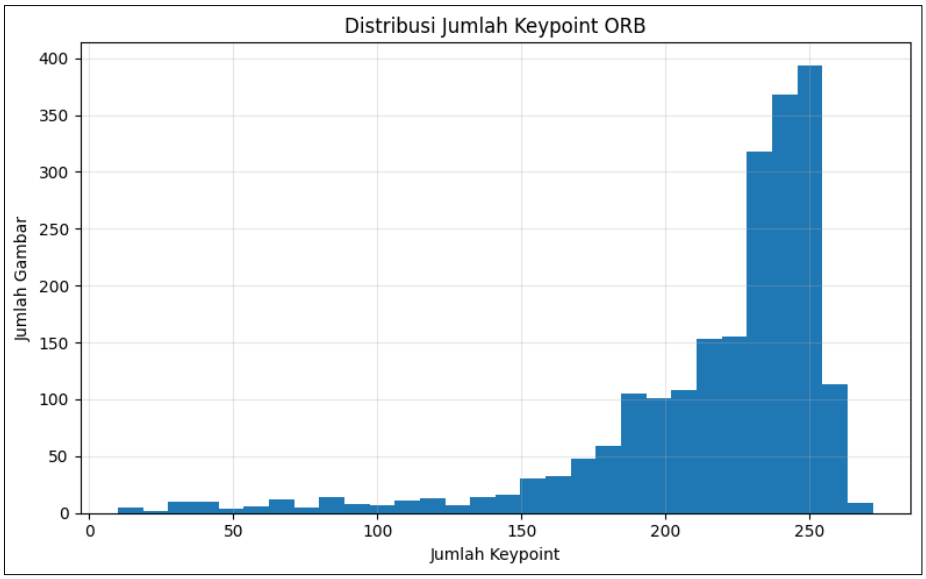
L_{des} = panjang *descriptor ORB*

Pada citra contoh, jumlah *keypoint* yang terdeteksi adalah 204, sedangkan panjang *descriptor ORB* adalah 32. Maka bentuk *descriptor* citra contoh adalah:

$$D_{ORB} = 204 \times 32$$

Hasil tersebut menunjukkan bahwa citra contoh memiliki cukup banyak titik lokal yang dapat digunakan untuk membentuk fitur BoF. Semakin banyak *keypoint* yang terdeteksi, semakin banyak informasi lokal yang dapat digunakan untuk

merepresentasikan pola bentuk dan tekstur pada citra. Setelah hasil *keypoint* contoh diperoleh, dilakukan pula analisis jumlah *keypoint* pada seluruh dataset untuk mengetahui sebaran fitur lokal yang berhasil dideteksi oleh ORB, sebagaimana ditunjukkan pada Gambar 4.19.



Gambar 4. 19 Grafik Distribusi Jumlah *Keypoint* ORB

Gambar 4.19 menunjukkan distribusi jumlah *keypoint* ORB pada seluruh citra *dataset*. Grafik tersebut digunakan untuk mengetahui sebaran jumlah titik penting yang berhasil dideteksi oleh ORB pada setiap citra tanaman cabai rawit. Berdasarkan hasil pengujian, jumlah citra yang digunakan adalah 2137 Gambar. Seluruh citra berhasil menghasilkan *descriptor* ORB, karena jumlah *descriptor* kosong adalah 0 Gambar atau 0,0%.

Rata-rata jumlah *keypoint* yang diperoleh adalah 217,46, dengan jumlah *keypoint* minimum 10 dan maksimum 272. Hasil tersebut menunjukkan bahwa sebagian besar citra memiliki jumlah *keypoint* yang cukup tinggi, sehingga citra tanaman cabai rawit memiliki pola lokal yang dapat digunakan dalam pembentukan fitur BoF. Ringkasan hasil *keypoint* dan *descriptor* ORB ditunjukkan pada Tabel 4.1.

Tabel 4. 1 Ringkasan *Keypoint* dan *Descriptor* ORB

Parameter	Nilai
Jumlah Gambar	2137
Descriptor kosong	0
Persentase *descriptor* kosong	0,0%
Rata-rata *keypoint*	217,46
Minimum *keypoint*	10
Maximum *keypoint*	272
Keypoint contoh citra	204
Shape *descriptor* contoh	204 × 32

Berdasarkan Tabel 4.1, seluruh citra pada *dataset* dapat diproses oleh metode ORB karena tidak terdapat *descriptor* kosong. Hal ini menunjukkan bahwa citra tanaman cabai rawit memiliki fitur lokal yang cukup untuk digunakan dalam pembentukan *visual vocabulary* pada metode BoF. Jumlah *keypoint* contoh citra sebesar 204 titik menghasilkan *descriptor* dengan bentuk 204 × 32, yang berarti setiap *keypoint* direpresentasikan oleh *descriptor* ORB sepanjang 32 elemen.

Descriptor tersebut kemudian digunakan pada proses BoF untuk membentuk histogram *visual word*. Dengan demikian, hasil deteksi ORB menjadi dasar dalam merepresentasikan pola lokal, bentuk, dan tekstur pada citra tanaman cabai rawit sebelum digabungkan dengan fitur warna HSV dan informasi spasial SPM.

3. Ekstraksi fitur HSV

Setelah proses ekstraksi fitur BoF dilakukan, tahap berikutnya adalah ekstraksi fitur warna menggunakan ruang warna HSV. Ekstraksi fitur HSV dilakukan karena warna merupakan salah satu ciri visual penting dalam menentukan kesiapan panen tanaman cabai rawit. Perubahan warna buah cabai rawit dari hijau menuju oranye hingga merah menjadi indikator visual yang berkaitan dengan tingkat kematangan buah.

Pada tahap ini, citra hasil *resize* dikonversi dari ruang warna BGR atau RGB ke ruang warna HSV. Ruang warna HSV terdiri dari tiga komponen, yaitu *Hue*,

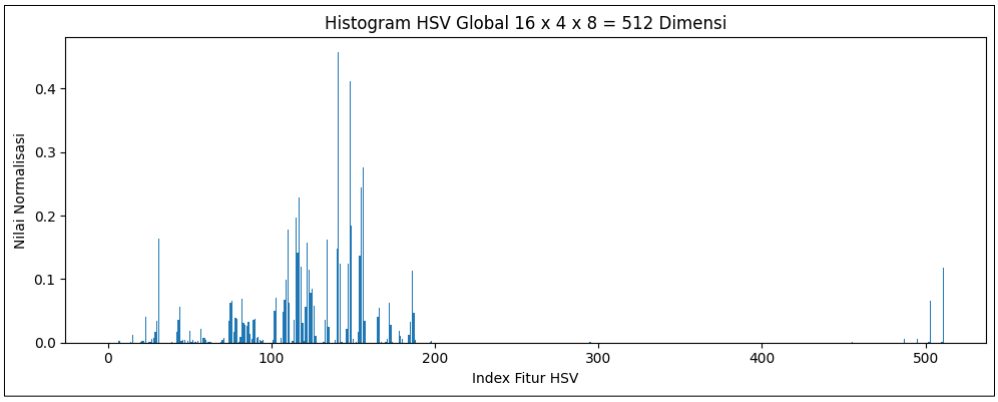
Saturation, dan *Value*. Komponen *Hue* merepresentasikan jenis warna, *Saturation* menunjukkan tingkat kejenuhan warna, sedangkan *Value* menunjukkan tingkat kecerahan warna. Dengan menggunakan ruang warna HSV, informasi warna pada citra tanaman cabai rawit dapat direpresentasikan secara lebih jelas dibandingkan hanya menggunakan citra RGB.



Gambar 4. 20 Visualisasi Citra dalam Ruang Warna HSV

Gambar 4.20 menunjukkan visualisasi citra tanaman cabai rawit dalam ruang warna HSV. Konversi ke ruang warna HSV dilakukan karena warna menjadi salah satu indikator penting dalam menentukan kesiapan panen, terutama perubahan warna cabai dari hijau, oranye, hingga merah. Ruang warna HSV merepresentasikan warna melalui komponen *Hue*, *Saturation*, dan *Value*, sehingga dapat digunakan untuk memperkuat informasi warna pada citra.

Setelah citra dikonversi ke ruang warna HSV, tahap berikutnya adalah pembentukan histogram HSV. Pada penelitian ini, histogram HSV global dibentuk menggunakan 16 *bin* pada kanal *Hue*, 4 *bin* pada kanal *Saturation*, dan 8 *bin* pada kanal *Value*, sehingga menghasilkan 512 fitur warna. Hasil histogram HSV ditunjukkan pada Gambar 4.21.



Gambar 4. 21 Histogram HSV Citra Tanaman Cabai Rawit

Gambar 4.21 menunjukkan histogram HSV global dari citra tanaman cabai rawit. Histogram HSV digunakan untuk merepresentasikan distribusi warna pada citra berdasarkan kanal *Hue*, *Saturation*, dan *Value*. Pada penelitian ini, histogram HSV dibentuk menggunakan konfigurasi 16 *bin* pada kanal *Hue*, 4 *bin* pada kanal *Saturation*, dan 8 *bin* pada kanal *Value*.

Dimensi fitur *HSV* dihitung menggunakan Persamaan:

$$F_{HSV} = B_H \times B_S \times B_V$$

Keterangan:

F_{HSV} = jumlah dimensi fitur *HSV*

B_H = jumlah bin pada kanal hue

B_S = jumlah bin pada kanal saturation

B_V = jumlah bin pada kanal value

Pada penelitian ini, jumlah dimensi fitur *HSV* adalah:

$$F_{HSV} = 16 \times 4 \times 8 = 512$$

Dengan demikian, setiap citra menghasilkan fitur HSV sebanyak 512 fitur. Nilai *histogram* tersebut telah dinormalisasi sehingga dapat digunakan bersama fitur BoF dan SPM pada proses pembentukan fitur gabungan. Selain histogram HSV, dilakukan juga perhitungan persentase warna hijau, oranye, dan merah sebagai informasi pendukung visual kesiapan panen. Hasil persentase warna HSV pada citra contoh ditunjukkan pada Tabel 4.2.

Tabel 4. 2 Persentase Warna HSV pada Citra

Warna	Persentase
Hijau	76,68%
Oranye	1,46%
Merah	8,01%

Persentase warna pada citra dihitung berdasarkan jumlah piksel warna tertentu dibandingkan dengan jumlah seluruh piksel pada citra hasil *resize*. Perhitungan persentase warna dapat dituliskan menggunakan Persamaan:

$$P_{warna} = \frac{N_{warna}}{N_{\ total}} \times 100\%$$

Keterangan:

P_{warna} = persentase warna tertentu

N_{warna} = jumlah piksel pada warna tertentu

N_{total} = jumlah seluruh piksel yang dianalisis

Perhitungan persentase warna hijau dapat dituliskan sebagai berikut:

$$P_{hijau} = \frac{N_{hijau}}{N_{total}} \times 100\%$$

$$P_{hijau} = \frac{12564}{16384} \times 100\%$$

$$P_{hijau} = 76{,}68\%$$

Perhitungan persentase warna oranye dapat dituliskan sebagai berikut:

$$P_{oranye} = \frac{N_{oranye}}{N_{total}} \times 100\%$$

$$P_{oranye} = \frac{239}{16384} \times 100\%$$

$$P_{oranye} = 1{,}46\%$$

Perhitungan persentase warna merah dapat dituliskan sebagai berikut:

$$P_{merah} = \frac{N_{merah}}{N_{total}} \times 100\%$$

$$P_{merah} = \frac{1313}{16384} \times 100\%$$

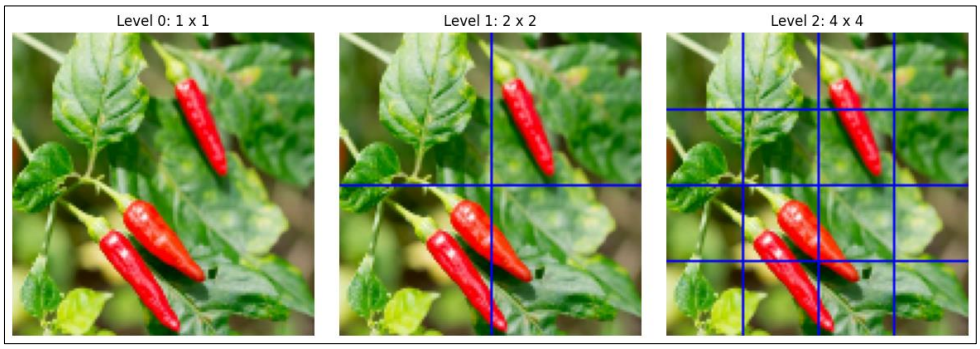
$$P_{merah} = 8{,}01\%$$

Berdasarkan hasil tersebut, citra contoh memiliki dominasi warna hijau sebesar 76,68%, sedangkan warna merah sebesar 8,01% dan warna oranye sebesar 1,46%. Warna hijau pada citra umumnya berasal dari daun, batang, dan bagian tanaman lainnya, sedangkan warna merah dan oranye berasal dari buah cabai yang mengalami proses pematangan. Informasi proporsi warna ini digunakan sebagai salah satu pendukung dalam analisis visual kesiapan panen tanaman cabai rawit.

4. Ekstraksi fitur SPM

Setelah fitur BoF dan HSV diperoleh, tahap berikutnya adalah ekstraksi fitur menggunakan *Spatial Pyramid Matching* (SPM). SPM digunakan untuk mempertahankan informasi spasial atau letak sebaran fitur pada citra. Pada fitur HSV global, informasi warna dihitung berdasarkan keseluruhan citra, sehingga posisi atau lokasi kemunculan warna belum direpresentasikan secara rinci. Oleh karena itu, SPM digunakan untuk membagi citra ke dalam beberapa level grid agar sistem dapat memperoleh informasi sebaran warna dan fitur pada bagian-bagian citra.

Pada penelitian ini, SPM diterapkan menggunakan tiga level, yaitu level 0, level 1, dan level 2. Level 0 membagi citra menjadi 1 × 1 bagian, level 1 membagi citra menjadi 2 × 2 bagian, dan level 2 membagi citra menjadi 4 × 4 bagian. Visualisasi pembagian grid pada metode SPM ditunjukkan pada Gambar 4.22.



Gambar 4. 22 Visualisasi Pembagian *Grid* SPM

Gambar 4.22 menunjukkan proses pembagian citra menggunakan metode *Spatial Pyramid Matching*. Pada level 0, citra dipandang sebagai satu bagian utuh

berukuran 1 × 1. Pada level 1, citra dibagi menjadi 2 × 2 sehingga menghasilkan 4 blok. Pada level 2, citra dibagi menjadi 4 × 4 sehingga menghasilkan 16 blok. Dengan pembagian tersebut, sistem tidak hanya memperoleh informasi warna secara global, tetapi juga mempertahankan informasi sebaran warna pada beberapa bagian citra.

Pada setiap blok SPM dilakukan perhitungan histogram HSV *patch*. Histogram HSV *patch* pada penelitian ini menghasilkan 128 fitur pada setiap blok. Jumlah fitur tersebut diperoleh dari konfigurasi histogram HSV *patch*, yaitu 8 *bin* pada kanal *Hue*, 4 *bin* pada kanal *Saturation*, dan 4 *bin* pada kanal *Value*. Dengan demikian, jumlah fitur pada setiap blok dapat dihitung sebagai berikut:

$$F_{patch} = B_H \times B_S \times B_V$$

Keterangan:

F_{patch}= jumlah fitur pada setiap *patch*

B_H= jumlah *bin* pada kanal *Hue*

B_S= jumlah *bin* pada kanal *Saturation*

B_V= jumlah *bin* pada kanal *Value*

Pada Penelitian ini, jumlah fitur pada setiap blok adalah:

$$F_{patch} = 8 \times 4 \times 4 = 128$$

Ringkasan jumlah blok dan fitur pada proses SPM ditunjukkan pada Tabel 4.3.

Tabel 4. 3 Hasil Pembagian *Grid* SPM

Level SPM	Ukuran Grid	Jumlah Blok	Fitur per Blok	Total Fitur
Level 0	1x1	1 blok	128 fitur	128 fitur
Level 1	2x2	4 blok	128 fitur	512 fitur
Level 2	4x4	16 blok	128 fitur	2048 fitur
Total		21 blok	-	2688 fitur

Berdasarkan Tabel 4.3, total blok yang digunakan pada metode SPM adalah 21 blok. Jumlah tersebut diperoleh dari penjumlahan 1 blok pada level 0, 4 blok pada level

1, dan 16 blok pada level 2. Karena setiap blok menghasilkan 128 fitur, maka total fitur SPM yang dihasilkan dapat dihitung sebagai berikut:

$$F_{SPM} = N_{blok} \times F_{patch}$$

Keterangan:

F_{SPM}= jumlah total fitur SPM

N_{blok}= jumlah seluruh blok SPM

F_{patch}= jumlah fitur pada setiap *patch*

Pada Penelitian ini total fitur SPM adalah:

$$F_{SPM} = 21 \times 128 = 2688$$

Dengan demikian, proses ekstraksi fitur SPM menghasilkan 2688 fitur. Fitur SPM ini digunakan untuk melengkapi fitur BoF dan HSV agar sistem dapat mempertimbangkan informasi sebaran warna dan fitur pada citra tanaman cabai rawit. Penggunaan SPM diharapkan dapat membantu model dalam membedakan kondisi tanaman yang memiliki warna serupa, tetapi memiliki sebaran warna dan pola visual yang berbeda pada area citra.

5. Penggabungan Fitur dan PCA

Setelah fitur BoF, HSV, dan SPM diperoleh, tahap selanjutnya adalah penggabungan fitur. Penggabungan fitur dilakukan untuk membentuk satu vektor fitur yang merepresentasikan citra tanaman cabai rawit secara lebih lengkap. Fitur BoF digunakan untuk merepresentasikan pola lokal dan tekstur, fitur HSV digunakan untuk merepresentasikan distribusi warna global, sedangkan fitur SPM digunakan untuk mempertahankan informasi sebaran fitur berdasarkan pembagian grid spasial.

Tabel 4. 4 Dimensi Fitur Hasil Ekstraksi

Jenis Fitur	Dimensi Fitur
BoF	128
HSV	512
SPM	2688

BOF+HSV	640
BOF+HSV+SPM	3328

Berdasarkan Tabel 4.4, fitur BoF menghasilkan dimensi sebanyak 128 fitur. Nilai tersebut diperoleh dari jumlah *visual word* yang digunakan pada proses pembentukan *vocabulary*. Fitur HSV menghasilkan 512 fitur dari histogram HSV global, sedangkan fitur SPM menghasilkan 2688 fitur berdasarkan pembagian 21 blok SPM dengan 128 fitur pada setiap blok.

Dimensi fitur gabungan dihitung menggunakan Persamaan:

$$F_{gabungan} = F_{BoF} + F_{HSV} + F_{SPM}$$

Keterangan:

$F_{gabungan}$= jumlah dimensi fitur gabungan

F_{BoF}= jumlah dimensi fitur BoF

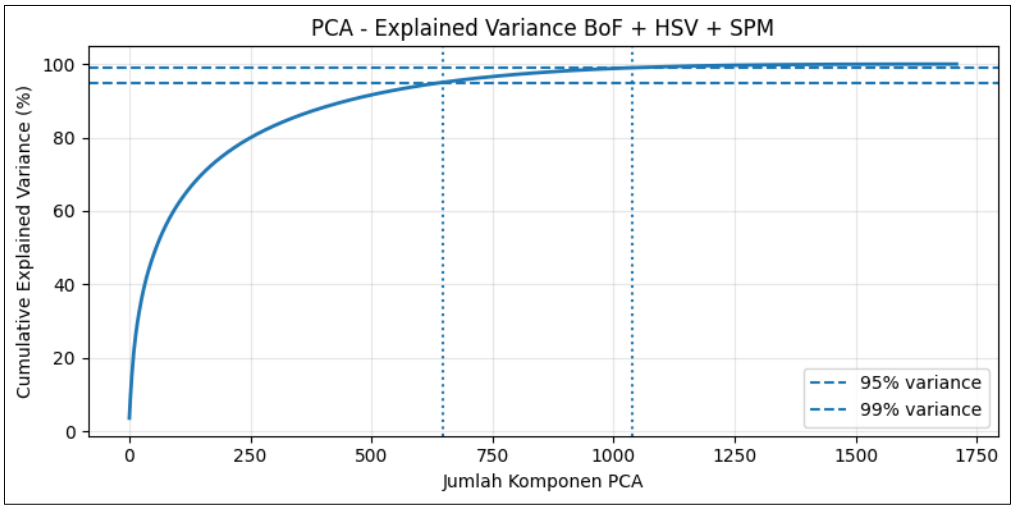
F_{HSV}= jumlah dimensi fitur HSV

F_{SPM}= jumlah dimensi fitur SPM

Pada penelitian ini, dimensi fitur gabungan adalah:

$$F_{gabungan} = 128 + 512 + 2688 = 3328$$

Dengan demikian, metode gabungan BoF + HSV + SPM menghasilkan fitur sebanyak 3328 fitur sebelum dilakukan reduksi dimensi menggunakan PCA. Hasil tersebut menunjukkan bahwa penambahan fitur HSV dan SPM menyebabkan dimensi fitur menjadi lebih besar. Oleh karena itu, pada metode BoF + HSV + SPM dilakukan normalisasi menggunakan *StandardScaler* dan reduksi dimensi menggunakan PCA agar fitur menjadi lebih ringkas sebelum digunakan pada proses klasifikasi SVM.



Gambar 4. 23 Grafik *Cumulative Explained Variance* PCA

Gambar 4.23 menunjukkan grafik Cumulative Explained Variance dari proses *PCA* pada fitur gabungan *BoF + HSV + SPM*. *PCA* digunakan untuk mengurangi dimensi fitur yang semula berjumlah 3328 fitur menjadi lebih ringkas, tetapi tetap mempertahankan informasi utama dari data fitur.

Nilai cumulative explained variance dihitung menggunakan Persamaan:

$$CEV(k) = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{j=1}^{d} \lambda_j} \times 100\%$$

Keterangan:

$CEV(k)$ = cumulative explained variance pada komponen ke-k

λ_i = nilai eigen komponen ke-i

d = jumlah seluruh komponen awal

Berdasarkan hasil pengujian, jumlah komponen *PCA* yang dibutuhkan untuk mempertahankan 95% variance adalah 647 komponen, sedangkan untuk mempertahankan 99% variance adalah 1038 komponen. Pada penelitian ini digunakan 647 komponen *PCA*, karena jumlah tersebut sudah mampu mempertahankan sebagian besar informasi fitur, tetapi dengan dimensi yang lebih kecil dibandingkan fitur awal.

Tabel 4. 5 Ringkasan Hasil PCA

Parameter	Nilai
Dimensi awal *BoF + HSV + SPM*	3328
Komponen *PCA* 95% variance	647
Komponen *PCA* 99% variance	1038
Dimensi setelah *PCA*	647

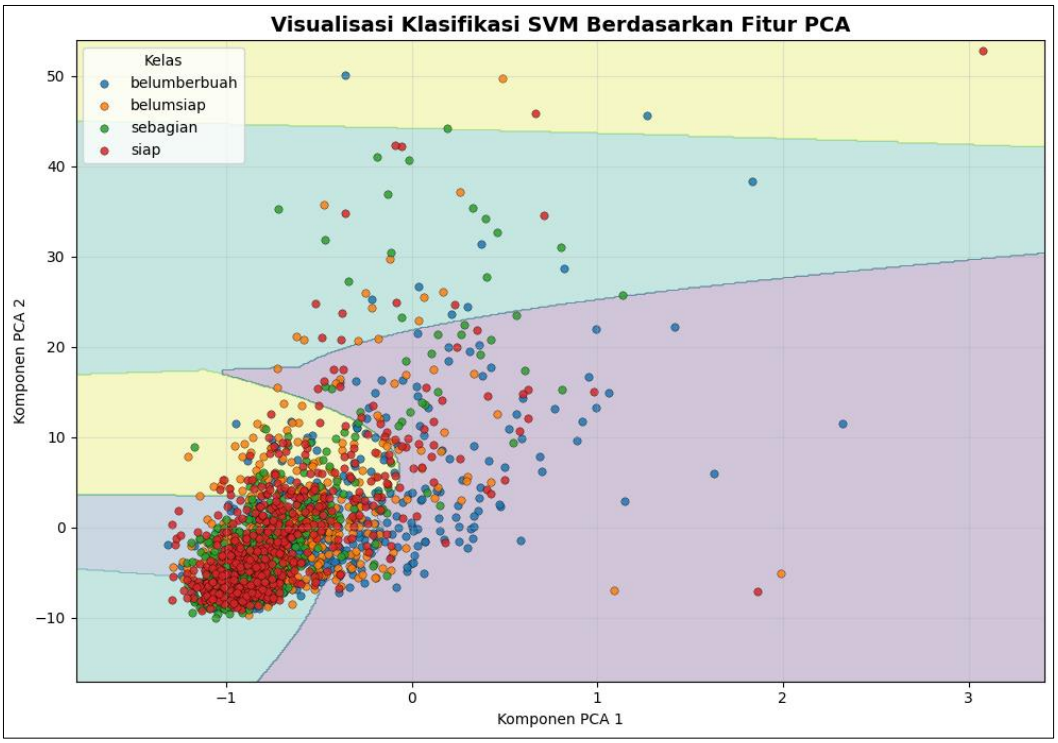
Berdasarkan Tabel 4.5, fitur gabungan *BoF + HSV + SPM* yang awalnya memiliki dimensi 3328 fitur berhasil direduksi menjadi 647 fitur menggunakan *PCA* dengan batas 95% variance. Reduksi dimensi ini bertujuan untuk mengurangi kompleksitas fitur, mempercepat proses komputasi, dan mengurangi kemungkinan fitur yang terlalu besar memengaruhi kinerja model klasifikasi.

6. Klasifikasi Menggunakan SVM

Tahap akhir dari proses pengolahan citra dan ekstraksi fitur adalah klasifikasi menggunakan Support Vector Machine (SVM). Pada sistem Chilivest, keputusan deteksi kesiapan panen tidak hanya ditentukan dari proporsi warna hijau, oranye, dan merah yang ditampilkan pada aplikasi, tetapi berdasarkan fitur gabungan BoF + HSV + SPM yang telah dinormalisasi dan direduksi menggunakan PCA.

BoF digunakan untuk merepresentasikan pola lokal, bentuk, dan tekstur citra, HSV digunakan untuk merepresentasikan informasi warna, sedangkan SPM digunakan untuk mempertahankan informasi sebaran spasial pada citra. Proporsi warna pada aplikasi digunakan sebagai informasi tambahan bagi pengguna, sedangkan keputusan akhir kelas prediksi tetap diperoleh dari hasil klasifikasi SVM.

Fitur gabungan BoF + HSV + SPM menghasilkan 3328 fitur, kemudian dinormalisasi menggunakan *StandardScaler* dan direduksi menggunakan PCA menjadi 647 komponen. Fitur hasil PCA tersebut digunakan sebagai *input* pada SVM dengan *kernel* RBF untuk mengklasifikasikan citra ke dalam empat kelas, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen.



Gambar 4. 24 Visualisasi Klasifikasi SVM Berdasarkan Fitur PCA

Gambar 4.24 menunjukkan visualisasi klasifikasi SVM berdasarkan fitur hasil PCA. Setiap titik merepresentasikan data citra tanaman cabai rawit, sedangkan warna titik menunjukkan kelas citra, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Area warna pada Gambar menunjukkan batas keputusan SVM dalam memisahkan data berdasarkan fitur gabungan BoF, HSV, dan SPM yang telah direduksi menggunakan PCA. Visualisasi ini digunakan sebagai ilustrasi proses klasifikasi dalam bentuk dua dimensi, sedangkan model utama pada aplikasi Chilivest tetap menggunakan fitur hasil PCA sebanyak 647 komponen.

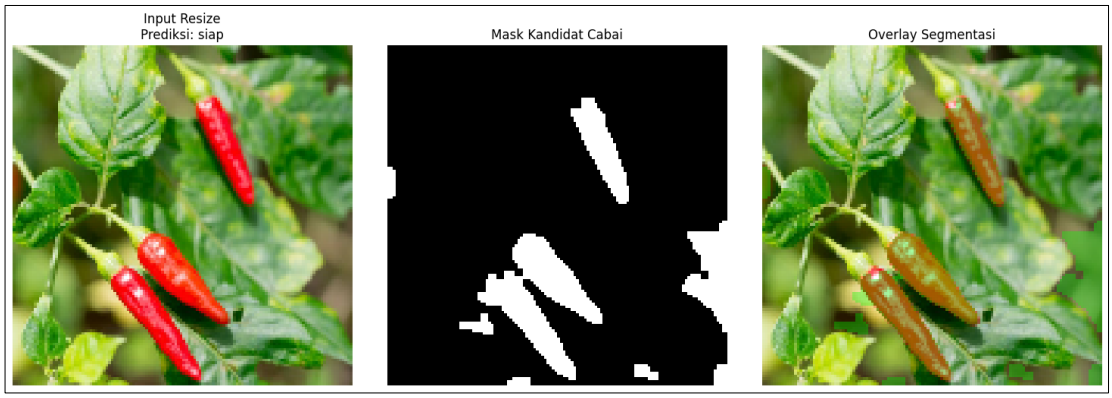
Tabel 4. 6 Ringkasan Visualisasi Klasifikasi SVM Berdasarkan Fitur PCA

Komponen Visual	Keterangan
Sumbu X	Menunjukkan Komponen PCA 1 hasil proyeksi fitur gabungan BoF + HSV + SPM
Sumbu Y	Menunjukkan Komponen PCA 2 hasil proyeksi fitur gabungan BoF + HSV + SPM

Titik data	Merepresentasikan citra tanaman cabai rawit yang telah diproses menjadi fitur numerik
Warna titik biru	Merepresentasikan kelas Belum Berbuah
Warna titik oranye	Merepresentasikan kelas Belum Siap Panen
Warna titik hijau	Merepresentasikan kelas Sebagian Siap Panen
Warna titik merah	Merepresentasikan kelas Siap Panen
Area warna latar	Menunjukkan wilayah keputusan SVM terhadap kelas tertentu
Titik yang berdekatan antar kelas	Menunjukkan adanya kemiripan karakteristik fitur antar kelas pada ruang PCA dua dimensi
Model utama sistem	Menggunakan fitur hasil PCA sebanyak 647 komponen

Tabel 4.6 menunjukkan keterangan dari visualisasi klasifikasi SVM berdasarkan fitur PCA. Titik pada Gambar merepresentasikan data citra tanaman cabai rawit, sedangkan warna titik menunjukkan kelas masing-masing citra. Area warna latar menunjukkan wilayah keputusan SVM dalam memisahkan data berdasarkan fitur hasil PCA. Visualisasi ini digunakan untuk mempermudah pemahaman proses klasifikasi, sedangkan proses klasifikasi utama pada sistem Chilivest tetap menggunakan fitur hasil PCA sebanyak 647 komponen.

Setelah fitur citra diproses melalui BoF, HSV, SPM, normalisasi *StandardScaler*, dan reduksi dimensi menggunakan PCA, fitur hasil PCA digunakan sebagai masukan pada model SVM. Model SVM kemudian melakukan klasifikasi terhadap citra uji untuk menentukan kelas kesiapan panen tanaman cabai rawit. Pada tahap akhir, sistem menghasilkan kelas prediksi, nilai *decision score*, serta informasi visual pendukung berupa citra *input resize*, *mask* kandidat cabai, dan *overlay* segmentasi. Hasil akhir proses klasifikasi SVM pada citra uji ditunjukkan pada Gambar 4.25.



Gambar 4. 25 Hasil Output Akhir Klasifikasi SVM pada Citra Uji

Gambar 4.25 menunjukkan hasil akhir proses klasifikasi SVM pada citra uji. Pada bagian kiri ditampilkan citra hasil *resize* berukuran 128 × 128 piksel dengan hasil prediksi Siap Panen. Bagian tengah menunjukkan *mask* kandidat cabai yang digunakan sebagai informasi visual tambahan, sedangkan bagian kanan menunjukkan *overlay* segmentasi pada citra. Visualisasi tersebut digunakan untuk memperlihatkan bahwa citra uji telah berhasil diproses oleh sistem dan menghasilkan kelas prediksi akhir. Hasil klasifikasi SVM pada citra uji kemudian diringkas pada Tabel 4.7.

Tabel 4. 7 Hasil *Output* Akhir Klasifikasi SVM pada Citra Uji

Komponen	**Nilai**
Kelas contoh	Siap Panen
Fitur masukan SVM	Fitur hasil PCA
Fitur pembentuk	BoF + HSV + SPM
Dimensi fitur awal	3328 fitur
Dimensi setelah PCA	647 komponen
Metode klasifikasi	SVM
Kernel	RBF
Parameter C	10
Gamma	*scale*
Kelas prediksi SVM	Siap Panen
Decision score tertinggi	3,2773
Kelas *score* tertinggi	Siap Panen

Output akhir SVM	Siap Panen

Tabel 4.7 menunjukkan hasil *output* akhir klasifikasi SVM pada citra uji. Citra uji yang digunakan merupakan citra contoh dengan kelas Siap Panen. Proses klasifikasi dilakukan menggunakan fitur hasil PCA yang berasal dari gabungan fitur BoF, HSV, dan SPM. Fitur gabungan tersebut memiliki dimensi awal sebesar 3328 fitur, kemudian direduksi menggunakan PCA menjadi 647 komponen sebelum digunakan sebagai masukan pada SVM.

Pada proses klasifikasi, SVM menggunakan *kernel* RBF dengan parameter C sebesar 10 dan gamma *scale*. Berdasarkan hasil klasifikasi pada citra uji, SVM menghasilkan kelas prediksi Siap Panen. Nilai *decision score* tertinggi yang diperoleh adalah 3,2773 pada kelas Siap Panen, sehingga kelas tersebut dipilih sebagai *output* akhir SVM. Untuk memperjelas proses pembentukan dan pemilihan kelas berdasarkan nilai *decision score*, perhitungan matematis dijelaskan sebelum Tabel 4.8.

Nilai *decision score* pada setiap kelas diperoleh dari hasil perhitungan fungsi keputusan SVM terhadap fitur citra uji. Sebelum masuk ke tahap klasifikasi SVM, citra uji terlebih dahulu melalui proses *resize*, ekstraksi fitur BoF, HSV, dan SPM, normalisasi menggunakan *StandardScaler*, serta reduksi dimensi menggunakan PCA. Fitur hasil PCA tersebut digunakan sebagai masukan pada model SVM untuk menentukan kelas kesiapan panen tanaman cabai rawit.

Proses pembentukan fitur gabungan dapat dinyatakan sebagai berikut:

$$F_{gabungan} = F_{BoF} + F_{HSV} + F_{SPM}$$
$$F_{gabungan} = 128 + 512 + 2688$$
$$F_{gabungan} = 3328 \text{ fitur}$$

Fitur gabungan yang telah diperoleh kemudian dinormalisasi menggunakan *StandardScaler* dengan persamaan sebagai berikut:

$$Z = \frac{F_{gabungan} - \mu}{\sigma}$$

Keterangan:

Z= fitur hasil normalisasi

$F_{gabungan}$= fitur gabungan BoF + HSV + SPM

μ= nilai rata-rata fitur pada data latih

σ= standar deviasi fitur pada data latih

Fitur hasil normalisasi selanjutnya direduksi menggunakan PCA agar dimensi fitur menjadi lebih ringkas. Proses reduksi dimensi tersebut dapat dinyatakan sebagai berikut:

$$X_{PCA} = Z \cdot W_{PCA}$$
$$X_{PCA} = 647 \text{ komponen}$$

Keterangan:

X_{PCA}= fitur hasil PCA yang digunakan sebagai masukan SVM

W_{PCA}= matriks komponen PCA

Pada tahap klasifikasi, model SVM menggunakan *kernel* RBF. Fungsi *kernel* RBF yang digunakan dapat dituliskan sebagai berikut:

$$K(x_i, x) = \exp(-\gamma \parallel x_i - x \parallel^2)$$

Keterangan:

$K(x_i, x)$= nilai *kernel* antara data latih dan data uji

x_i= data latih yang menjadi *support vector*

x= data uji yang diklasifikasikan

γ= parameter *kernel* RBF

$\parallel x_i - x \parallel^2$= jarak kuadrat antara data latih dan data uji

Nilai *decision score* pada setiap kelas dihitung menggunakan fungsi keputusan SVM sebagai berikut:

$$D_k(x) = \sum_{i=1}^{n} \alpha_i \, y_i K(x_i, x) + b$$

Keterangan:

$D_k(x)$= nilai *decision score* untuk kelas ke-k

α_i= bobot pada *support vector*

y_i= label kelas data latih

$K(x_i, x)$= fungsi *kernel* RBF

b= nilai bias

k= kelas yang diuji

Berdasarkan hasil perhitungan fungsi keputusan SVM pada citra uji, nilai *decision score* pada setiap kelas ditunjukkan sebagai berikut:

$$D_{Belum\ Berbuah}(x) = -0{,}2616$$
$$D_{Belum\ Siap\ Panen}(x) = 0{,}7883$$
$$D_{Sebagian\ Siap\ Panen}(x) = 2{,}1009$$
$$D_{Siap\ Panen}(x) = 3{,}2773$$

Keputusan akhir klasifikasi ditentukan berdasarkan kelas dengan nilai *decision score* tertinggi. Aturan pemilihan kelas dapat dinyatakan sebagai berikut:

$$\hat{y} = \arg\max_k D_k(x)$$
$$\hat{y} = \arg\max(-0{,}2616,\ 0{,}7883,\ 2{,}1009,\ 3{,}2773)$$
$$\hat{y} = Siap\ Panen$$

Keterangan:

$\hat{y}$= kelas prediksi Akhir

$\arg\max_k D_k(x)$= proses pemilihan kelas dengan nilai *decision score* tertinggi

Berdasarkan hasil perhitungan fungsi keputusan SVM pada citra uji, diperoleh nilai *decision score* pada masing-masing kelas. Nilai *decision score* tersebut digunakan untuk menentukan kelas akhir dengan cara memilih nilai terbesar. Hasil perhitungan nilai *decision score* pada setiap kelas disajikan pada Tabel 4.8.

Tabel 4. 8 Nilai *Decision Score* SVM pada Setiap Kelas

Kelas	***Decision Score***
Belum Berbuah	-0,2616
Belum Siap Panen	0,7883
Sebagian Siap Panen	2,1009
Siap Panen	3,2773

Berdasarkan Tabel 4.8, kelas Siap Panen memperoleh nilai *decision score* tertinggi sebesar 3,2773. Nilai tersebut lebih tinggi dibandingkan kelas Sebagian Siap Panen sebesar 2,1009, Belum Siap Panen sebesar 0,7883, dan Belum Berbuah sebesar -0,2616. Oleh karena itu, SVM menentukan citra uji sebagai kelas Siap Panen. Hasil ini menunjukkan bahwa *output* akhir klasifikasi diperoleh dari perbandingan nilai *decision score* pada setiap kelas, sehingga kelas dengan nilai tertinggi dipilih sebagai hasil prediksi akhir.

Dengan demikian, proses klasifikasi SVM pada sistem Chilivest tidak hanya menghasilkan kelas prediksi, tetapi juga menyediakan nilai *decision score* yang menunjukkan dasar pemilihan kelas. Nilai tersebut memperjelas bahwa kelas Siap Panen dipilih karena memiliki skor keputusan paling tinggi dibandingkan tiga kelas lainnya.

4.2.3 Pengujian Perbandingan Metode

Pengujian perbandingan metode dilakukan untuk mengetahui kinerja setiap metode ekstraksi fitur dalam mengklasifikasikan kesiapan panen tanaman cabai rawit. Metode yang diuji terdiri dari BoF, BoF + HSV, dan BoF + HSV + SPM. Pengujian ini bertujuan untuk melihat pengaruh penambahan fitur warna HSV dan informasi spasial SPM terhadap hasil klasifikasi.

Setiap metode diuji menggunakan data uji yang sama dan model klasifikasi SVM dengan parameter terbaik hasil proses pencarian parameter. Hasil pengujian kemudian dievaluasi berdasarkan nilai accuracy, macro precision, macro recall, dan macro F1-score. Selain itu, setiap metode juga dianalisis menggunakan confusion

matrix untuk mengetahui jumlah prediksi benar dan kesalahan klasifikasi pada masing-masing kelas.

1. Penjelasan Skenario Metode

Kelas yang digunakan dalam pengujian terdiri dari empat kelas, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Keempat kelas tersebut digunakan sebagai label klasifikasi dalam pengujian setiap metode. Hasil prediksi pada masing-masing kelas kemudian dievaluasi menggunakan *confusion matrix*, *accuracy*, *precision*, *recall*, dan *F1-score*.

a. BoF

Metode BoF digunakan sebagai metode dasar dalam pengujian. Pada metode ini, citra tanaman cabai rawit direpresentasikan berdasarkan fitur lokal yang diperoleh dari deteksi *keypoint* dan *descriptor* ORB. *Descriptor* tersebut kemudian dikelompokkan menggunakan K-Means untuk membentuk *visual vocabulary*. Hasil akhirnya berupa histogram BoF yang merepresentasikan pola lokal dan tekstur pada citra.

b. BoF + HSV

Metode BoF + HSV merupakan pengembangan dari metode BoF dengan menambahkan fitur warna HSV. Pada skenario ini, fitur BoF digunakan untuk merepresentasikan pola lokal dan tekstur, sedangkan fitur HSV digunakan untuk merepresentasikan distribusi warna pada citra. Penambahan fitur HSV dilakukan karena warna merupakan salah satu indikator penting dalam membedakan tingkat kesiapan panen cabai rawit.

c. BoF + HSV + SPM

Metode BoF + HSV + SPM merupakan metode utama yang digunakan dalam penelitian ini. Pada skenario ini, fitur BoF, HSV, dan SPM digabungkan untuk memperoleh representasi citra yang lebih lengkap. BoF merepresentasikan pola lokal, HSV merepresentasikan informasi warna, sedangkan SPM digunakan untuk mempertahankan informasi spasial atau sebaran fitur pada beberapa bagian

citra. Fitur gabungan tersebut kemudian direduksi menggunakan PCA sebelum diklasifikasikan menggunakan SVM.

2. Rumus Evaluasi Klasifikasi

Pada pengujian multi-kelas, nilai *accuracy* dapat dihitung berdasarkan jumlah prediksi benar dibandingkan dengan jumlah seluruh data uji, sehingga Persamaan yang digunakan adalah:

$$Accuracy = \frac{Jumlah\ prediksi\ benar}{Jumlah\ seluruh\ data\ uji} \times 100\%$$

Precision digunakan untuk mengetahui ketepatan model dalam memprediksi suatu kelas. Nilai *precision* dihitung menggunakan Persamaan:

$$Precision = \frac{TP}{TP + FP}$$

Recall digunakan untuk mengetahui kemampuan model dalam menemukan seluruh data yang benar pada suatu kelas dan dihitung menggunakan Persamaan:

$$Recall = \frac{TP}{TP + FN}$$

F1-score digunakan untuk mengetahui keseimbangan antara *precision* dan *recall*. Nilai *F1-score* dihitung menggunakan Persamaan:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Keterangan:

TP= *True Positive*

FP= *False Positive*

FN= *False Negative*

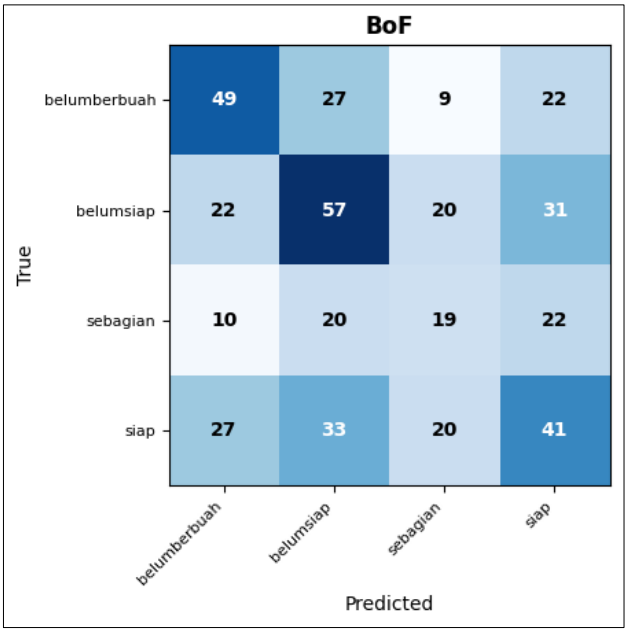
Selanjutnya, karena pengujian dilakukan pada empat kelas, nilai *precision*, *recall*, dan *F1-score* ditampilkan dalam bentuk macro average. Nilai macro average dihitung dengan merata-ratakan nilai evaluasi dari seluruh kelas.

- $Precision_{macro} = \frac{Precision_1 + Precision_2 + Precision_3 + Precision_4}{4}$

- $Recall_{macro} = \frac{Recall_1+Recall_2+Recall_3+Recall_4}{4}$
- $F1_{macro} = \frac{F1_1+F1_2+F1_3+F1_4}{4}$

3. Metode BoF

Pengujian pertama dilakukan menggunakan metode BoF sebagai metode dasar. Pada metode ini, fitur yang digunakan berasal dari pola lokal dan tekstur citra tanaman cabai rawit. Fitur BoF diperoleh dari hasil deteksi *keypoint* dan *descriptor* ORB, kemudian dikelompokkan menggunakan K-Means untuk membentuk *visual vocabulary*. Hasil akhirnya berupa histogram BoF yang digunakan sebagai masukan pada proses klasifikasi menggunakan SVM.



Gambar 4. 26 *Confusion Matrix* Metode BoF

Gambar 4.26 menunjukkan hasil *confusion matrix* metode BoF pada empat kelas, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Nilai pada diagonal utama menunjukkan jumlah data yang berhasil diprediksi dengan benar, sedangkan nilai di luar diagonal utama menunjukkan jumlah data yang salah diklasifikasikan ke kelas lain. Ringkasan jumlah data uji, prediksi benar, dan prediksi salah pada metode BoF ditunjukkan pada Tabel 4.9 sebagai berikut.

Tabel 4. 9 Hasil C*onfusion* M*atrix* Metode BoF

Kelas Aktual	Jumlah Data Uji	Prediksi Benar	Prediksi Salah
Belum Berbuah	107	49	58
Belum Siap Panen	130	57	73
Sebagian Siap Panen	71	19	52
Siap Panen	121	41	80
Total	**429**	**166**	**263**

Berdasarkan Tabel 4.9, metode BoF menghasilkan prediksi benar sebanyak 166 data dari total 429 data uji. Prediksi benar diperoleh dari jumlah nilai pada diagonal utama *confusion matrix*, sedangkan prediksi salah diperoleh dari nilai di luar diagonal utama. Nilai tersebut kemudian digunakan sebagai dasar untuk menghitung *accuracy*, macro *precision*, macro *recall*, dan macro *F1-score* pada metode BoF.

- *Accuracy*

$$Accuracy = \frac{Jumlah\ prediksi\ benar}{Jumlah\ seluruh\ data\ uji} \times 100\%$$

$$Accuracy_{BoF} = \frac{166}{429} \times 100\%$$

$$Accuracy_{BoF} = 38{,}69\%$$

- *Precision Macro*

$$Precision_{macro} = \frac{Precision_1 + Precision_2 + Precision_3 + Precision_4}{4}$$

$$Precision_{macro} = 37{,}57\%$$

- *Recall Macro*

$$Recall_{macro} = \frac{Recall_1 + Recall_2 + Recall_3 + Recall_4}{4}$$

$$Recall_{macro} = 37{,}57\%$$

- *F1-Score Macro*

$$F1_{macro} = \frac{F1_1 + F1_2 + F1_3 + F1_4}{4}$$

$$F1_{macro} = 37{,}55\%$$

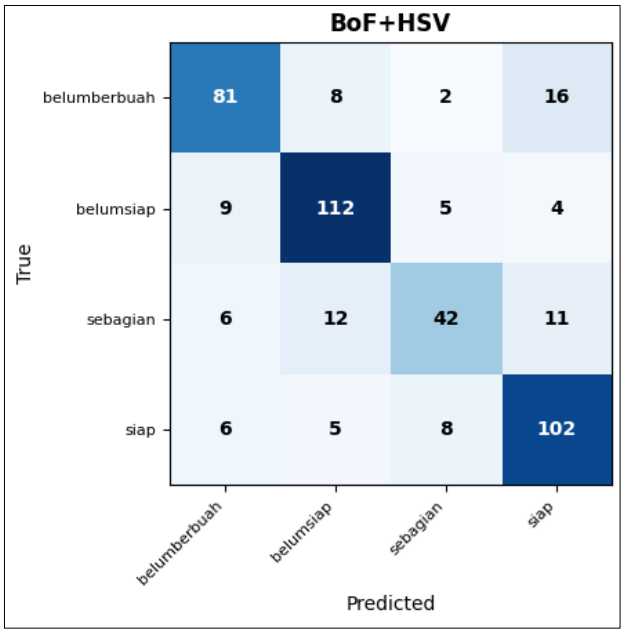
Tabel 4. 10 Hasil Evaluasi Metode BoF

Parameter Evaluasi	Nilai
Accuracy	38,69%
Macro *Precision*	37,57%
Macro *Recall*	37,57%
Macro *F1-score*	37,55%

Berdasarkan Tabel 4.10, metode BoF memperoleh nilai *accuracy* sebesar 38,69%, macro *precision* sebesar 37,57%, macro *recall* sebesar 37,57%, dan macro *F1-score* sebesar 37,55%. Nilai tersebut menunjukkan bahwa metode BoF belum mampu memberikan performa klasifikasi yang optimal dalam mendeteksi kesiapan panen tanaman cabai rawit. Hal ini disebabkan karena BoF hanya merepresentasikan pola lokal dan tekstur citra, sedangkan perbedaan tingkat kesiapan panen tanaman cabai rawit sangat dipengaruhi oleh informasi warna buah, seperti hijau, oranye, dan merah. Oleh karena itu, metode BoF masih menghasilkan banyak kesalahan klasifikasi pada beberapa kelas.

4. Metode BoF + HSV

Pengujian kedua dilakukan menggunakan metode BoF + HSV. Pada metode ini, fitur BoF yang merepresentasikan pola lokal dan tekstur citra digabungkan dengan fitur warna HSV. Penambahan fitur HSV bertujuan untuk memberikan informasi warna pada citra tanaman cabai rawit, karena tingkat kesiapan panen sangat dipengaruhi oleh perubahan warna buah, seperti hijau, oranye, dan merah. Fitur gabungan BoF + HSV kemudian digunakan sebagai masukan pada proses klasifikasi menggunakan SVM.



Gambar 4. 27 *Confusion Matrix* Metode BoF + HSV

Gambar 4.27 menunjukkan hasil *confusion matrix* metode BoF + HSV pada empat kelas, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Nilai pada diagonal utama menunjukkan jumlah data yang berhasil diprediksi dengan benar, sedangkan nilai di luar diagonal utama menunjukkan jumlah data yang salah diklasifikasikan ke kelas lain. Ringkasan jumlah data uji, prediksi benar, dan prediksi salah pada metode BoF + HSV ditunjukkan pada Tabel 4.11 sebagai berikut.

Tabel 4. 11 Hasil *Confusion Matrix* Metode BoF + HSV

Kelas Aktual	Jumlah Data Uji	Prediksi Benar	Prediksi Salah
Belum Berbuah	107	81	26
Belum Siap Panen	130	112	18
Sebagian Siap Panen	71	42	29
Siap Panen	121	102	19
Total	**429**	**337**	**92**

Berdasarkan Tabel 4.11, metode BoF + HSV menghasilkan prediksi benar sebanyak 337 data dari total 429 data uji. Prediksi benar diperoleh dari jumlah nilai pada diagonal utama *confusion matrix*, sedangkan prediksi salah diperoleh dari nilai di

luar diagonal utama. Nilai tersebut kemudian digunakan sebagai dasar untuk menghitung *accuracy*, macro *precision*, macro *recall*, dan macro *F1-score* pada metode BoF + HSV.

- *Accuracy*

$$Accuracy = \frac{Jumlah\ prediksi\ benar}{Jumlah\ seluruh\ data\ uji} \times 100\%$$
$$Accuracy_{BoF+HSV} = \frac{337}{429} \times 100\%$$
$$Accuracy_{BoF+HSV} = 78{,}55\%$$

- Macro *Precision*

$$Precision_{macro} = \frac{Precision_1 + Precision_2 + Precision_3 + Precision_4}{4}$$
$$Precision_{macro} = 77{,}88\%$$

- Macro *Recall*

$$Recall_{macro} = \frac{Recall_1 + Recall_2 + Recall_3 + Recall_4}{4}$$
$$Recall_{macro} = 76{,}33\%$$

- Macro *F1-score*

$$F1_{macro} = \frac{F1_1 + F1_2 + F1_3 + F1_4}{4}$$
$$F1_{macro} = 76{,}84\%$$

Tabel 4. 12 Hasil Evaluasi Metode BoF + HSV

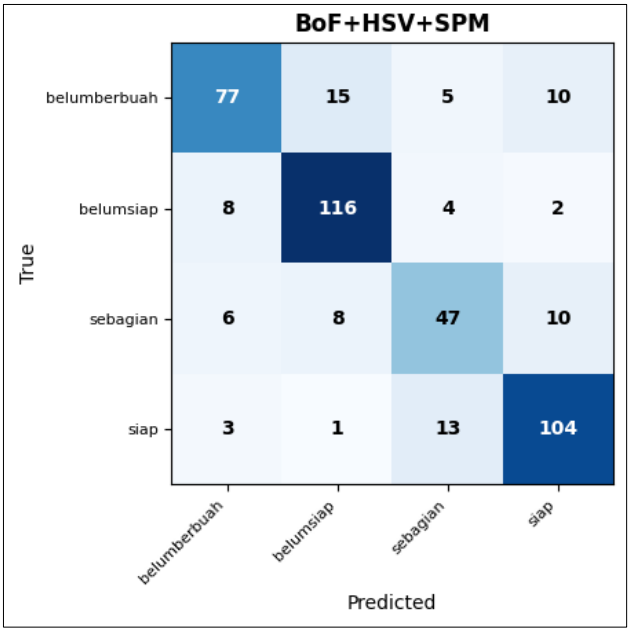
Parameter Evaluasi	Nilai
Accuracy	78,55%
Macro *Precision*	77,88%
Macro *Recall*	76,33%
Macro *F1-score*	76,84%

Berdasarkan Tabel 4.12, metode BoF + HSV memperoleh nilai *accuracy* sebesar 78,55%, macro *precision* sebesar 77,88%, macro *recall* sebesar 76,33%, dan macro *F1-score* sebesar 76,84%. Nilai tersebut menunjukkan bahwa penambahan fitur warna HSV mampu meningkatkan performa klasifikasi secara signifikan dibandingkan metode BoF. Hal ini terjadi karena fitur HSV dapat merepresentasikan informasi warna

cabai rawit yang menjadi indikator penting dalam menentukan tingkat kesiapan panen. Dengan demikian, metode BoF + HSV mampu mengurangi jumlah kesalahan klasifikasi dibandingkan metode BoF yang hanya menggunakan pola lokal dan tekstur citra.

5. Metode BoF + HSV + SPM

Pengujian ketiga dilakukan menggunakan metode BoF + HSV + SPM. Pada metode ini, fitur BoF digunakan untuk merepresentasikan pola lokal dan tekstur citra, fitur HSV digunakan untuk merepresentasikan informasi warna, sedangkan SPM digunakan untuk mempertahankan informasi spasial atau sebaran fitur pada citra Kombinasi ketiga fitur tersebut bertujuan agar citra tanaman cabai rawit dapat direpresentasikan secara lebih lengkap. Fitur gabungan BoF + HSV + SPM kemudian direduksi menggunakan PCA sebelum diklasifikasikan menggunakan SVM.



Gambar 4. 28 *Confusion Matrix* Metode BoF + HSV + SPM

Gambar 4.28 menunjukkan hasil *confusion matrix* metode BoF + HSV + SPM pada empat kelas, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Nilai pada diagonal utama menunjukkan jumlah data yang berhasil diprediksi dengan benar, sedangkan nilai di luar diagonal utama menunjukkan jumlah

data yang salah diklasifikasikan ke kelas lain. Ringkasan jumlah data uji, prediksi benar, dan prediksi salah pada metode BoF + HSV + SPM ditunjukkan pada Tabel 4.13 sebagai berikut.

Tabel 4. 13 Hasil *Confusion Matrix* Metode BoF + HSV + SPM

Kelas Aktual	Jumlah Data Uji	Prediksi Benar	Prediksi Salah
Belum Berbuah	107	77	30
Belum Siap Panen	130	116	14
Sebagian Siap Panen	71	47	24
Siap Panen	121	104	17
Total	**429**	**344**	**85**

Berdasarkan Tabel 4.13, metode BoF + HSV + SPM menghasilkan prediksi benar sebanyak 344 data dari total 429 data uji. Prediksi benar diperoleh dari jumlah nilai pada diagonal utama *confusion matrix*, sedangkan prediksi salah diperoleh dari nilai di luar diagonal utama. Nilai tersebut kemudian digunakan sebagai dasar untuk menghitung *accuracy*, macro *precision*, macro *recall*, dan macro *F1-score* pada metode BoF + HSV + SPM.

- *Accuracy*

$$Accuracy = \frac{Jumlah\ prediksi\ benar}{Jumlah\ seluruh\ data\ uji} \times 100\%$$
$$Accuracy_{BoF+HSV+SPM} = \frac{344}{429} \times 100\%$$
$$Accuracy_{BoF+HSV+SPM} = 80{,}19\%$$

- Macro *Precision*

$$Precision_{macro} = \frac{Precision_1 + Precision_2 + Precision_3 + Precision_4}{4}$$
$$Precision_{macro} = 78{,}86\%$$

- Macro *Recall*

$$Recall_{macro} = \frac{Recall_1 + Recall_2 + Recall_3 + Recall_4}{4}$$
$$Recall_{macro} = 78{,}34\%$$

- Macro *F1-score*

$$F1_{macro} = \frac{F1_1 + F1_2 + F1_3 + F1_4}{4}$$

$$F1_{macro} = 78{,}47\%$$

Tabel 4. 14 Hasil Evaluasi Metode BoF + HSV + SPM

Parameter Evaluasi	Nilai
Accuracy	80,19%
Macro *Precision*	78,86%
Macro *Recall*	78,34%
Macro *F1-score*	78,47%

Berdasarkan Tabel 4.14, metode BoF + HSV + SPM memperoleh nilai *accuracy* sebesar 80,19%, macro *precision* sebesar 78,86%, macro *recall* sebesar 78,34%, dan macro *F1-score* sebesar 78,47%. Nilai tersebut menunjukkan bahwa metode BoF + HSV + SPM menghasilkan performa terbaik dibandingkan metode BoF dan BoF + HSV. Peningkatan ini terjadi karena metode BoF + HSV + SPM tidak hanya memanfaatkan pola lokal dan tekstur citra, tetapi juga menambahkan informasi warna dan sebaran spasial fitur pada citra. Dengan demikian, metode ini mampu memberikan representasi citra yang lebih lengkap dalam mendeteksi kesiapan panen tanaman cabai rawit.

6. Hasil Perbandingan Metode

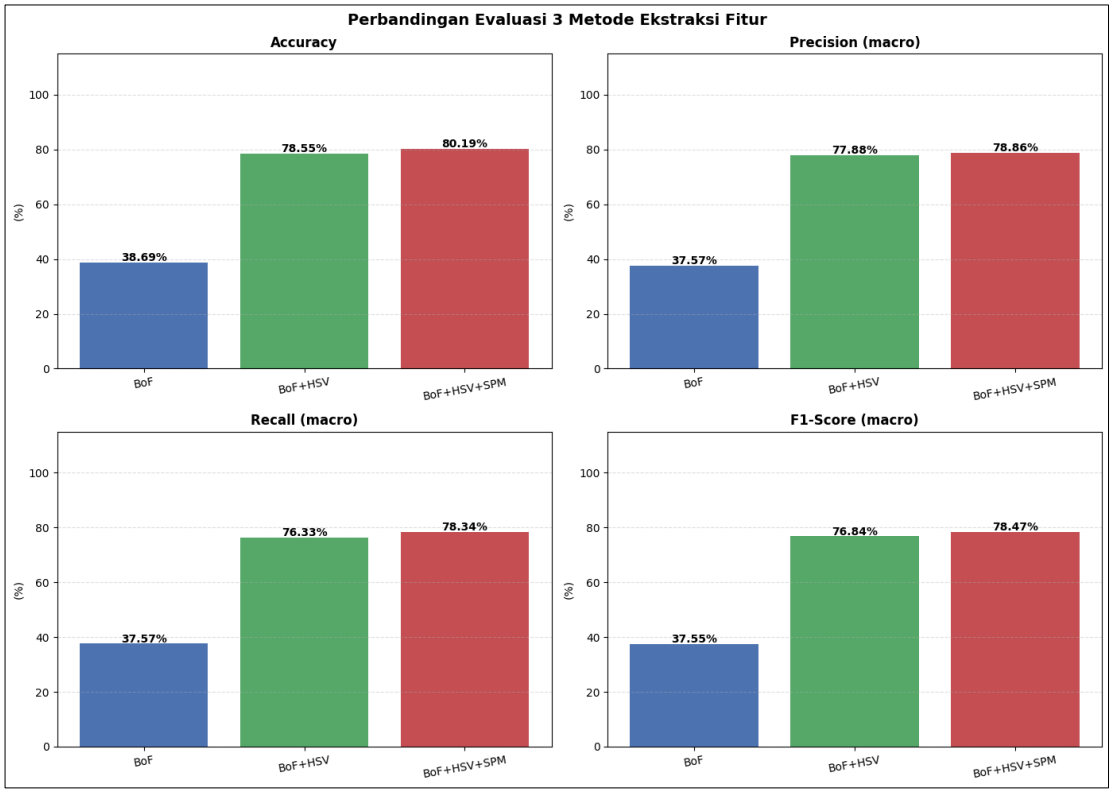
Setelah dilakukan pengujian pada masing-masing metode, hasil evaluasi kemudian dirangkum untuk mengetahui perbandingan performa antara metode BoF, BoF + HSV, dan BoF + HSV + SPM. Perbandingan dilakukan berdasarkan nilai *accuracy*, macro *precision*, macro *recall*, dan macro *F1-score*. Ringkasan hasil perbandingan metode ditunjukkan pada Tabel 4.15.

Tabel 4. 15 Ringkasan Perbandingan Metrik Evaluasi

Metode	*Accuracy*	Macro *Precision*	Macro *Recall*	Macro *F1-score*
BoF	38,69%	37,57%	37,57%	37,55%

BoF + HSV	78,55%	77,88%	76,33%	76,84%
BoF + HSV + SPM	80,19%	78,86%	78,34%	78,47%

Berdasarkan Tabel 4.15, hasil evaluasi menunjukkan adanya peningkatan performa dari metode BoF, BoF + HSV, hingga BoF + HSV + SPM. Untuk memperjelas perbandingan nilai *accuracy*, macro *precision*, macro *recall*, dan macro *F1-score* pada masing-masing metode, hasil evaluasi tersebut juga divisualisasikan dalam bentuk grafik sebagaimana ditunjukkan pada Gambar 4.29.



Gambar 4. 29 Grafik Perbandingan Metrik Evaluasi Metode

Gambar 4.29 menunjukkan bahwa metode BoF memiliki nilai evaluasi paling rendah pada seluruh parameter, yaitu *accuracy* sebesar 38,69%, macro *precision* sebesar 37,57%, macro *recall* sebesar 37,57%, dan macro *F1-score* sebesar 37,55%. Setelah fitur HSV ditambahkan, metode BoF + HSV mengalami peningkatan yang signifikan dengan *accuracy* sebesar 78,55%, macro *precision* sebesar 77,88%, macro *recall* sebesar 76,33%, dan macro *F1-score* sebesar 76,84%.

Selanjutnya, metode BoF + HSV + SPM memperoleh nilai tertinggi pada seluruh parameter evaluasi, yaitu *accuracy* sebesar 80,19%, macro *precision* sebesar 78,86%, macro *recall* sebesar 78,34%, dan macro *F1-score* sebesar 78,47%. Hasil tersebut menunjukkan bahwa penambahan fitur HSV memberikan pengaruh besar terhadap peningkatan performa klasifikasi, sedangkan penambahan SPM memberikan peningkatan lanjutan karena mampu mempertahankan informasi spasial atau sebaran fitur pada citra.

Dengan demikian, berdasarkan Tabel 4.12 dan Gambar 4.24, metode BoF + HSV + SPM dipilih sebagai metode utama dalam sistem deteksi kesiapan panen tanaman cabai rawit. Metode ini memiliki performa terbaik karena mampu menggabungkan pola lokal, informasi warna, dan informasi spasial citra.

4.2.4 Pengujian Fungsionalitas Aplikasi

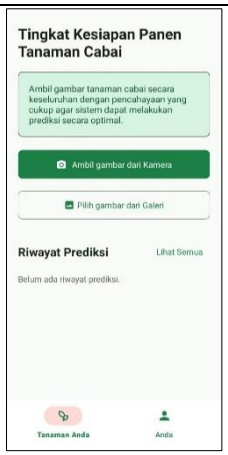
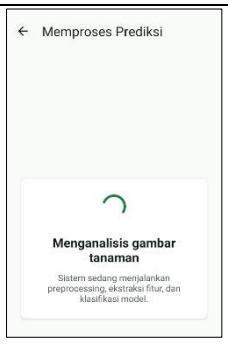
Pengujian fungsionalitas aplikasi dilakukan untuk memastikan setiap menu dan fitur pada aplikasi *Chilivest* dapat berjalan sesuai dengan rancangan sistem. Pengujian ini dilakukan dengan metode black box, yaitu mengamati hasil keluaran aplikasi berdasarkan aksi yang dilakukan oleh pengguna tanpa melihat proses internal program secara langsung. Pengujian dibagi menjadi dua bagian, yaitu pengujian fungsionalitas aplikasi pada menu petani dan pengujian fungsionalitas aplikasi pada menu peneliti.

1. Pengujian Fungsionalitas Aplikasi Menu Petani

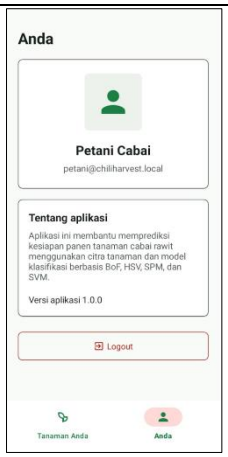
Pengujian fungsionalitas menu petani dilakukan untuk memastikan fitur yang digunakan oleh pengguna petani dapat berjalan dengan baik. Menu petani meliputi *splash screen*, pemilihan jenis pengguna, *onboarding* petani, menu tanaman, pengambilan Gambar melalui kamera, pemilihan Gambar melalui galeri, proses prediksi, tampilan hasil prediksi, riwayat prediksi, penghapusan riwayat, menu akun, dan *logout*.

Tabel 4. 16 Pengujian Fungsionalitas Aplikasi Menu Petani

No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian	Keterangan
1.	*Splash screen*	Pengguna membuka aplikasi *Chilivest*	Aplikasi menampilkan logo dan nama aplikasi sebelum masuk ke halaman berikutnya	Berhasil	
2.	Pemilihan jenis pengguna	Pengguna memilih menu *Login* Sebagai Petani	Aplikasi berpindah ke halaman *onboarding* petani	Berhasil	
3.	*Onboarding* petani	Pengguna menekan tombol Berikutnya	Aplikasi menampilkan halaman *onboarding* berikutnya	Berhasil	
4.	Lewati *onboarding*	Pengguna menekan tombol Lewati	Aplikasi langsung masuk ke menu utama petani	Berhasil	

5.	Menu Tanaman Anda	Pengguna membuka menu Tanaman Anda	Aplikasi menampilkan instruksi, tombol kamera, galeri, dan riwayat prediksi	Berhasil	
6.	Kamera	Pengguna menekan tombol Ambil Gambar dari Kamera	Aplikasi membuka kamera untuk mengambil Gambar tanaman cabai rawit	Berhasil	
7.	Galeri	Pengguna menekan tombol Pilih dari Galeri	Aplikasi membuka galeri untuk memilih Gambar tanaman cabai rawit	Berhasil	
8.	Proses prediksi	Pengguna memilih atau mengambil Gambar	Aplikasi menampilkan halaman *loading* saat proses analisis citra berlangsung	Berhasil	

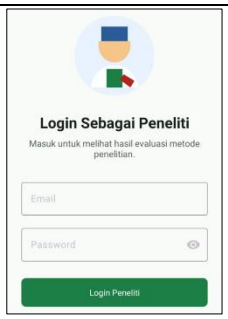
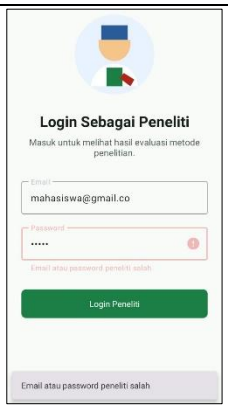
9.	Hasil prediksi	Proses prediksi selesai	Aplikasi menampilkan hasil prediksi, *confidence*, proporsi warna, alasan, dan rekomendasi	Berhasil	
10.	Riwayat prediksi	Pengguna membuka menu riwayat prediksi	Aplikasi menampilkan daftar hasil prediksi yang pernah dilakukan	Berhasil	
11.	Detail riwayat	Pengguna memilih salah satu data riwayat	Aplikasi menampilkan kembali detail hasil prediksi yang dipilih	Berhasil	

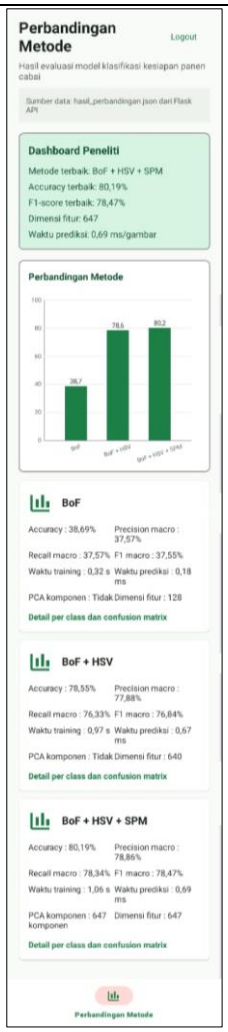
12.	Hapus riwayat	Pengguna menekan tombol Hapus pada salah satu riwayat	Aplikasi menampilkan konfirmasi dan menghapus data riwayat yang dipilih	Berhasil	
13.	Menu Anda	Pengguna membuka menu Anda	Aplikasi menampilkan nama pengguna, email, informasi aplikasi, versi aplikasi, dan tombol *logout*	Berhasil	
14	*Logout*	Pengguna menekan tombol *Logout*	Aplikasi keluar dari sesi pengguna dan kembali ke halaman pemilihan jenis pengguna	Berhasil	

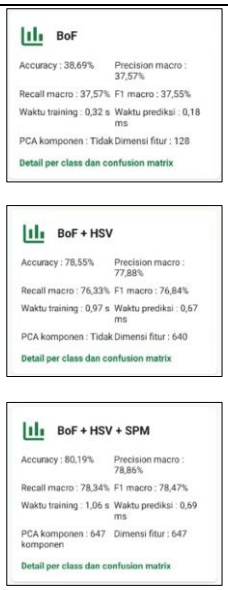
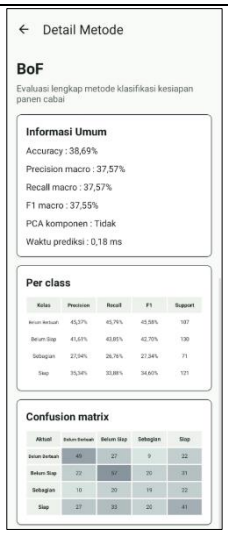
2. Pengujian Fungsionalitas Aplikasi Menu Peneliti

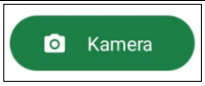
Pengujian fungsionalitas menu peneliti dilakukan untuk memastikan fitur yang digunakan oleh pengguna peneliti dapat berjalan sesuai rancangan. Menu peneliti berfungsi untuk menampilkan hasil evaluasi dan perbandingan metode yang digunakan dalam penelitian, yaitu *BoF*, *BoF + HSV*, dan *BoF + HSV + SPM*. Fitur yang diuji meliputi *login* peneliti, validasi *login*, *dashboard* perbandingan metode, grafik perbandingan, kartu evaluasi metode, detail metode, dan *logout*.

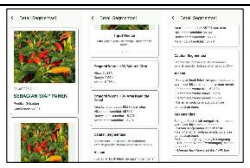
Tabel 4. 17 Pengujian Fungsionalitas Aplikasi Menu Peneliti

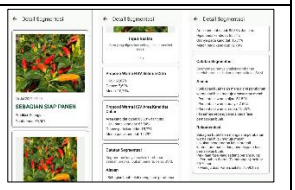
No.	Fitur yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian	Keterangan
1.	Pemilihan jenis pengguna	Pengguna memilih menu *Login* Sebagai Peneliti	Aplikasi berpindah ke halaman *onboarding* petani	Berhasil	
2.	*Login* peneliti valid	Pengguna memasukkan email dan password yang benar	Aplikasi masuk ke halaman utama peneliti	Berhasil	
3.	*Login* peneliti tidak valid	Pengguna memasukkan email atau password yang salah	Aplikasi menampilkan pesan kesalahan *login*	Berhasil	

4.	*Dashboard* peneliti	Pengguna berhasil *login* sebagai peneliti	Aplikasi menampilkan *dashboard* perbandingan metode	Berhasil	
5.	Metode terbaik	Pengguna membuka menu perbandingan metode	Aplikasi menampilkan metode terbaik berdasarkan nilai *accuracy* tertinggi	Berhasil	

6.	Grafik perbandingan metode	Pengguna membuka menu perbandingan metode	Aplikasi menampilkan grafik perbandingan *accuracy* dari setiap metode	Berhasil	
7.	Kartu evaluasi metode	Pengguna melihat daftar metode pada *dashboard*	Aplikasi menampilkan kartu evaluasi *BoF*, *BoF + HSV*, dan *BoF + HSV + SPM*	Berhasil	
8.	Detail metode *BoF*	Pengguna memilih detail metode *BoF*	Aplikasi menampilkan informasi detail evaluasi metode *BoF*	Berhasil	

9.	Detail metode *BoF + HSV*	Pengguna memilih detail metode *BoF + HSV*	Aplikasi menampilkan informasi detail evaluasi metode *BoF + HSV*	Berhasil	
10.	Detail metode *BoF + HSV + SPM*	Pengguna memilih detail metode *BoF + HSV + SPM*	Aplikasi menampilkan informasi detail evaluasi metode *BoF + HSV + SPM*	Berhasil	
11.	*Logout* peneliti	Pengguna menekan tombol *Logout*	Aplikasi keluar dari sesi peneliti dan kembali ke halaman pemilihan jenis pengguna	Berhasil	
12.	Kamera	Pengguna menekan	Aplikasi membuka kamera untuk	Berhasil	

		tombol Kamera	mengambil citra tanaman cabai rawit		
13.	Galeri	Pengguna menekan tombol Galeri	Aplikasi membuka galeri untuk memilih citra tanaman cabai rawit	Berhasil	
14.	Analisis Segmentasi	Peneliti memilih citra dan menekan tombol Analisis Segmentasi	Aplikasi mengirim citra ke *backend* melalui *endpoint* /predict-segmentation	Berhasil	
15.	Hasil Segmentasi	Proses analisis selesai	Aplikasi menampilkan hasil prediksi, *input resize*, *mask*, *overlay*, proporsi warna, alasan, dan rekomendasi	Berhasil	
16.	Riwayat Segmentasi	Peneliti membuka Riwayat Segmentasi	Aplikasi menampilkan daftar hasil segmentasi	Berhasil	

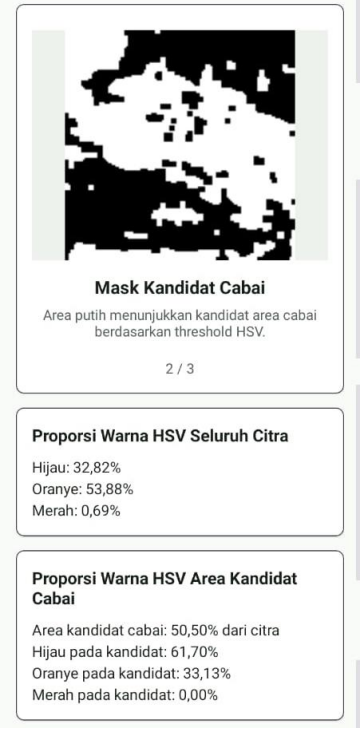
			yang tersimpan		
17.	Detail riwayat	Peneliti memilih salah satu riwayat	Aplikasi menampilkan detail hasil segmentasi yang dipilih	Berhasil	
18.	Hapus riwayat	Peneliti menekan tombol Hapus	Aplikasi menghapus data riwayat segmentasi yang dipilih	Berhasil	

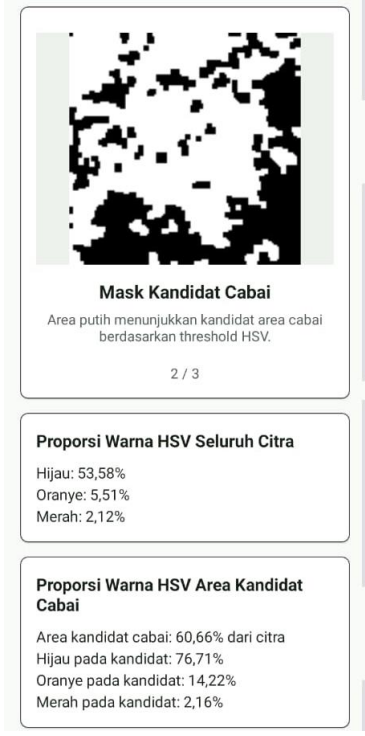
4.2.5 Pengujian Deteksi Menggunakan Citra Eksternal

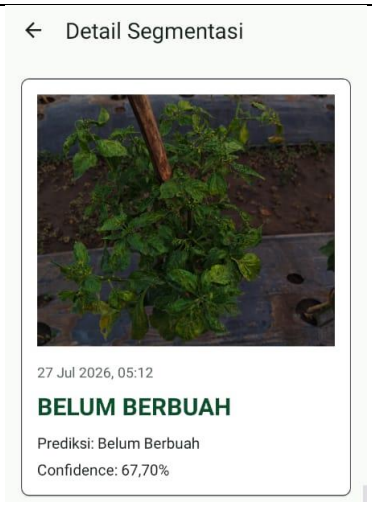
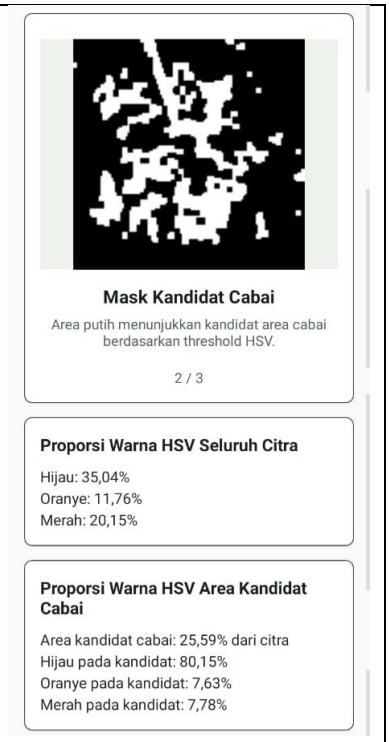
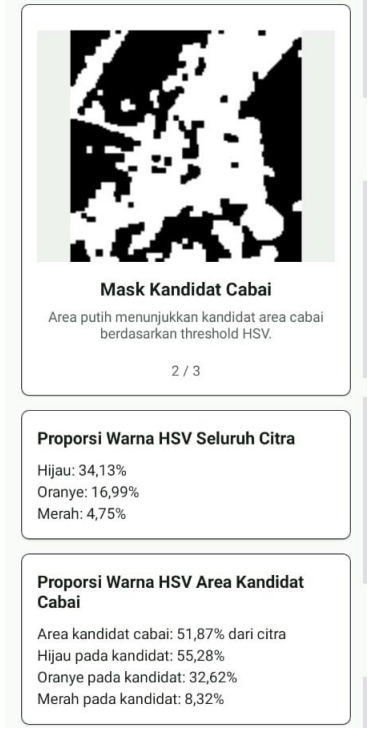
Pengujian deteksi menggunakan citra eksternal dilakukan untuk mengetahui kemampuan sistem Chilivest dalam memproses citra tanaman cabai rawit yang berasal dari luar data uji pada pengujian perbandingan metode. Citra yang digunakan pada pengujian ini merupakan Gambar langsung yang dimasukkan melalui aplikasi untuk melihat hasil prediksi model terhadap citra baru. Pengujian ini bertujuan untuk mengetahui apakah sistem dapat menerima citra, memproses citra melalui *backend*, dan menampilkan hasil prediksi berupa kelas kesiapan panen serta nilai *confidence*.

Pada pengujian ini, hasil prediksi dikatakan berhasil apabila kelas yang dihasilkan oleh sistem sesuai dengan kondisi visual citra yang diuji. Sebaliknya, hasil prediksi dikatakan tidak berhasil apabila kelas prediksi tidak sesuai dengan kondisi visual citra. Hasil pengujian ditampilkan pada Tabel 4.18 yang memuat Gambar uji, status keberhasilan, kelas hasil prediksi, dan nilai *confidence*.

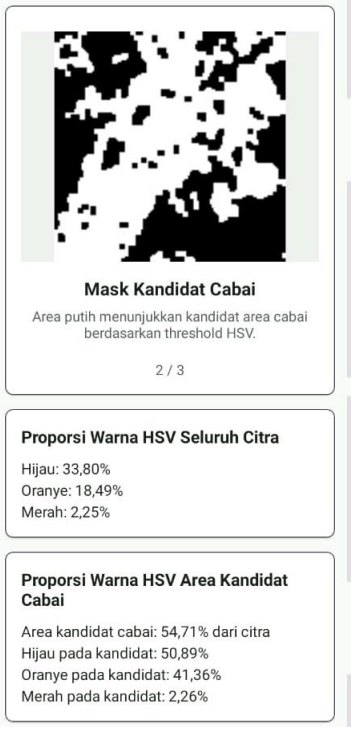
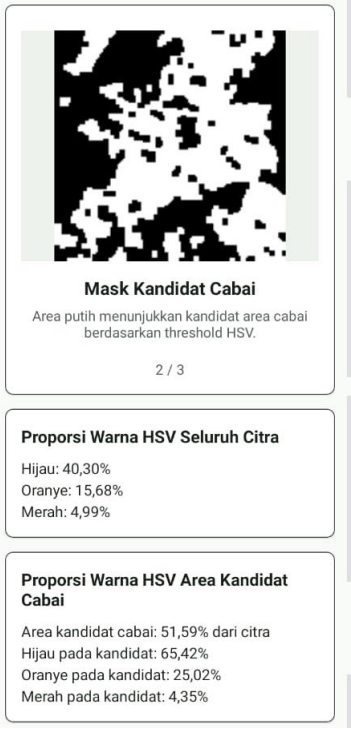
Tabel 4. 18 Hasil Pengujian Menggunakan Gambar Langsung

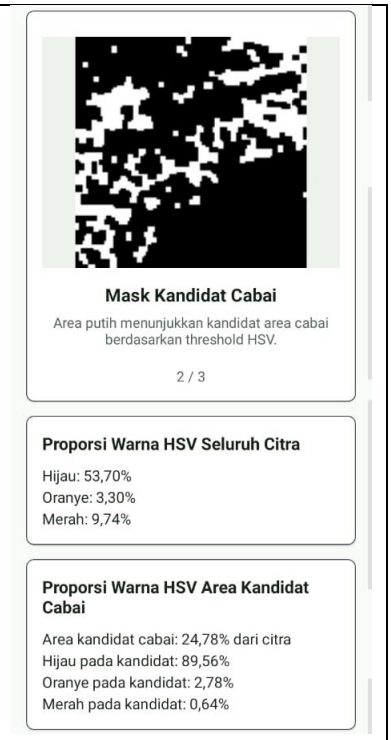
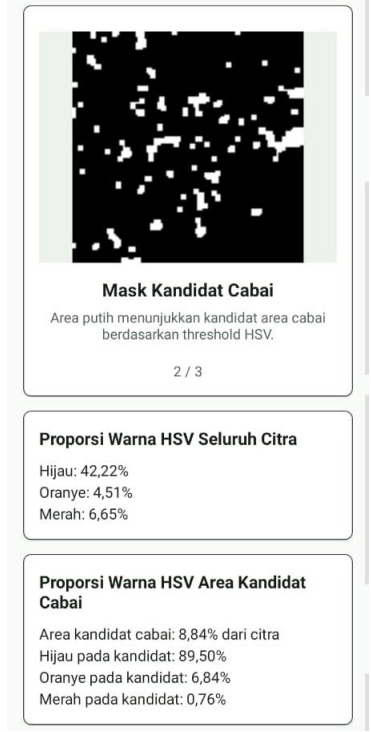
No	Gambar	Status	Hasil
1.		Berhasil (belumberbuah) Confidence 67,80%	
2.		Berhasil (belumberbuah) Confidence 71,30%	

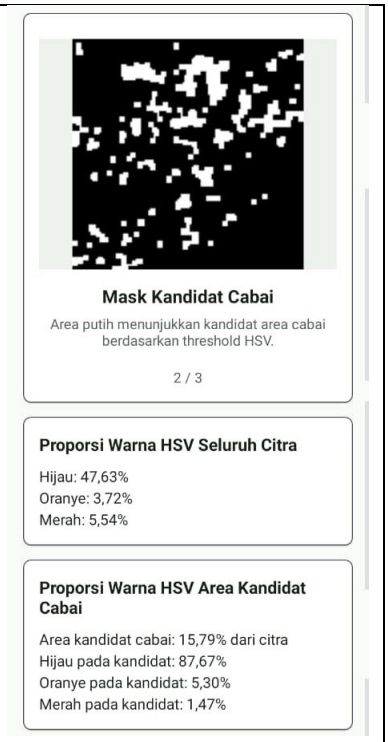
3.		Berhasil (belumberbuah) Confidence 34,10%	
4.		Tidak Berhasil (belumberbuah) Confidence 67.30%	

5.		Berhasil (belumberbuah) Confidence 67.70%	
6.		Berhasil (belumsiap) Confidence 69.80%	

7.		Berhasil (belumsiap) Confidence 70.00%	
8.		Berhasil (belumsiap) Confidence 67.10%	

9.		Berhasil (belumsiap) Confidence 68.10%	
10.		Berhasil (belumsiap) Confidence 68.80%	

11.		Berhasil (sebagian) Confidence 67.30%	
12.		Berhasil (sebagian) Confidence 67.60%	

13.		Berhasil (sebagian) Confidence 67.60%	

Berdasarkan Tabel 4.18, sistem Chilivest mampu memproses citra eksternal dan menampilkan hasil prediksi pada setiap Gambar uji. Hasil prediksi yang ditampilkan berupa kelas kesiapan panen dan nilai *confidence*.

Pada pengujian ini, terdapat beberapa hasil prediksi yang sesuai dengan kondisi visual citra dan terdapat pula hasil prediksi yang berbeda. Hasil tersebut menunjukkan bahwa citra eksternal dapat digunakan sebagai data pendukung untuk melihat respons sistem terhadap Gambar baru di luar *dataset* pengujian perbandingan metode. Secara keseluruhan, pengujian deteksi menggunakan citra eksternal menunjukkan bahwa aplikasi Chilivest dapat menerima Gambar, memproses Gambar melalui *backend*, dan menampilkan hasil prediksi pada aplikasi Android.

4.2.6 Pengujian Komunikasi *Client-Server*

Pengujian implementasi jaringan *client-server* dilakukan untuk mengetahui apakah aplikasi *Android Chilivest* dapat berkomunikasi dengan *backend Flask* yang telah *dideploy* secara *online*. Pengujian ini meliputi komunikasi data antara aplikasi *Android* sebagai *client* dengan *server Flask*, pengujian kualitas koneksi melalui *packet*

loss dan *delay*, serta pengujian *response time* saat sistem diakses oleh dua pengguna. Pengujian dinyatakan berhasil apabila *request* dari *client* dapat diterima oleh *server*, *server* dapat memproses data, dan *response* dapat dikirim kembali ke aplikasi tanpa terjadi kegagalan komunikasi.

1. Pengujian Implementasi Komunikasi *Client-Server*

Pengujian komunikasi *client-server* dilakukan dengan menguji alur pertukaran data antara aplikasi *Android Chilivest* dan *backend Flask*. Pengujian ini mencakup akses status *server*, pengiriman citra tanaman cabai rawit untuk prediksi, penerimaan *response JSON* hasil prediksi, penyimpanan riwayat prediksi pada aplikasi, serta pengambilan data perbandingan metode pada menu peneliti.

Tabel 4. 19 Pengujian Implementasi Komunikasi *Client-Server*

No	Pengujian Komunikasi Data	Alur Data	Hasil yang Diharapkan	Status
1.	Akses aplikasi	*Client* → Aplikasi *Chilivest*	Aplikasi dapat dibuka dan menampilkan halaman awal	Berhasil
2.	Masuk sebagai petani	*Client* → Aplikasi *Chilivest*	Aplikasi menampilkan menu utama petani	Berhasil
3.	Ambil atau pilih Gambar prediksi	*Client* → Kamera/Galeri → Aplikasi *Chilivest*	Aplikasi berhasil mengambil atau memilih citra tanaman cabai rawit	Berhasil
4.	Pengiriman citra prediksi	*Client* → Internet → *Server Flask*	Citra tanaman cabai rawit berhasil dikirim ke *endpoint* /predict	Berhasil
5.	Penerimaan hasil prediksi	*Server Flask* → Internet → *Client*	*Client* menerima *response JSON* berisi hasil prediksi, *confidence*, proporsi	Berhasil

			warna, alasan, dan rekomendasi	
6.	Penyimpanan riwayat prediksi	*Client* → Penyimpanan Lokal	Hasil prediksi berhasil disimpan ke riwayat prediksi aplikasi	Berhasil
7.	Masuk sebagai peneliti	*Client* → Aplikasi *Chilivest*	Aplikasi menampilkan halaman *login* dan menu utama peneliti setelah *login* berhasil	Berhasil
8.	Pengambilan data perbandingan metode	*Client* → Internet → *Server Flask*	*Client* berhasil mengambil data evaluasi metode dari *endpoint* /method-comparison	Berhasil
9.	Menampilkan detail metode	*Server Flask* → Internet → *Client*	Aplikasi menampilkan detail metode berupa *accuracy*, *precision*, *recall*, *F1-score*, per-class, dan *confusion matrix*	Berhasil

Berdasarkan Tabel 4.19, pengujian implementasi komunikasi *client-server* dilakukan untuk memastikan bahwa setiap alur pertukaran data pada aplikasi *Chilivest* dapat berjalan sesuai dengan rancangan. Pengujian diawali dari proses akses aplikasi, yaitu ketika pengguna membuka aplikasi *Chilivest* melalui perangkat *Android*. Pada tahap ini, aplikasi berhasil menampilkan halaman awal dan dapat digunakan untuk memilih jenis pengguna, baik sebagai petani maupun peneliti.

Pada alur pengguna petani, komunikasi dimulai ketika pengguna masuk ke menu petani dan memilih citra tanaman cabai rawit melalui kamera atau galeri. Setelah citra dipilih, aplikasi bertindak sebagai *client* yang mengirimkan data citra ke *backend Flask* melalui jaringan internet menuju *endpoint* /predict. *Server Flask* kemudian menerima citra tersebut dan menjalankan proses pengolahan citra, ekstraksi fitur,

reduksi dimensi, serta klasifikasi menggunakan model yang telah disiapkan. Setelah proses prediksi selesai, *server* mengirimkan kembali *response* dalam format *JSON* ke aplikasi *Android*. *Response* tersebut berisi hasil prediksi, nilai *confidence*, persentase warna, status tanaman, alasan hasil prediksi, dan rekomendasi. Hasil pengujian menunjukkan bahwa data hasil prediksi dapat diterima oleh aplikasi dan ditampilkan pada halaman hasil prediksi.

Setelah hasil prediksi diterima, aplikasi menyimpan data tersebut ke penyimpanan lokal sebagai riwayat prediksi. Penyimpanan riwayat ini tidak dilakukan ke *server*, melainkan pada sisi *client* agar pengguna dapat melihat kembali hasil prediksi sebelumnya tanpa harus melakukan prediksi ulang. Dengan demikian, alur komunikasi pada menu petani terdiri dari proses pengambilan citra, pengiriman citra ke *server*, penerimaan hasil prediksi dari *server*, dan penyimpanan hasil prediksi pada aplikasi.

Pada alur pengguna peneliti, pengujian dilakukan dengan masuk ke menu peneliti untuk menampilkan data evaluasi metode. Aplikasi mengirimkan *request* ke *server Flask* melalui *endpoint* /method-comparison. *Server* kemudian mengirimkan *response JSON* berisi data perbandingan metode, seperti nilai *accuracy*, *precision macro*, *recall macro*, *F1-score macro*, waktu *training*, waktu prediksi, hasil per kelas, dan *confusion matrix*. Data tersebut kemudian ditampilkan pada *dashboard* peneliti dan halaman detail metode. Hasil pengujian menunjukkan bahwa aplikasi dapat mengambil data evaluasi metode dari *server* dan menampilkannya sesuai dengan rancangan.

Secara keseluruhan, hasil pengujian menunjukkan bahwa implementasi komunikasi antara aplikasi *Android Chilivest* sebagai *client* dan *backend Flask* sebagai *server* telah berjalan dengan baik. Setiap *request* yang dikirimkan oleh *client* dapat diterima dan diproses oleh *server*, kemudian *server* mengirimkan *response* kembali ke aplikasi dalam format *JSON*. Dengan demikian, komunikasi *client-server* pada aplikasi *Chilivest* dinyatakan berhasil karena seluruh alur data, baik pada menu petani maupun menu peneliti, dapat berjalan sesuai dengan hasil yang diharapkan.

2. Hasil Pengujian *QoS* Koneksi *Client* ke *Server*

Pengujian *QoS* koneksi *client* ke *server* dilakukan untuk mengetahui kualitas koneksi jaringan antara perangkat *client* dan *server backend*. Parameter yang diamati meliputi jumlah paket yang dikirim, jumlah paket yang diterima, paket yang hilang, *delay* minimum, *delay* maksimum, dan rata-rata *delay*. Pengujian *packet loss* dilakukan untuk mengetahui apakah terdapat paket data yang gagal diterima oleh *server* selama proses komunikasi berlangsung.

Tabel 4. 20 Hasil Pengujian *QoS Koneksi Client ke Server*

Uji ke-	Jumlah Paket Dikirim	Jumlah Paket Diterima	Paket yang Hilang	*Delay* Minimum	*Delay* Maksimum	Rata-rata *Delay*	Status
1.	20	20	0	28 ms	190 ms	52 ms	Berhasil
2.	20	20	0	28 ms	62 ms	36 ms	Berhasil
3.	20	20	0	28 ms	83 ms	39 ms	Berhasil
4.	20	20	0	28 ms	85 ms	36 ms	Berhasil
5.	20	20	0	28 ms	151 ms	43 ms	Berhasil
6.	20	20	0	28 ms	82 ms	39 ms	Berhasil
7.	20	20	0	28 ms	53 ms	32 ms	Berhasil
8.	20	20	0	28 ms	85 ms	38 ms	Berhasil
9.	20	20	0	28 ms	91 ms	42 ms	Berhasil
10	20	20	0	28 ms	81 ms	36 ms	Berhasil

Berdasarkan Tabel 4.20, seluruh pengujian menunjukkan bahwa jumlah paket yang dikirim sama dengan jumlah paket yang diterima, yaitu sebanyak 20 paket pada setiap uji. Total paket yang dikirim pada 10 kali pengujian adalah 200 paket, dan seluruh paket berhasil diterima kembali oleh *client*. Dengan demikian, tidak terdapat paket yang hilang selama proses pengujian.

Perhitungan *packet loss* dilakukan menggunakan Persamaan.

$$Packet\ Loss = \frac{Jumlah\ Paket\ Dikirim - Jumlah\ Paket\ Diterima}{Jumlah\ Paket\ Dikirim} \times 100\%$$

Berdasarkan hasil pengujian, total paket yang dikirim adalah 200 paket, sedangkan total paket yang diterima adalah 200 paket. Maka *packet loss* dapat dihitung sebagai berikut.

$$Packet\ Loss = \frac{200 - 200}{200} \times 100\%$$

$$Packet\ Loss = \frac{0}{200} \times 100\%$$

$$Packet\ Loss = 0\%$$

3. Hasil Pengujian *Delay*

Tabel 4. 21 Hasil Pengujian *Delay*

Skenario Pengujian	Uji ke-	*Response Time* User 1	*Response Time* User 2	Rata-rata *Response Time*	Hasil Pengujian	Status
Akses aplikasi *Chilivest*	1	224 ms	236 ms	230 ms	Kedua user berhasil membuka aplikasi dan menampilkan halaman awal	Sangat Baik
	2	218 ms	229 ms	224 ms		
	3	231 ms	240 ms	236 ms		
Masuk ke menu petani	1	28 ms	31 ms	30 ms	Kedua user berhasil menampilkan menu utama petani	Sangat Baik
	2	30 ms	33 ms	32 ms		
	3	27 ms	29 ms	28 ms		
Unggah Gambar dan proses prediksi	1	293 ms	305 ms	299 ms	Kedua user berhasil mengunggah citra dan menampilkan hasil prediksi	Sangat Baik
	2	287 ms	301 ms	294 ms		
	3	298 ms	312 ms	305 ms		

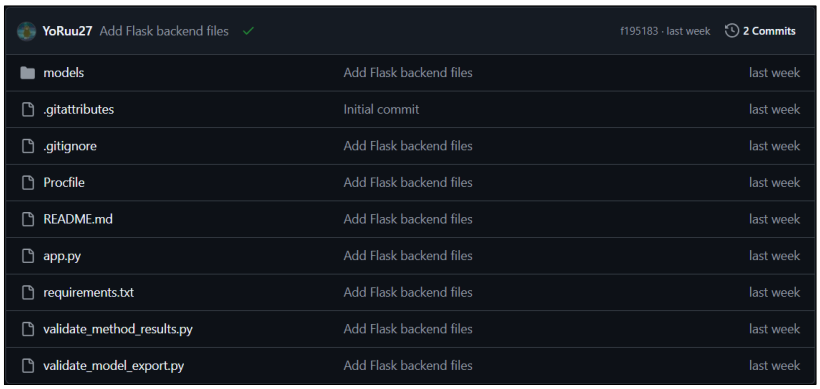
Hapus riwayat prediksi	1	58 ms	61 ms	60 ms	Kedua user berhasil menghapus data riwayat prediksi	Sangat Baik
	2	55 ms	59 ms	57 ms		
	3	60 ms	64 ms	62 ms		
Proses *logout* menu petani	1	38 ms	41 ms	40 ms	Kedua user berhasil keluar dari menu petani	Sangat Baik
	2	36 ms	39 ms	38 ms		
	3	40 ms	43 ms	42 ms		
Masuk ke menu peneliti	1	28 ms	30 ms	29 ms	Kedua user berhasil menampilkan menu utama peneliti	Sangat Baik
	2	29 ms	32 ms	31 ms		
	3	27 ms	31 ms	29 ms		
Akses detail metode	1	32 ms	35 ms	34 ms	Kedua user berhasil menampilkan detail evaluasi Metode	Sangat Baik
	2	34 ms	37 ms	36 ms		
	3	31 ms	34 ms	33 ms		
Unggah Gambar dan proses segmentasi	1	88 ms	92 ms	90 ms	Kedua *user* berhasil mengunggah citra dan menampilkan hasil segmentasi	Sangat Baik
	2	91 ms	89 ms	90 ms		
	3	87 ms	93 ms	90 ms		
Hapus riwayat segmentasi	1	61 ms	65 ms	63 ms	Kedua *user* berhasil menghapus data riwayat segmentasi	Sangat Baik
	2	59 ms	63 ms	61 ms		
	3	64 ms	67 ms	66 ms		
Proses *logout*	1	38 ms	42 ms	39 ms	Kedua user berhasil keluar dari menu peneliti	Sangat Baik
	2	37 ms	40 ms	40 ms		
	3	39 ms	41 ms	40 ms		

menu peneliti						

Berdasarkan Tabel 4.21, pengujian *delay* dilakukan sebanyak tiga kali pada setiap skenario untuk mengetahui *response time* aplikasi *Chilivest* ketika digunakan oleh dua user. Hasil pengujian menunjukkan bahwa seluruh skenario berhasil dijalankan dengan status sangat baik. *Response time* tertinggi terdapat pada skenario unggah Gambar dan proses prediksi, karena pada proses tersebut aplikasi melakukan pengambilan citra, pengiriman data ke *server*, penerimaan *response*, dan penampilan hasil prediksi pada aplikasi.

4.2.7 Pengujian *Deployment Backend*

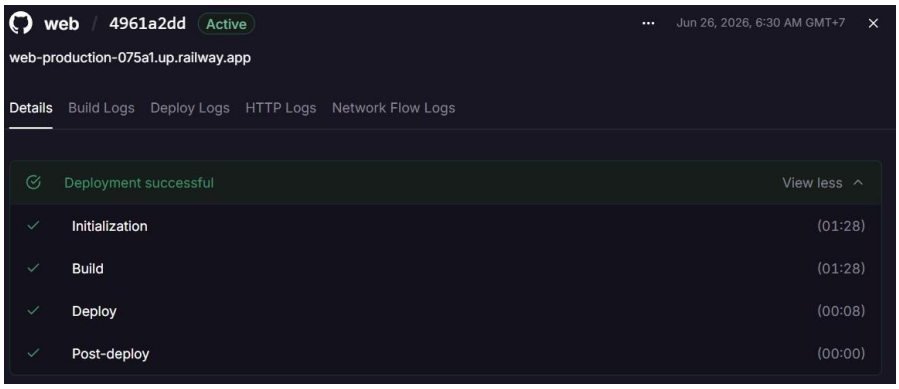
Pengujian *deployment backend* dilakukan untuk memastikan bahwa *backend Flask* pada sistem *Chilivest* berhasil dijalankan secara *online* menggunakan *Railway*. *Deployment backend* bertujuan agar aplikasi *Android Chilivest* dapat mengakses *server* melalui jaringan internet tanpa harus menjalankan *backend* secara lokal. Pengujian ini dilakukan dengan memeriksa *repository backend* pada *GitHub*, status *deployment* pada *Railway*, serta konfigurasi *deployment* dan URL publik *backend* yang digunakan oleh aplikasi.



Gambar 4. 30 *Repository Backend Chilivest di GitHub*

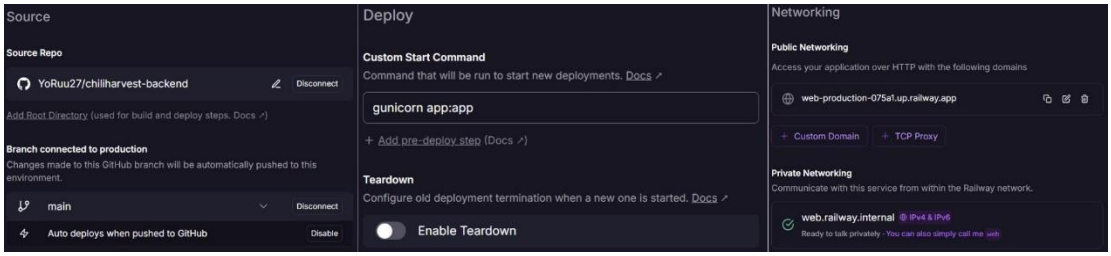
Gambar 4.30 menunjukkan *repository backend Chilivest* yang tersimpan pada *GitHub*. *Repository* tersebut berisi beberapa file utama yang digunakan untuk menjalankan *backend*, seperti folder models, file *app.py*, *requirements.txt*, *Procfile*,

serta file pendukung lain seperti validate_model_result.py dan validate_model_export.py. File *app.py* digunakan sebagai program utama *backend Flask*, sedangkan *requirements.txt* berisi daftar *library* yang dibutuhkan agar *backend* dapat berjalan pada lingkungan *deployment*. *Repository GitHub* ini digunakan sebagai sumber kode yang dihubungkan dengan *Railway*, sehingga proses *deployment* dapat dilakukan secara otomatis dari *repository* tersebut.



Gambar 4. 31 *Status Backend Chilivest pada Railway*

Gambar 4.31 menunjukkan status *backend Chilivest* pada *Railway*. Berdasarkan Gambar tersebut, *service backend* berada dalam kondisi Active dan proses *deployment* dinyatakan *Deployment* successful. Proses *deployment* telah melalui beberapa tahapan, yaitu *initialization*, *build*, *deploy*, dan *post-deploy*. Keberhasilan pada setiap tahapan tersebut menunjukkan bahwa *Railway* berhasil membaca *source code* dari *GitHub*, melakukan proses *build*, menjalankan *service backend*, dan membuat *backend* siap diakses secara *online*.



Gambar 4. 32 Konfigurasi *dan URL Backend Chilivest pada Railway*

Gambar 4.32 menunjukkan konfigurasi *deployment backend Chilivest* pada *Railway*. Pada bagian *Source*, *Railway* terhubung dengan *repository GitHub*

YoRuu27/*chiliharvest-backend* dan *branch* main, sehingga perubahan kode pada *repository* dapat digunakan dalam proses *deployment*. Pada bagian *Deploy*, *backend* dijalankan menggunakan custom start command *gunicorn* app:app. *Gunicorn* digunakan sebagai *WSGI server* untuk menjalankan aplikasi *Flask* pada lingkungan *production*. Selain itu, bagian Networking menunjukkan URL publik *backend*, yaitu *web-production-075a1.up.railway.app*, yang digunakan sebagai alamat *server* untuk diakses oleh aplikasi *Android Chilivest*.

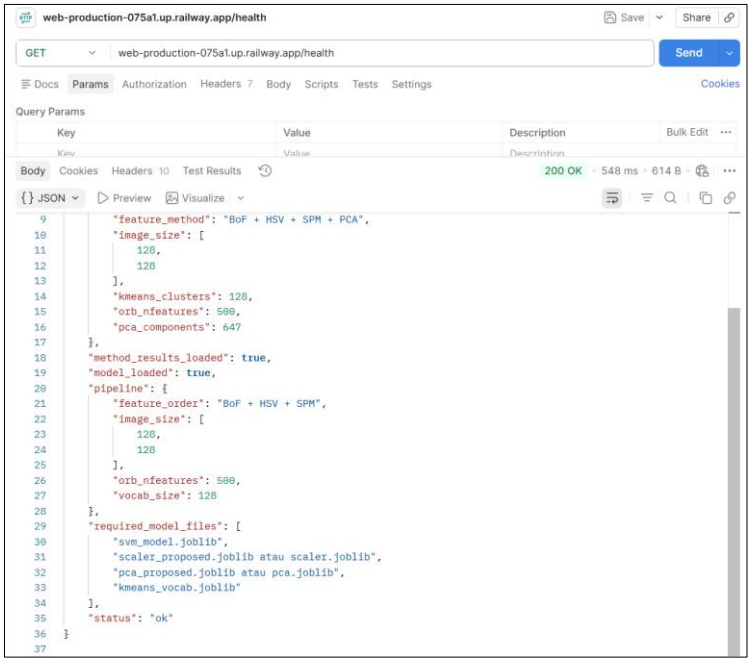
Tabel 4. 22 Hasil Pengujian *Deployment Backend*

No	Komponen yang Diuji	Skenario Pengujian	Hasil yang Diharapkan	Hasil Pengujian	Status
1.	*Repository GitHub*	Memeriksa file *backend* pada *repository GitHub*	File *backend*, model, *dependency*, dan konfigurasi *deployment* tersedia	Sesuai	Berhasil
2.	Status *deployment Railway*	Memeriksa status *service backend* pada *Railway*	*Service backend* aktif dan *deployment* berhasil	Sesuai	Berhasil
3.	Proses *deployment*	Memeriksa tahapan *initialization*, *build*, *deploy*, dan *post-deploy*	Seluruh tahapan *deployment* berjalan tanpa error	Sesuai	Berhasil
4.	*Source repository*	Memeriksa *repository GitHub* yang terhubung dengan *Railway*	*Railway* terhubung dengan *repository backend Chilivest*	Sesuai	Berhasil

5.	Start command	Memeriksa perintah untuk menjalankan *backend*	*Backend* dijalankan menggunakan *gunicorn* app:app	Sesuai	Sesuai
6.	Public networking	Memeriksa URL publik *backend* pada *Railway*	*Backend* memiliki URL publik yang dapat diakses melalui internet	Sesuai	Sesuai

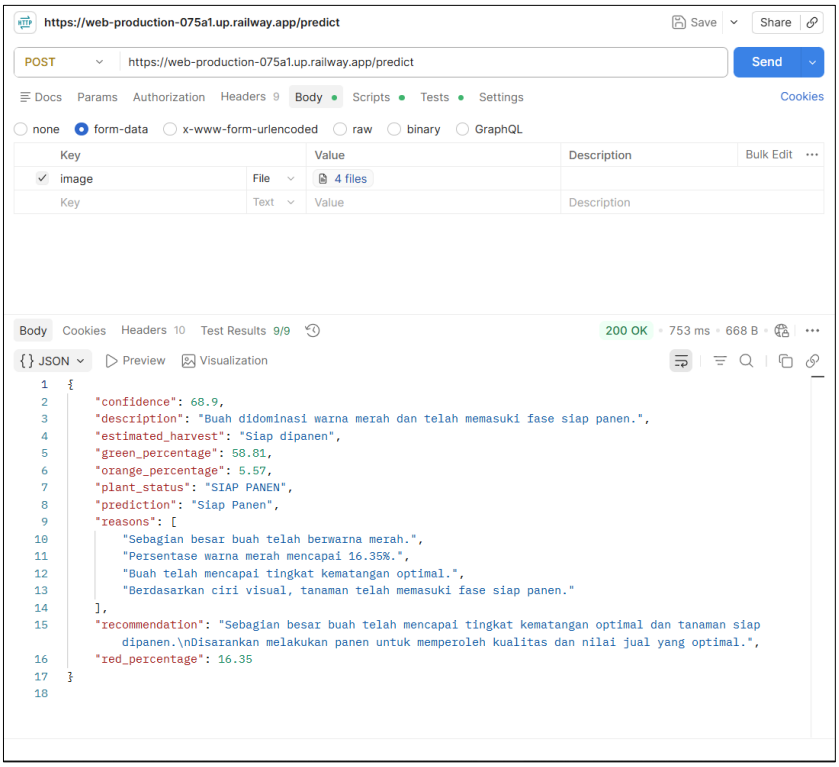
4.2.8 Pengujian *REST API*

Pengujian *REST API* dilakukan untuk memastikan *backend Flask* pada sistem *Chilivest* dapat diakses dan memberikan *response* sesuai dengan *endpoint* yang telah dibuat. Pengujian dilakukan menggunakan Postman dengan mengakses *endpoint* /health, /predict, dan /method-comparison. Hasil pengujian dinyatakan berhasil apabila *server* memberikan *status response 200 OK* dan mengembalikan data dalam format *JSON*.



Gambar 4. 33 Pengujian *Endpoint* /health

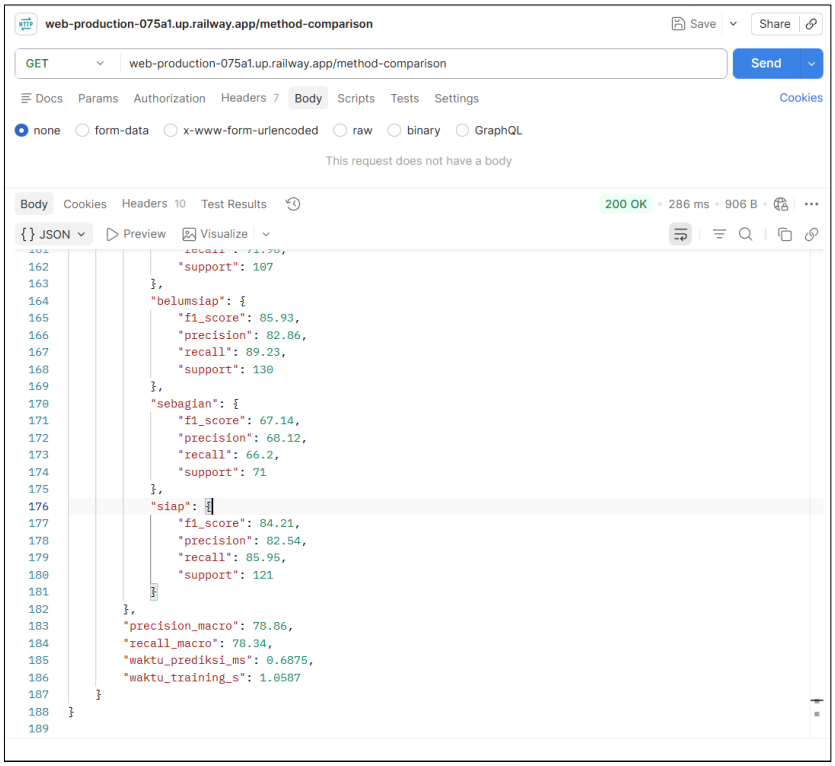
Gambar 4.33 menunjukkan hasil pengujian *endpoint* /health menggunakan *method GET* pada Postman. *Endpoint* ini digunakan untuk memeriksa apakah *backend Flask* dalam kondisi aktif dan model yang dibutuhkan sudah berhasil dimuat. Berdasarkan hasil pengujian, *server* memberikan *status response* 200 OK. *Response JSON* yang ditampilkan menunjukkan bahwa status *server* dalam kondisi ok, model berhasil dimuat, data hasil metode tersedia, serta konfigurasi *pipeline* seperti ukuran citra, jumlah fitur *ORB*, jumlah cluster *K-Means*, dan jumlah komponen *PCA* dapat terbaca dengan baik.



Gambar 4. 34 Pengujian *Endpoint* /predict

Gambar 4.34 menunjukkan hasil pengujian *endpoint* /predict menggunakan *method POST* pada Postman. Pengujian dilakukan dengan mengirim file citra tanaman cabai rawit melalui menu Body dengan tipe *form-data* dan key *image*. Berdasarkan hasil pengujian, *endpoint* /predict berhasil memberikan *status response* 200 OK dan mengembalikan *response JSON* berupa hasil prediksi.

Response JSON yang diperoleh memuat beberapa informasi, yaitu nilai *confidence* sebesar 68,9, hasil prediksi Siap Panen, status tanaman SIAP PANEN, persentase warna hijau sebesar 5,81%, oranye sebesar 5,57%, dan merah sebesar 16,35%. Selain itu, *response* juga menampilkan alasan hasil prediksi serta rekomendasi bahwa sebagian besar buah telah mencapai tingkat kematangan optimal dan tanaman siap dipanen.

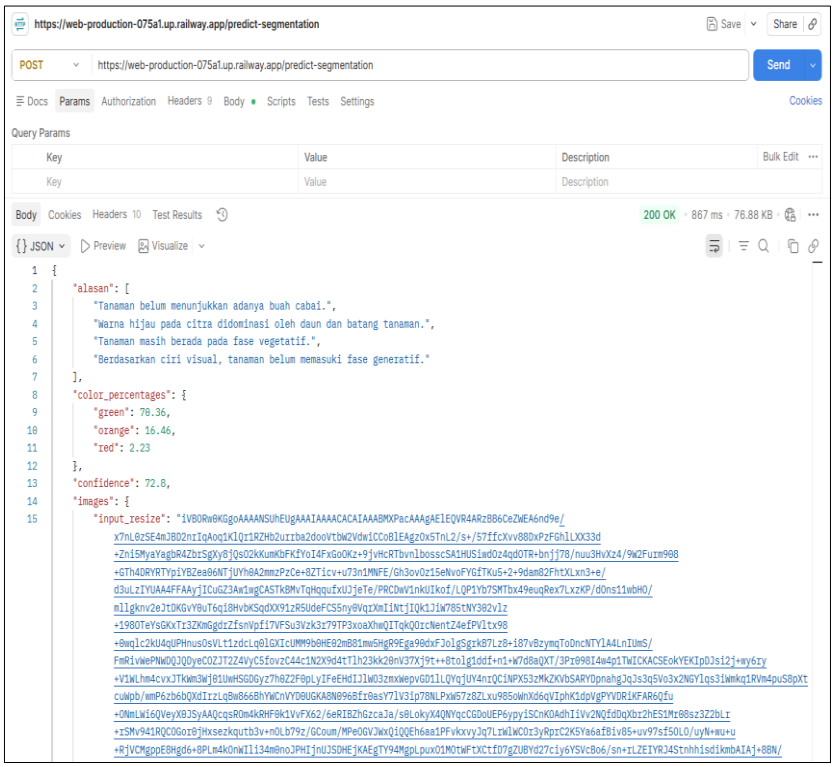


Gambar 4. 35 Pengujian *Endpoint* /method-comparison

Gambar 4.35 menunjukkan hasil pengujian *endpoint* /method-comparison menggunakan *method GET* pada Postman. *Endpoint* ini digunakan untuk mengambil data hasil perbandingan metode yang ditampilkan pada menu peneliti di aplikasi *Chilivest*. Berdasarkan hasil pengujian, *server* memberikan *status response* 200 OK dan mengembalikan *response JSON* berisi data evaluasi metode.

Response JSON yang dikembalikan oleh *endpoint* /method-comparison memuat informasi evaluasi dari setiap metode, seperti nilai *accuracy*, *precision macro*, *recall macro*, *F1-score*, waktu *training*, waktu prediksi, dimensi fitur, serta hasil

evaluasi *per class*. Data tersebut kemudian digunakan oleh aplikasi Chilivest pada menu Peneliti untuk menampilkan ringkasan perbandingan metode BoF, BoF + HSV, dan BoF + HSV + SPM secara langsung melalui aplikasi.



Gambar 4. 36 Pengujian *Endpoint* /predict-segmentation

Gambar 4.36 menunjukkan hasil pengujian *endpoint* /predict-segmentation menggunakan *method* POST pada Postman. Pengujian dilakukan dengan mengirim citra tanaman cabai rawit melalui *request body* dalam format *form-data*. *Endpoint* ini digunakan untuk mendukung fitur Hasil Segmentasi pada akun Peneliti, yaitu menampilkan hasil prediksi, proporsi warna HSV seluruh citra, proporsi warna HSV area kandidat cabai, serta Gambar hasil segmentasi berupa citra *input resize*, *mask* kandidat cabai, dan *overlay* segmentasi.

Berdasarkan hasil pengujian, *endpoint* /predict-segmentation berhasil memberikan *status response* 200 OK dengan *response time* sebesar 867 ms. *Response* yang diterima berbentuk JSON dan memuat data prediksi, nilai *confidence*, alasan, rekomendasi, proporsi warna, serta Gambar hasil segmentasi dalam bentuk *base64*. Dengan demikian, pengujian *endpoint* /predict-segmentation dinyatakan berhasil

karena *backend* dapat menerima *request* citra, memproses prediksi dan segmentasi, serta mengirimkan *response* yang dapat digunakan oleh aplikasi Android Chilivest.

Pengujian *endpoint* REST API dilakukan untuk memastikan bahwa *backend* Flask Chilivest dapat menerima *request* dari aplikasi Android dan mengirimkan *response* sesuai kebutuhan sistem. Pengujian dilakukan menggunakan Postman terhadap beberapa *endpoint*, yaitu /health, /predict, /method-comparison, dan /predict-segmentation. Setiap *endpoint* diuji berdasarkan skenario pengujian dan hasil yang diharapkan.

Tabel 4. 23 Hasil Pengujian *Endpoint REST API*

No	*Endpoint*	*Method*	Skenario Pengujian	Hasil yang Diharapkan	*Status Response*	Status
1.	/health	*GET*	Mengakses *endpoint* untuk memeriksa kondisi *backend Flask*	*Server* menampilkan status *backend*, model, dan *pipeline* dalam format *JSON*	200 OK	Berhasil
2.	/predict	*POST*	Mengirim citra tanaman cabai rawit melalui *form-data* dengan key *image*	*Server* mengembalikan hasil prediksi, *confidence*, proporsi warna, alasan, dan rekomendasi	200 OK	Berhasil

3.	/method-comparison	*GET*	Mengakses *endpoint* data perbandingan metode	*Server* mengembalikan data evaluasi metode *BoF*, *BoF + HSV*, dan *BoF + HSV + SPM* dalam format *JSON*	200 OK	Berhasil
4.	/predict-segmentation	*POST*	Mengirim citra tanaman cabai rawit melalui *form-data* dengan *key image*	*Server* mengembalikan hasil prediksi, proporsi warna, dan Gambar segmentasi dalam format JSON	200 OK	Berhasil

Berdasarkan Tabel 4.23, seluruh *endpoint* REST API pada *backend* Flask Chilivest berhasil diuji dan memperoleh *status response* 200 OK. *Endpoint* /health berhasil digunakan untuk memeriksa status *backend*, model, dan konfigurasi *pipeline*. *Endpoint* /predict berhasil menerima citra melalui *form-data* dengan *key* image, memproses citra pada *backend*, dan mengembalikan hasil prediksi dalam format JSON. Sementara itu, *endpoint* /method-comparison berhasil mengirimkan data evaluasi metode BoF, BoF + HSV, dan BoF + HSV + SPM yang digunakan pada menu Peneliti.

Selain itu, *endpoint* /predict-segmentation juga berhasil diuji untuk mendukung fitur Hasil Segmentasi pada menu Peneliti. *Endpoint* ini menerima citra tanaman cabai rawit, kemudian mengembalikan hasil prediksi dan informasi segmentasi berupa

proporsi warna HSV seluruh citra, proporsi warna HSV area kandidat cabai, citra *input resize*, *mask* kandidat cabai, dan *overlay* segmentasi. Dengan demikian, pengujian REST API menunjukkan bahwa komunikasi *request* dan *response* antara *client* dan *server* telah berjalan sesuai rancangan.

4.3 Pembahasan

Pembahasan dilakukan untuk menjelaskan hasil implementasi dan pengujian sistem deteksi kesiapan panen tanaman cabai rawit yang telah diperoleh pada subbab hasil penelitian. Sistem yang dikembangkan terdiri dari aplikasi *Android Chilivest* sebagai antarmuka pengguna, *backend Flask* sebagai *server* pemrosesan, serta model klasifikasi berbasis kombinasi fitur *BoF + HSV + SPM* yang direduksi menggunakan *PCA* dan diklasifikasikan menggunakan *SVM*. Pembahasan ini mencakup hasil implementasi aplikasi, pengolahan citra dan ekstraksi fitur, perbandingan metode, pengujian fungsionalitas dan respons aplikasi, komunikasi *client-server*, *deployment backend*, serta pengujian *endpoint REST API*.

4.3.1 Pembahasan Hasil Implementasi Aplikasi *Android*

Berdasarkan hasil implementasi pada subbab 4.2.1, aplikasi Chilivest telah berhasil dikembangkan sebagai aplikasi *Android* untuk mendukung sistem deteksi kesiapan panen tanaman cabai rawit. Aplikasi ini memiliki dua alur utama, yaitu alur pengguna Petani dan alur pengguna Peneliti. Alur Petani digunakan untuk melakukan prediksi kesiapan panen melalui citra tanaman cabai rawit, sedangkan alur Peneliti digunakan untuk menampilkan hasil evaluasi metode serta analisis tambahan berupa hasil segmentasi.

Pada alur Petani, aplikasi menyediakan fitur pengambilan citra melalui kamera, pemilihan citra melalui galeri, proses analisis, tampilan hasil prediksi, riwayat prediksi, dan menu akun. Hasil prediksi ditampilkan dalam bentuk kelas kesiapan panen, nilai *confidence*, proporsi warna hijau, oranye, dan merah, alasan prediksi, serta rekomendasi. Hal ini menunjukkan bahwa aplikasi tidak hanya berfungsi sebagai media klasifikasi, tetapi juga memberikan informasi pendukung yang mudah dipahami oleh pengguna.

Pada alur Peneliti, aplikasi menyediakan halaman *login*, menu Perbandingan Metode, Detail Metode, Hasil Segmentasi, dan Riwayat Segmentasi. Menu Perbandingan Metode menampilkan informasi evaluasi seperti *accuracy*, *precision macro*, *recall macro*, *F1-score macro*, waktu *training*, waktu prediksi, hasil evaluasi *per class*, dan *confusion matrix*. Sementara itu, menu Hasil Segmentasi menampilkan citra *input resize*, *mask* kandidat cabai, *overlay* segmentasi, hasil prediksi, proporsi warna HSV, alasan, dan rekomendasi. Hasil segmentasi hanya digunakan sebagai informasi pendukung analisis visual dan tidak menjadi penentu kelas prediksi SVM.

Secara keseluruhan, hasil implementasi aplikasi Chilivest telah sesuai dengan rancangan sistem. Aplikasi mampu menjalankan fungsi utama pada menu Petani dan menu Peneliti, serta dapat digunakan sebagai antarmuka antara pengguna dan *backend* Flask. Dengan adanya penambahan fitur Hasil Segmentasi, aplikasi Chilivest menjadi lebih informatif bagi Peneliti karena dapat menampilkan hasil evaluasi metode sekaligus visualisasi segmentasi citra tanaman cabai rawit.

4.3.2 Pembahasan Hasil Pengolahan Citra dan Ekstraksi Fitur

Berdasarkan hasil pengolahan citra pada subbab 4.2.2, proses awal dilakukan dengan mengubah ukuran citra tanaman cabai rawit menjadi 128 × 128 piksel. Tahap *resize* ini dilakukan agar seluruh citra memiliki ukuran yang seragam sebelum masuk ke proses ekstraksi fitur. Dengan ukuran citra yang sama, proses pembentukan fitur BoF, HSV, SPM, PCA, dan klasifikasi SVM dapat dilakukan secara konsisten pada seluruh data citra.

Hasil visualisasi citra menunjukkan bahwa citra tanaman cabai rawit dapat diproses dari citra *input* menjadi citra hasil *resize* dan citra dalam ruang warna HSV. Konversi ke HSV digunakan karena warna buah cabai rawit menjadi salah satu informasi visual yang berkaitan dengan tingkat kesiapan panen. Berdasarkan hasil perhitungan pada citra contoh, diperoleh persentase warna hijau sebesar 76,68%, warna oranye sebesar 1,46%, dan warna merah sebesar 8,01%. Nilai tersebut menunjukkan bahwa citra contoh masih memiliki dominasi warna hijau dari daun dan bagian tanaman, tetapi terdapat bagian buah cabai berwarna merah yang menjadi indikator

visual kelas Siap Panen. Informasi proporsi warna ini digunakan sebagai informasi pendukung untuk membantu interpretasi visual pada aplikasi, bukan sebagai satu-satunya dasar keputusan deteksi.

Pada proses ekstraksi fitur warna, histogram HSV global dibentuk menggunakan konfigurasi 16 *bin hue*, 4 *bin saturation*, dan 8 *bin value*, sehingga menghasilkan 512 fitur. Histogram HSV merepresentasikan sebaran warna pada citra dalam bentuk numerik. Fitur ini berperan untuk melengkapi BoF karena BoF lebih berfokus pada pola lokal, bentuk, dan tekstur, sedangkan HSV memberikan informasi warna yang berkaitan dengan perubahan visual tanaman cabai rawit.

Hasil deteksi *keypoint* menggunakan ORB menunjukkan bahwa citra contoh menghasilkan 175 *keypoint* dengan bentuk *descriptor* 175 × 32. Sementara itu, hasil pengujian pada seluruh *dataset* menunjukkan bahwa dari 2137 citra, tidak terdapat *descriptor* kosong atau 0,0% *descriptor* kosong. Rata-rata jumlah *keypoint* yang diperoleh adalah 217,46, dengan jumlah minimum 10 dan maksimum 272. Hasil ini menunjukkan bahwa ORB mampu menghasilkan fitur lokal pada seluruh citra, sehingga fitur tersebut dapat digunakan sebagai dasar pembentukan *visual vocabulary* pada metode BoF.

Dari sisi dimensi fitur, metode BoF menghasilkan 128 fitur, fitur HSV menghasilkan 512 fitur, dan fitur SPM menghasilkan 2688 fitur. Ketika ketiga fitur tersebut digabungkan, dimensi fitur menjadi 3328 fitur. Penggabungan fitur ini membuat representasi citra menjadi lebih lengkap karena sistem tidak hanya menggunakan fitur lokal dari BoF, tetapi juga mempertimbangkan informasi warna dari HSV dan sebaran spasial dari SPM. SPM berperan untuk mempertahankan informasi posisi fitur melalui pembagian citra ke dalam beberapa level *grid*, sehingga distribusi fitur pada area citra tetap dapat diperhitungkan.

Reduksi dimensi dilakukan menggunakan PCA pada fitur gabungan BoF + HSV + SPM. Berdasarkan hasil pengujian, fitur awal yang berjumlah 3328 fitur dapat direduksi menjadi 647 komponen. Pada penelitian ini digunakan 647 komponen karena jumlah tersebut sesuai dengan model akhir yang digunakan pada aplikasi Chilivest.

Proses PCA membantu meringkas fitur yang berdimensi besar agar lebih efisien sebelum masuk ke tahap klasifikasi menggunakan SVM.

Tahap akhir dari proses pengolahan citra adalah klasifikasi menggunakan SVM. Pada tahap ini, fitur hasil PCA digunakan sebagai *input* model SVM untuk menentukan kelas kesiapan panen tanaman cabai rawit. Kelas yang dihasilkan terdiri dari Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Dengan demikian, keputusan deteksi pada aplikasi Chilivest diperoleh dari hasil klasifikasi SVM terhadap fitur gabungan BoF + HSV + SPM yang telah direduksi menggunakan PCA.

Visualisasi klasifikasi SVM berdasarkan fitur PCA menunjukkan bahwa data citra dari beberapa kelas memiliki pola sebaran yang dapat dipisahkan oleh area keputusan SVM. Visualisasi tersebut digunakan sebagai ilustrasi dua dimensi untuk memudahkan pemahaman proses klasifikasi, sedangkan model utama tetap menggunakan fitur hasil PCA sebanyak 647 komponen. Hal ini menunjukkan bahwa SVM berperan sebagai pengambil keputusan akhir dalam sistem, sementara proporsi warna hijau, oranye, dan merah yang ditampilkan pada aplikasi digunakan sebagai informasi tambahan bagi pengguna.

Secara keseluruhan, hasil pengolahan citra dan ekstraksi fitur menunjukkan bahwa citra tanaman cabai rawit berhasil direpresentasikan ke dalam bentuk fitur numerik. BoF berperan dalam merepresentasikan pola lokal, bentuk, dan tekstur citra, HSV berperan dalam merepresentasikan informasi warna, sedangkan SPM berperan dalam mempertahankan informasi spasial. Gabungan fitur tersebut kemudian direduksi menggunakan PCA dan diklasifikasikan menggunakan SVM untuk menghasilkan kelas kesiapan panen. Dengan demikian, proses deteksi pada aplikasi Chilivest tidak hanya mengacu pada proporsi warna yang ditampilkan pada aplikasi, tetapi berdasarkan hasil klasifikasi dari fitur gabungan BoF + HSV + SPM.

4.3.3 Pembahasan Hasil Perbandingan Metode

Berdasarkan hasil pengujian pada subbab 4.2.3, perbandingan metode dilakukan terhadap tiga metode ekstraksi fitur, yaitu *BoF*, *BoF + HSV*, dan *BoF + HSV + SPM*. Pengujian dilakukan untuk mengetahui pengaruh penambahan fitur warna *HSV*

dan informasi spasial *SPM* terhadap performa klasifikasi kesiapan panen tanaman cabai rawit. Parameter evaluasi yang digunakan meliputi *accuracy*, *precision macro*, *recall macro*, *F1-score macro*, waktu *training*, dan waktu prediksi.

Hasil pengujian menunjukkan bahwa metode *BoF* memperoleh *accuracy* sebesar 38,69%, *precision macro* sebesar 37,57%, *recall macro* sebesar 37,57%, dan *F1-score macro* sebesar 37,55%. Nilai tersebut menunjukkan bahwa penggunaan fitur *BoF* saja belum cukup untuk menghasilkan klasifikasi yang baik. Hal ini dapat terjadi karena *BoF* lebih berfokus pada fitur lokal seperti titik penting, bentuk, dan tekstur, sedangkan indikator utama dalam penentuan kesiapan panen cabai rawit banyak dipengaruhi oleh perubahan warna buah. Oleh karena itu, metode *BoF* tanpa tambahan fitur warna belum mampu merepresentasikan karakteristik visual cabai rawit secara optimal.

Setelah fitur *BoF* digabungkan dengan fitur warna *HSV*, performa model meningkat cukup besar. Metode *BoF + HSV* memperoleh *accuracy* sebesar 78,55%, *precision macro* sebesar 77,88%, *recall macro* sebesar 76,33%, dan *F1-score macro* sebesar 76,84%. Peningkatan ini menunjukkan bahwa fitur *HSV* memiliki pengaruh penting dalam klasifikasi kesiapan panen cabai rawit. Hal tersebut sesuai dengan karakteristik objek penelitian, karena tingkat kesiapan panen cabai rawit dapat diamati dari perubahan warna buah, mulai dari hijau, oranye, hingga merah. Dengan adanya fitur *HSV*, sistem dapat menangkap informasi warna yang sebelumnya tidak direpresentasikan secara kuat oleh *BoF*.

Metode *BoF + HSV + SPM* memperoleh hasil terbaik dibandingkan dua metode lainnya, dengan *accuracy* sebesar 80,19%, *precision macro* sebesar 78,86%, *recall macro* sebesar 78,34%, dan *F1-score macro* sebesar 78,47%. Hasil ini menunjukkan bahwa penambahan *SPM* mampu memberikan peningkatan performa meskipun tidak sebesar peningkatan dari *BoF* ke *BoF + HSV*. *SPM* membantu sistem mempertahankan informasi spasial atau letak sebaran fitur pada citra. Dengan demikian, sistem tidak hanya memperhatikan fitur lokal dan warna, tetapi juga mempertimbangkan posisi atau sebaran warna dan tekstur pada bagian-bagian citra.

Dari sisi waktu komputasi, metode *BoF* memiliki waktu *training* dan waktu prediksi paling rendah, yaitu 0,3242 detik dan 0,1812 ms/Gambar. Namun, performa klasifikasinya paling rendah. Sebaliknya, metode *BoF + HSV + SPM* memiliki waktu *training* dan waktu prediksi paling tinggi, yaitu 1,0587 detik dan 0,6875 ms/Gambar, tetapi menghasilkan performa klasifikasi terbaik. Hal ini menunjukkan adanya konsekuensi dari penambahan fitur, yaitu meningkatnya dimensi fitur dan waktu komputasi. Meskipun demikian, waktu prediksi metode *BoF + HSV + SPM* masih tergolong rendah karena berada di bawah 1 ms/Gambar, sehingga masih layak digunakan dalam sistem aplikasi *Chilivest*.

Berdasarkan *confusion matrix* metode terbaik, model *BoF + HSV + SPM* mampu mengenali kelas Belum Siap Panen dan Siap Panen dengan cukup baik. Kedua kelas tersebut memiliki nilai *recall* yang tinggi, yaitu 0,89 untuk kelas Belum Siap Panen dan 0,86 untuk kelas Siap Panen. Hal ini menunjukkan bahwa model cukup mampu mengenali citra tanaman cabai rawit yang belum siap panen dan siap panen. Namun, kelas Sebagian Siap Panen memiliki nilai *recall* lebih rendah, yaitu 0,66. Kondisi ini menunjukkan bahwa sebagian data pada kelas tersebut masih salah diklasifikasikan ke kelas lain.

Rendahnya performa pada kelas Sebagian Siap Panen dapat terjadi karena kelas ini memiliki karakteristik visual yang berada di antara kelas Belum Siap Panen dan Siap Panen. Pada kelas sebagian siap panen, dalam satu citra dapat terdapat buah cabai yang masih hijau dan buah cabai yang mulai berubah menjadi oranye atau merah. Kondisi campuran tersebut membuat pola warna dan fitur visualnya lebih sulit dibedakan secara tegas oleh model.

Secara keseluruhan, hasil perbandingan metode menunjukkan bahwa kombinasi *BoF + HSV + SPM* merupakan metode terbaik dalam penelitian ini. Metode tersebut mampu menggabungkan informasi lokal, warna, dan spasial sehingga menghasilkan representasi fitur yang lebih lengkap. Dengan *accuracy* sebesar 80,19% dan waktu prediksi sebesar 0,6875 ms/Gambar, metode *BoF + HSV + SPM* dapat digunakan sebagai metode utama pada sistem deteksi kesiapan panen tanaman cabai

rawit berbasis aplikasi *Android Chilivest*. Hasil ini sesuai dengan data pengujian perbandingan metode yang ditampilkan pada subbab 4.2.3.

4.3.4 Pembahasan Fungsionalitas dan Respons Aplikasi *Android*

Berdasarkan hasil pengujian pada subbab 4.2.4, pengujian fungsionalitas aplikasi *Android* Chilivest dilakukan menggunakan metode *black box*. Pengujian ini bertujuan untuk memastikan setiap fitur pada aplikasi dapat berjalan sesuai rancangan berdasarkan aksi yang dilakukan oleh pengguna. Fitur yang diuji dibagi menjadi dua bagian, yaitu fitur pada menu Petani dan fitur pada menu Peneliti.

Pada menu Petani, seluruh fitur utama memperoleh status berhasil. Fitur tersebut meliputi tampilan awal aplikasi, pemilihan jenis pengguna, *onboarding* Petani, pengambilan citra melalui kamera, pemilihan citra melalui galeri, proses prediksi, tampilan hasil prediksi, riwayat prediksi, hapus riwayat, menu akun, dan *logout*. Hasil ini menunjukkan bahwa alur penggunaan aplikasi untuk Petani telah berjalan dengan baik, mulai dari memasukkan citra tanaman cabai rawit hingga menampilkan hasil prediksi dan menyimpan riwayat.

Pada menu Peneliti, fitur *login*, menu Perbandingan Metode, grafik evaluasi, kartu metode, detail metode, Hasil Segmentasi, Riwayat Segmentasi, detail riwayat segmentasi, hapus riwayat segmentasi, dan *logout* juga berhasil dijalankan. Menu Perbandingan Metode mampu menampilkan informasi evaluasi metode BoF, BoF + HSV, dan BoF + HSV + SPM sesuai rancangan. Sementara itu, menu Hasil Segmentasi mampu menerima citra dari kamera atau galeri, mengirim citra ke *backend*, serta menampilkan hasil prediksi, citra *input resize*, *mask* kandidat cabai, *overlay* segmentasi, proporsi warna HSV, alasan, dan rekomendasi.

Secara keseluruhan, hasil pengujian menunjukkan bahwa aplikasi Chilivest telah berjalan sesuai rancangan. Seluruh fitur utama pada menu Petani dan Peneliti memperoleh status berhasil, serta respons aplikasi sudah sesuai dengan aksi pengguna. Dengan demikian, aplikasi Chilivest dapat dinyatakan berhasil dari sisi fungsionalitas dan respons aplikasi *Android*.

4.3.5 Pembahasan Deteksi Menggunakan Citra Eksternal

Berdasarkan hasil pengujian pada subbab 4.2.5, pengujian deteksi menggunakan citra eksternal dilakukan untuk mengetahui respons sistem Chilivest terhadap Gambar baru yang berasal dari luar *dataset* pengujian perbandingan metode. Citra eksternal yang digunakan merupakan Gambar langsung yang dimasukkan melalui aplikasi, kemudian diproses oleh *backend* untuk menghasilkan kelas prediksi kesiapan panen dan nilai *confidence*. Pengujian ini menjadi data pendukung untuk melihat kemampuan sistem dalam menerima, memproses, dan menampilkan hasil prediksi pada citra yang memiliki karakteristik visual lebih beragam.

Hasil pada Tabel 4.18 menunjukkan bahwa sistem Chilivest mampu memproses seluruh Gambar uji dan menampilkan hasil prediksi pada aplikasi Android. Hasil prediksi yang ditampilkan meliputi status keberhasilan, kelas prediksi, dan nilai *confidence*. Nilai *confidence* yang dihasilkan berada pada rentang sekitar 67% sampai 70%, sehingga menunjukkan bahwa model tetap memberikan tingkat keyakinan terhadap kelas prediksi yang dihasilkan pada citra eksternal.

Pada beberapa citra eksternal, hasil prediksi yang diperoleh sesuai dengan kondisi visual citra sehingga diberi status berhasil. Sementara itu, terdapat pula beberapa hasil prediksi yang berbeda dari kondisi visual citra. Perbedaan hasil tersebut dapat dipengaruhi oleh karakteristik citra eksternal, seperti pencahayaan, sudut pengambilan Gambar, jarak kamera, latar belakang, serta komposisi warna daun dan buah cabai dalam Gambar. Karena citra eksternal tidak berasal dari data uji pada pengujian perbandingan metode, variasi visual yang muncul dapat menjadi lebih beragam.

Berdasarkan Tabel 4.18, kelas prediksi yang muncul pada pengujian citra eksternal meliputi Belum Berbuah**,** Belum Siap Panen**,** dan Sebagian Siap Panen. Hal ini menunjukkan bahwa sistem dapat memberikan respons klasifikasi terhadap beberapa kondisi kesiapan panen pada Gambar baru. Selain itu, pengujian ini juga menunjukkan bahwa aplikasi Chilivest dapat menjalankan alur deteksi secara utuh, mulai dari memasukkan citra, mengirim citra ke *backend*, menerima *response*, hingga menampilkan hasil prediksi kepada pengguna.

Secara keseluruhan, pengujian deteksi menggunakan citra eksternal menunjukkan bahwa sistem Chilivest dapat digunakan untuk memproses Gambar baru di luar *dataset* pengujian perbandingan metode. Hasil pengujian ini memperkuat bahwa aplikasi Android, *backend* Flask, dan model klasifikasi telah terintegrasi dengan baik dalam menghasilkan prediksi kesiapan panen tanaman cabai rawit. Dengan demikian, pengujian citra eksternal dapat digunakan sebagai pendukung evaluasi penerapan sistem pada kondisi penggunaan yang lebih beragam.

4.3.6 Pembahasan Komunikasi *Client-Server*

Berdasarkan hasil pengujian komunikasi *client-server*, aplikasi *Android* Chilivest telah berhasil berkomunikasi dengan *backend* Flask yang berjalan secara *online* pada Railway. Komunikasi dilakukan melalui jaringan internet dengan aplikasi *Android* sebagai *client* dan *backend* Flask sebagai *server*. Pada alur ini, aplikasi mengirimkan citra tanaman cabai rawit ke *server* melalui *endpoint* `/predict` untuk proses prediksi utama dan *endpoint* `/predict-segmentation` untuk proses prediksi yang dilengkapi dengan hasil segmentasi. Selanjutnya, *server* memproses citra dan mengirimkan kembali *response* dalam format JSON.

Pada alur pengguna Petani, komunikasi *client-server* berjalan sesuai rancangan. Aplikasi berhasil mengambil atau memilih citra dari kamera maupun galeri, kemudian mengirimkan citra tersebut ke *backend* Flask melalui *endpoint* `/predict`. Setelah citra diterima, *server* melakukan proses pengolahan citra, ekstraksi fitur, reduksi dimensi, dan klasifikasi. Hasil prediksi yang dikirimkan kembali oleh *server* dapat diterima oleh aplikasi dan ditampilkan dalam bentuk kelas prediksi, nilai *confidence*, persentase warna, alasan prediksi, dan rekomendasi.

Pada alur pengguna Peneliti, komunikasi juga berjalan sesuai rancangan. Aplikasi berhasil mengambil data evaluasi metode dari *server* melalui *endpoint* `/method-comparison`, kemudian data tersebut ditampilkan pada menu Perbandingan Metode dan halaman Detail Metode. Selain itu, fitur Hasil Segmentasi pada menu Peneliti juga berhasil mengirim citra ke *backend* melalui *endpoint* `/predict-segmentation`. *Response* yang diterima meliputi hasil prediksi, proporsi warna HSV

seluruh citra, proporsi warna HSV area kandidat cabai, citra *input resize*, *mask* kandidat cabai, *overlay* segmentasi, alasan, dan rekomendasi.

Hasil pengujian QoS menunjukkan bahwa koneksi antara *client* dan *server* memiliki nilai *packet loss* sebesar 0%. Dari total 200 paket yang dikirim, seluruh paket berhasil diterima kembali tanpa adanya paket yang hilang. Kondisi ini menunjukkan bahwa koneksi jaringan yang digunakan pada saat pengujian cukup stabil dan tidak mengalami kehilangan paket selama proses komunikasi berlangsung.

Selain itu, hasil pengujian *delay* menunjukkan bahwa seluruh skenario pengujian berhasil dijalankan oleh dua pengguna. *Response time* tertinggi terdapat pada skenario unggah Gambar dan proses prediksi, yaitu sekitar 299 ms sampai 305 ms. Pada fitur segmentasi, skenario unggah Gambar dan proses segmentasi menghasilkan *response time* sekitar 90 ms, sedangkan skenario hapus riwayat segmentasi memiliki *response time* yang lebih rendah karena proses yang dilakukan lebih sederhana. Perbedaan waktu tersebut dipengaruhi oleh jenis proses yang dilakukan, seperti pengiriman citra, pemrosesan pada *server*, penerimaan *response*, dan penampilan hasil pada aplikasi.

Secara keseluruhan, hasil pengujian menunjukkan bahwa komunikasi *client-server* pada sistem Chilivest telah berjalan dengan baik. Aplikasi *Android* mampu mengirim *request* ke *backend* Flask, *server* mampu memproses data yang diterima, dan *response* dari *server* dapat ditampilkan kembali pada aplikasi. Dengan *packet loss* sebesar 0% dan *response time* yang masih tergolong rendah, komunikasi antara aplikasi *Android* dan *backend* Flask dapat dinyatakan berhasil serta layak digunakan dalam sistem deteksi kesiapan panen tanaman cabai rawit.

4.3.7 Pembahasan Hasil *Deployment Backend*

Berdasarkan hasil pengujian *deployment backend*, *backend Flask* pada sistem *Chilivest* telah berhasil dijalankan secara *online* menggunakan *Railway*. Proses *deployment* dilakukan dengan menghubungkan *repository GitHub backend Chilivest* ke *Railway*, sehingga *source code*, file model, *dependency*, dan konfigurasi *deployment* dapat dijalankan pada *server*.

Hasil pengujian menunjukkan bahwa *service backend* berada dalam kondisi Active dan proses *deployment* dinyatakan *Deployment* successful. Tahapan *deployment* seperti *initialization*, *build*, *deploy*, dan *post-deploy* berhasil berjalan tanpa error. Hal ini menunjukkan bahwa *Railway* berhasil membaca *repository*, menginstal *dependency*, dan menjalankan *backend Flask* sebagai *server* aplikasi.

Backend dijalankan menggunakan command *gunicorn* app:app dan memiliki URL publik *web-production-075a1.up.railway.app.* URL tersebut digunakan oleh aplikasi *Android Chilivest* untuk mengakses *backend* melalui *REST API*. Dengan demikian, *deployment backend* dapat dinyatakan berhasil karena *backend* telah berjalan secara *online* dan siap digunakan untuk menerima *request*, memproses citra, serta mengirimkan hasil prediksi ke aplikasi *Chilivest*.

4.3.8 Pembahasan Hasil *Endpoint REST API*

Berdasarkan hasil pengujian *endpoint* REST API, *backend* Flask pada sistem Chilivest telah berhasil memberikan *response* sesuai fungsi masing-masing *endpoint*. Pengujian dilakukan menggunakan Postman pada *endpoint* `/health`, `/predict`, `/method-comparison`, dan `/predict-segmentation`. Seluruh *endpoint* tersebut memperoleh *status response* 200 OK, sehingga menunjukkan bahwa *backend* dapat diakses dan mampu mengembalikan data dalam format JSON.

Endpoint `/health` berhasil menampilkan status *backend*, status model, dan konfigurasi *pipeline*. *Endpoint* `/predict` berhasil menerima citra tanaman cabai rawit melalui *form-data* dengan *key* `image`, kemudian mengembalikan hasil prediksi, nilai *confidence*, persentase warna, alasan prediksi, dan rekomendasi. Sementara itu, *endpoint* `/method-comparison` berhasil mengirimkan data evaluasi metode BoF, BoF + HSV, dan BoF + HSV + SPM yang digunakan pada menu Peneliti.

Selain itu, *endpoint* `/predict-segmentation` berhasil digunakan untuk mendukung fitur Hasil Segmentasi pada menu Peneliti. *Endpoint* ini menerima citra tanaman cabai rawit melalui *form-data* dengan *key* `image`, kemudian mengembalikan hasil prediksi, proporsi warna HSV seluruh citra, proporsi warna HSV area kandidat cabai, citra *input resize*, *mask* kandidat cabai, *overlay* segmentasi, alasan, dan

rekomendasi. Hasil segmentasi tersebut hanya digunakan sebagai informasi tambahan dan tidak menjadi penentu kelas prediksi SVM.

Secara keseluruhan, hasil pengujian REST API menunjukkan bahwa *backend* Flask Chilivest telah mampu menerima *request* dan mengirimkan *response* sesuai kebutuhan aplikasi *Android*. Dengan demikian, REST API pada sistem Chilivest dapat dinyatakan berhasil karena telah mendukung proses pengecekan *server*, prediksi citra, pengambilan data evaluasi metode, serta pengujian segmentasi pada menu Peneliti.

BAB V
PENUTUP

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem deteksi kesiapan panen tanaman cabai rawit menggunakan kombinasi metode *Bag-of-Features* (*BoF*), *HSV*, *Spatial Pyramid Matching* (*SPM*), *PCA*, dan *Support Vector Machine* (*SVM*), maka diperoleh beberapa kesimpulan sebagai berikut:

1. Sistem deteksi kesiapan panen tanaman cabai rawit berhasil dirancang dan diimplementasikan dalam bentuk aplikasi *Android* bernama *Chilivest*. Aplikasi ini memiliki dua jenis pengguna, yaitu petani dan peneliti. Menu petani digunakan untuk mengambil atau memilih citra tanaman cabai rawit, mengirimkan citra ke *backend*, menampilkan hasil prediksi, serta menyimpan riwayat prediksi. Sementara itu, menu peneliti digunakan untuk menampilkan hasil evaluasi dan perbandingan metode. Berdasarkan hasil pengujian fungsionalitas, seluruh fitur utama aplikasi dapat berjalan sesuai dengan rancangan.
2. Metode *Bag-of-Features* (*BoF*) berhasil diterapkan untuk mengekstraksi fitur bentuk dan tekstur dari citra tanaman cabai rawit. Pada penelitian ini, *BoF* menggunakan *ORB* untuk mendeteksi *keypoint* dan menghasilkan *descriptor*, kemudian *descriptor* dikelompokkan menggunakan *K-Means Clustering* untuk membentuk *visual vocabulary*. Fitur *BoF* menghasilkan 128 fitur yang merepresentasikan pola lokal pada citra. Namun, hasil pengujian menunjukkan bahwa penggunaan *BoF* saja masih belum optimal, karena memperoleh *accuracy* sebesar 38,69%.
3. Penambahan fitur warna *HSV* dan informasi spasial *SPM* berhasil meningkatkan performa klasifikasi dibandingkan penggunaan *BoF* saja. Fitur *HSV* menghasilkan 512 fitur untuk merepresentasikan distribusi warna, sedangkan *SPM* menghasilkan 2688 fitur untuk mempertahankan informasi

spasial citra. Gabungan fitur *BoF + HSV + SPM* menghasilkan total 3328 fitur, kemudian direduksi menggunakan *PCA* menjadi 647 komponen. Hasil pengujian menunjukkan bahwa metode *BoF + HSV* memperoleh *accuracy* sebesar 78,55%, sedangkan metode *BoF + HSV + SPM* memperoleh *accuracy* tertinggi sebesar 80,19%.

4. Algoritma *Support Vector Machine* (*SVM*) berhasil digunakan untuk mengklasifikasikan citra tanaman cabai rawit ke dalam empat kelas, yaitu Belum Berbuah, Belum Siap Panen, Sebagian Siap Panen, dan Siap Panen. Metode terbaik adalah *BoF + HSV + SPM* dengan *accuracy* sebesar 80,19%, *precision macro* sebesar 78,86%, *recall macro* sebesar 78,34%, dan *F1-score macro* sebesar 78,47%. Selain itu, sistem *Chilivest* berhasil diintegrasikan dengan *backend Flask* melalui *REST API*, *backend* berhasil *dideploy* secara *online* menggunakan *Railway*, *endpoint* utama memperoleh *response* 200 OK, dan komunikasi *client-server* memperoleh *packet loss* sebesar 0%. Dengan demikian, sistem yang dikembangkan dapat digunakan sebagai prototipe aplikasi deteksi kesiapan panen tanaman cabai rawit berbasis citra.

5.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat digunakan untuk pengembangan penelitian selanjutnya, yaitu sebagai berikut:

1. *Dataset* citra tanaman cabai rawit sebaiknya ditambah dengan variasi yang lebih beragam, seperti perbedaan kondisi pencahayaan, jarak pengambilan Gambar, sudut kamera, latar belakang, dan kondisi tanaman. Penambahan variasi *dataset* diharapkan dapat meningkatkan kemampuan model dalam mengenali citra tanaman cabai rawit pada kondisi nyata di lapangan.
2. Metode klasifikasi dapat dikembangkan dengan membandingkan algoritma lain, seperti *Random Forest*, *K-Nearest Neighbor*, atau metode *deep learning* seperti *CNN* dan *MobileNet*. Perbandingan tersebut dapat digunakan untuk mengetahui metode yang lebih optimal dalam mendeteksi kesiapan panen tanaman cabai rawit berbasis citra.

3. Sistem aplikasi *Chilivest* dapat dikembangkan dengan menambahkan penyimpanan berbasis database *online*. Pada penelitian ini, riwayat prediksi masih disimpan secara lokal pada perangkat *Android*, sehingga data hanya dapat diakses dari perangkat yang sama. Dengan database *online*, hasil prediksi dapat disimpan secara terpusat dan dapat diakses kembali dari perangkat lain.
4. Pengujian sistem sebaiknya dilakukan secara langsung pada lingkungan perkebunan atau lahan tanaman cabai rawit. Pengujian lapangan diperlukan untuk mengetahui kinerja aplikasi terhadap kondisi nyata, seperti perubahan cahaya, gangguan latar belakang, kualitas kamera, serta variasi bentuk dan warna cabai pada tanaman.
5. *Backend* yang telah *dideploy* secara *online* dapat dikembangkan lebih lanjut dengan fitur monitoring *server*, pencatatan log *request*, dan optimasi *response time*. Hal ini diperlukan agar sistem lebih stabil ketika digunakan oleh banyak pengguna secara bersamaan.

DAFTAR PUSTAKA

[1] W. S. Sijabat, M. Syukur, A. W. Ritonga, M. Ridha, and A. Istiqlal, "Performa Komponen Hasil dan Karakter Agronomi berbagai Genotipe Galur Cabai Rawit Performance of Yield Components and Agronomic Characteristics of Various Genotypes of Cayenne Pepper Lines," *Agrotechnology Research Journal*, vol. 7, no. 2, pp. 110–118, 2023, doi: 10.20961/agrotechresj.v7i2.79804.

[2] M. Rizzo, M. Marcuzzo, A. Zangari, A. Gasparetto, and A. Albarelli, "Arti fi cial Intelligence in Agriculture Fruit ripeness classi fi cation : A survey," *KeAi Chinese Roots Global Impact*, vol. 7, pp. 44–57, 2023, doi: 10.1016/j.aiia.2023.02.004.

[3]G. Moreira, S. A. Magalhães, T. Pinho, F. N. Dos Santos, and M. Cunha, "*Benchmark* of *Deep Learning* and a Proposed *HSV* Colour Space Models for the Detection and Classification of Greenhouse Tomato," *MDPI (Multidisciplinary Digital Publishing Institute)*, vol. 12, no. 2, 2022, doi: 10.3390/agronomy12020356.

[4]Y. Zhang, J. Cui, Z. Wang, J. Kang, and Y. Min, "Leaf *image* recognition based on *bag of features*," *MDPI (Multidisciplinary Digital Publishing Institute)*, vol. 10, no. 15, 2020, doi: 10.3390/app10155177.

[5] M. R. Saputra and H. Irsyad, "Klasifikasi Tingkat Kemanisan Alpukat Berdasarkan Fitur Hue Saturation Value (*HSV*) dengan Menggunakan *Support Vector Machine* (*SVM*)," *Jurnal Algoritme*, vol. 2, no. 2, pp. 113–119, 2022, doi: 10.35957/algoritme.v2i2.2361.

[6]Y. Yang, D. Zhu, F. Ren, and C. Cheng, "A NOVEL SELF-TAUGHT LEARNING *FRAMEWORK* USING *SPATIAL PYRAMID MATCHING* for *SCENE CLASSIFICATION*," *International Archives of the Photogrammetry*, vol. 43, no. B2, pp. 725–729, 2020, doi: 10.5194/isprs-archives-XLIII-B2-2020-725-2020.

[7] J. Siswantoro, “Enhanced *Support Vector Machine* Based on *Grey Wolf Optimizer* for Fruits *Image* Classification using *MPEG-7* Color and Texture Features Fusion,” *International Journal of Intelligent Engineering and Systems*, vol. 17, no. 3, pp. 1–11, 2024, doi: 10.22266/ijies2024.0630.01.

[8] N. Astrianda, “Vocational education and technolgy journal,” *Akademi Komunitas Negeri Aceh Barat*, vol. 1, no. 2, 2020.

[9] Y. Amrozi, D. Yuliati, A. Susilo, N. Novianto, and R. Ramadhan, “Klasifikasi Jenis Buah Pisang Berdasarkan Citra Warna dengan Metode *SVM*,” *Jurnal SISFOKOM (Sistem Informasi dan Komputer)*, vol. 11, pp. 394–399, 2022.

[10] M. Muchtar *et al.*, “PERBANDINGAN METODE *KNN* DAN *SVM* DALAM KLASIFIKASI KEMATANGAN BUAH MANGGA BERDASARKAN CITRA *HSV* DAN FITUR STATISTIK,” *JITET (Jurnal Informatika dan Teknik Elektro Terapan)*, vol. 12, no. 2, pp. 876–884, 2024.

[11] B. M. Alfaruq, D. Erwanto, and I. Yanuartanti, “Klasifikasi Kematangan Buah Tomat Dengan Metode *Support Vector Machine*,” *Generation Journal*, vol. 7, no. 3, pp. 93–101, 2023.

[12] V. Vinoharan and A. Ramanan, “An Efficient *Bag-of-Feature* Representation for Object Classification,” *Electronic Letters on Computer Vision and Image Analysis*, vol. 20, no. 2, pp. 51–68, 2021, doi: 10.5565/rev/elcvia.1403.

[13] B. Gajjar, H. Mewada, and A. Patani, “*Sparse coded spatial pyramid matching* and *multi-kernel* integrated *SVM* for *non-linear scene classification*,” *Journal of Electrical Engineering*, vol. 72, no. 6, pp. 374–380, 2021, doi: 10.2478/jee-2021-0053.

[14]L. Mohammed and Y. Yusoff, “DETECTION AND CLASSIFICATION OF PLANT LEAF DISEASES USING DIGTAL *IMAGE PROCESSING* METHODS : A REVIEW,” *ASEAN Engineering Journal*, vol. 1, pp. 1–9, 2023.

[15] B. Gajjar, H. Mewada, and A. Patani, "*Sparse coded spatial pyramid matching* and *multi-kernel* integrated *SVM* for *non-linear scene classification*," *Journal of Electrical Engineering*, vol. 72, no. 6, pp. 374–380, 2021, doi: 10.2478/jee-2021-0053.

[16] F. L. Gewers, S. Paulo, and S. Paulo, "Principal Component Analysis : A Natural Approach to," *ACM (Association for Computing Machinery)*, vol. 54, no. 4, 2021.

[17] B. Rasti *et al.*, "*Feature Extraction* for Hyperspectral Imagery : The Evolution from Shallow to Deep (Overview and Toolbox)," pp. 1–23, 2020.

[18] F. Solihin, M. Syarief, E. M. S. Rochman, and A. Rachmad, "Comparison of *Support Vector Machine* (*SVM*), *K-Nearest Neighbor* (*K-NN*), and Stochastic Gradient Descent (*SGD*) for Classifying Corn Leaf Disease based on *Histogram* of Oriented Gradients (*HOG*) *Feature Extraction*," *Elinvo (Electronics, Informatics, and Vocational Education)*, vol. 8, no. 1, pp. 121–129, 2023, doi: 10.21831/elinvo.v8i1.55759.

[19] L. Nanni, S. Ghidoni, and S. Brahnam, "Deep features for *training support* vector machines," *MDPI (Multidisciplinary Digital Publishing Institute)*, vol. 7, no. 9, pp. 1–13, 2021, doi: 10.3390/jimaging7090177.

[20] Sunardi, A. Yudhana, and M. Fahmi, "*SVM-CNN* Hybrid Classification for Waste *Image* Using Morphology and *HSV* Color Model *Image Processing*," 2023. doi: 10.18280/ts.400446.

[21] R. A. Saputra, S. Wasyianti, A. Supriyatna, and D. F. Saefudin, "Penerapan Algoritma *Convolutional Neural Network* Dan Arsitektur *MobileNet* Pada Aplikasi Deteksi Penyakit Daun Padi," *Jurnal SWABUMI*, vol. 9, no. 2, 2021.

[22] T. Aristan and G. P. Kusuma, "Revue d ' Intelligence Artificielle Evaluation of *CNN* Models in Identifying Plant Diseases on a *Mobile* Device," *IIETA*

(International Information and Engineering Technology Association), vol. 37, no. 2, pp. 441–449, 2023.

[23] E. Suherlan, S. Arti, S. Nabilah, Z. Hazimah, F. Teknik, and U. N. Jakarta, "PENERAPAN *FLASK FRAMEWORK* UNTUK *DEPLOYMENT* MODEL *MACHINE LEARNING* DALAM MENDUKUNG ANALISIS ADAPTASI Available at :," *Jurnal PINTER*, vol. 9, no. 1, 2025.

[24] A. Pudoli and I. Suwandi, "Implementasi Web *Service* Restful Dengan Autentikasi *JSon* Web Token Berbasis Web dan *Android*," *AJCSR (Academic Journal of Computer Science Research)*, vol. 5, no. 2, pp. 95–103, 2023.

[25] H. S. Jelita Ritonga, Suendri, "Jurnal Sistem Informasi Bisnis (JUNSIBI)," *Jurnal Sistem Informasi Bisnis (JUNSIBI)*, vol. 6, no. 1, pp. 99–111, 2025.

LAMPIRAN

Lampiran 1. Hasil Script Aplikasi

1. Logo Aplikasi

```
<application
android:icon="@drawable/ic_app_icon"
android:label="@string/app_name"
android:roundIcon="@drawable/ic_app_icon"
android:theme="@style/Theme.ChiliHarvest"
```

2. *Splash Screen*

```
class SplashActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_splash)

        Handler(Looper.getMainLooper()).postDelayed({
            navigateNext()
        }, SPLASH_DELAY_MS)
    }

companion object {
        private const val SPLASH_DELAY_MS = 900L
    }
}
```

3. Jenis Pengguna

```
class RoleSelectionActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_role_selection)
```

```
        findViewById<MaterialCardView>(R.id.farmerCard).setOnClickListener {
            startActivity(Intent(this, FarmerOnboardingActivity::class.java))
        }

        findViewById<MaterialCardView>(R.id.researcherCard).setOnClickListener {
            startActivity(Intent(this, ResearcherLoginActivity::class.java))
        }
    }
}
```

4. *Onboarding Login* Petani

```
private val pages = listOf(
    OnboardingPage("Deteksi Instan Kesiapan Panen", R.drawable.ic_onboarding_disease),
    OnboardingPage("Prediksi Tanaman Cabai Rawit", R.drawable.ic_onboarding_product),
    OnboardingPage("Riwayat Prediksi yang Mendukung",
R.drawable.ic_onboarding_group)
)

nextButton.setOnClickListener {
    val nextPage = viewPager.currentItem + 1
    if (nextPage < pages.size) {
        viewPager.currentItem = nextPage
    } else {
        enterFarmerArea()
    }
}

findViewById<TextView>(R.id.skipText).setOnClickListener {
    enterFarmerArea()
}
```

5. Menu Tanaman Anda

```
buttonCamera.setOnClickListener {
    requestCameraOrOpen()
}

buttonGallery.setOnClickListener {
    pickImageLauncher.launch("image/*")
}

seeAllButton.setOnClickListener {
    startActivity(HistoryListActivity.createIntent(requireContext()))
}
```

6. *Loading* Proses Prediksi

```
private fun showLoading() {
    progress.isVisible = true
    retryButton.isVisible = false
    closeButton.isVisible = false
    titleText.text = "Menganalisis Gambar tanaman"
    messageText.text = "Sistem sedang menjalankan preprocessing, ekstraksi fitur, dan klasifikasi model."
}
```

7. Hasil Prediksi

```
private fun bindResult(history: PredictionHistoryEntity) {
    findViewById<TextView>(R.id.resultDateText).text = DateUtil.format(history.createdAt)
    findViewById<TextView>(R.id.predictionText).text = history.prediction
    findViewById<TextView>(R.id.confidenceText).text =
        "Confidence: %.1f%%".format(history.confidence)

    findViewById<TextView>(R.id.greenText).text =
        "Hijau : %.2f%%".format(history.greenPercentage)
    findViewById<TextView>(R.id.orangeText).text =
```

```
        "Oranye : %.2f%%".format(history.orangePercentage)
    findViewById<TextView>(R.id.redText).text =
        "Merah : %.2f%%".format(history.redPercentage)

    findViewById<TextView>(R.id.plantStatusText).text =
        history.plantStatus.ifBlank { history.prediction.uppercase() }
    findViewById<TextView>(R.id.reasonsText).text = formatReasons(history)
    findViewById<TextView>(R.id.recommendationText).text =
        "Rekomendasi:\n${history.recommendation.ifBlank { history.description }}"
```

8. Riwayat Prediksi

- Riwayat Prediksi

```
val historyAdapter = HistoryAdapter(
    compact = false,
    onItemClick = { history ->
        startActivity(PredictionResultActivity.createIntent(this, history.id))
    },
    onDeleteClick = { history ->
        showDeleteConfirmation(history)
    }
)

findViewById<RecyclerView>(R.id.recyclerHistory).apply {
    layoutManager = LinearLayoutManager(this@HistoryListActivity)
    adapter = historyAdapter
}

viewModel.allHistory.observe(this) { history ->
    historyAdapter.submitList(history)
    emptyText.isVisible = history.isEmpty()
}
```

- Hapus Riwayat

```
private fun showDeleteConfirmation(history: PredictionHistoryEntity) {
    AlertDialog.Builder(this)
        .setTitle("Hapus riwayat?")
        .setMessage("Riwayat prediksi ${history.prediction} akan dihapus dari daftar riwayat.")
        .setNegativeButton("Batal", null)
        .setPositiveButton("Hapus") { _, _ ->
            viewModel.deleteHistory(history)
            Toast.makeText(this, "Riwayat berhasil dihapus", Toast.LENGTH_SHORT).show()
        }
        .show()
}
```

9. Menu Anda

```
class ProfileFragment : Fragment(R.layout.fragment_profile) {
    private val viewModel: ProfileViewModel by viewModels {
        ProfileViewModel.Factory(
            (requireActivity().application as ChiliHarvestApplication).authRepository
        )
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val session = viewModel.getSession()
        view.findViewById<TextView>(R.id.profileNameText).text = session.name
        view.findViewById<TextView>(R.id.profileEmailText).text = session.email
        view.findViewById<TextView>(R.id.versionText).text =
            "Versi aplikasi ${BuildConfig.VERSION_NAME}"

        view.findViewById<MaterialButton>(R.id.logoutButton).setOnClickListener {
```

```
            viewModel.logout()
            val intent = Intent(requireContext(), RoleSelectionActivity::class.java).apply {
                flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK
            }
            startActivity(intent)
            requireActivity().finish()
        }
    }
}
```

10. *Login* Peneliti

```
loginButton.setOnClickListener {
    emailInputLayout.error = null
    passwordInputLayout.error = null
    viewModel.login(
        email = emailEditText.text?.toString().orEmpty(),
        password = passwordEditText.text?.toString().orEmpty()
    )
}

viewModel.loginState.observe(this) { state ->
    when (state) {
        ResearcherLoginState.Idle -> Unit
        ResearcherLoginState.Success -> navigateToResearcherArea()
        is ResearcherLoginState.Error -> {
            passwordInputLayout.error = state.message
            Snackbar.make(loginButton, state.message, Snackbar.LENGTH_LONG).show()
        }
    }
}
```

11. Menu Perbandingan Metode

```
viewModel.methods.observe(viewLifecycleOwner) { methods ->
    methods.maxByOrNull { it.accuracy }?.let { bestMethod ->
        bindDashboard(view, bestMethod)
    }
    setupChart(chart, methods)
    setupCards(cardContainer, methods)
}

viewModel.sourceLabel.observe(viewLifecycleOwner) { sourceLabel ->
    view.findViewById<TextView>(R.id.methodDataSourceText).text = sourceLabel
}
private fun bindDashboard(view: View, bestMethod: MethodComparison) {
    view.findViewById<TextView>(R.id.bestMethodText).text =
        "Metode terbaik: ${bestMethod.methodName}"
    view.findViewById<TextView>(R.id.bestAccuracyText).text =
        "Accuracy terbaik: %.2f%%".format(bestMethod.accuracy)
    view.findViewById<TextView>(R.id.bestF1Text).text =
        "F1-score terbaik: %.2f%%".format(bestMethod.f1Macro)
    view.findViewById<TextView>(R.id.bestDimensionText).text =
        "Dimensi fitur: ${bestMethod.featureDimension}"
    view.findViewById<TextView>(R.id.bestPredictionTimeText).text =
        "Waktu prediksi: %.2f ms/Gambar".format(bestMethod.predictionTimeMs)
}
```

12. Detail Metode

```
private fun bindMethod(method: MethodComparison) {
    findViewById<TextView>(R.id.detailMethodNameText).text = method.methodName

    findViewById<LinearLayout>(R.id.generalInfoContainer).apply {
        removeAllViews()
        addSectionTitle("Informasi Umum")
```

```
        addMetric("Accuracy", "%.2f%%".format(method.accuracy))
        addMetric("Precision macro", "%.2f%%".format(method.precisionMacro))
        addMetric("Recall macro", "%.2f%%".format(method.recallMacro))
        addMetric("F1 macro", "%.2f%%".format(method.f1Macro))
        addMetric("PCA komponen", method.pcaComponents)
        addMetric("Waktu prediksi", "%.2f ms".format(method.predictionTimeMs))
    }

    findViewById<LinearLayout>(R.id.perClassContainer).apply {
        removeAllViews()
        addSectionTitle("Per class")
        addPerClassTable(method.perClass)
    }

    findViewById<LinearLayout>(R.id.confusionMatrixContainer).apply {
        removeAllViews()
        addSectionTitle("Confusion matrix")
        addConfusionMatrix(method.confusionMatrix)
    }
```

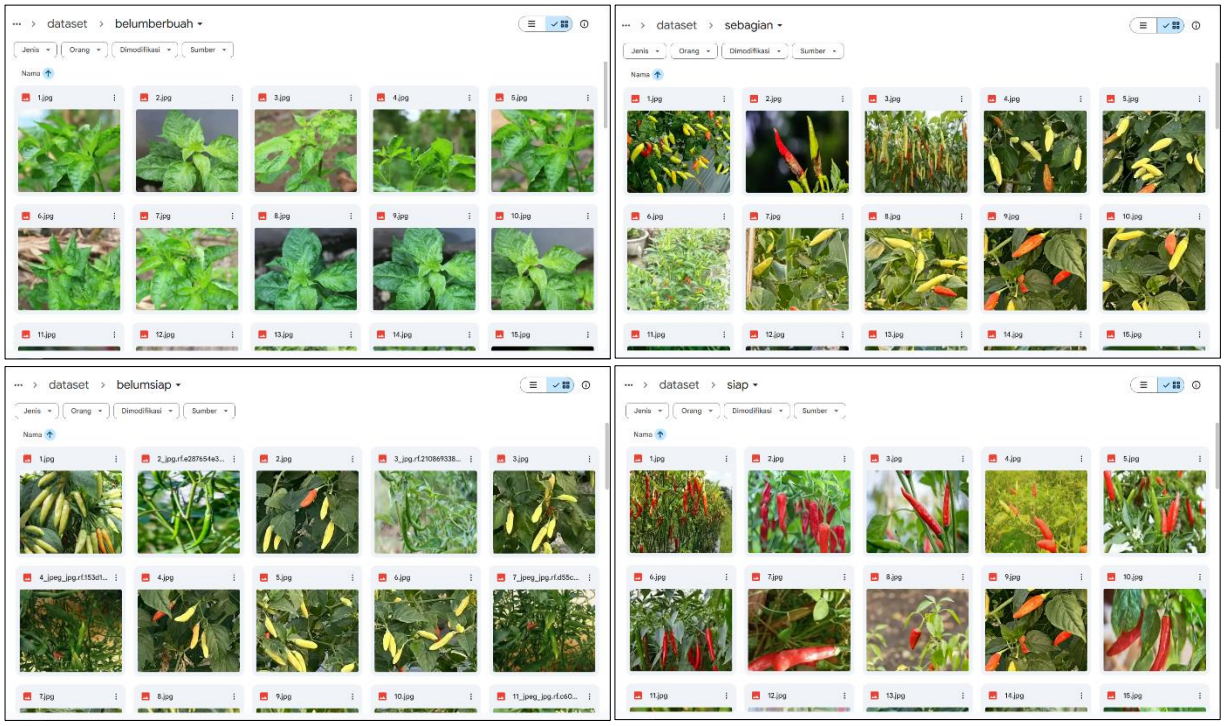
13. Repository Full Program



Lampiran 2. Dataset Tersimpan di Google Drive

Link Google Drive:

https://drive.google.com/drive/folders/1_nlA7kAePm7L7JtBPAeI5rhESgGGfZG-?usp=sharing



Lampiran 3. Dokumentasi Pengujian Aplikasi



Lampiran 4. Daftar Riwayat Hidup

DAFTAR RIWAYAT HIDUP



A. Data Pribadi

Data Diri	: Muhammad Nur Asy'ari
Tempat, Tanggal Lahir	: Malang, 27 April 2004
Jenis Kelamin	: Laki-laki
Agama	: Islam
Program Studi	: D-IV Jaringan Telekomunikasi Digital
Jurusan	: Teknik Elektro
Perguruan Tinggi	: Politeknik Negeri Malang
Alamat	: Jl. Sarangan No. 71 Karang Pandan, Kec. Pakisaji, Kab. Malang
No. Telepon	: 08983607177
E-mail	: nurasyari27@gmail.com

B. Riwayat Pendidikan

2010-2016	: SD Plus Qurrota A'yun
2016-2019	: MTs Negeri 6 Malang
2019-2022	: SMK Negeri 4 Malang
2022-2026	: Politeknik Negeri Malang